



OPEN UNIVERSITY OF CATALONIA (UOC) MASTER'S DEGREE IN DATA SCIENCE

MASTER'S THESIS

AREA 2: NATURAL LANGUAGE PROCESSING AND VISUAL ANALYTICS

AI-Assisted ESG Company Profiling

A Retrieval-Augmented Generation (RAG) Approach for Data-Driven Corporate Analysis

Author: Juan Aguirre Calderón

Tutor: Josep-Anton Mir Tutusaus

Professor: Josep-Anton Mir Tutusaus

Barcelona, May 24, 2025

Credits/Copyright

This project is carried out within the framework of the Master's in Data Science at the Universitat Oberta de Catalunya (UOC). Unless otherwise stated, the code and documentation within this work are licensed under the following terms:



Attribution-NonCommercial-NoDerivs 3.0 Spain (CC BY-NC-ND 3.0 ES)

3.0 Spain of Creative Commons.

FINAL PROJECT RECORD

Title of the project:	AI-Assisted ESG Company Profiling
Author's name:	Juan Aguirre Calderón
Collaborating teacher's name:	Josep-Anton Mir Tutusaus
PRA's name:	Josep-Anton Mir Tutusaus
Delivery date (mm/yyyy):	06/2025
Degree or program:	Master's in Data Science (UOC)
Final Project area:	Area 2
Language of the project:	English
Keywords	ESG (Environmental, Social, Governance) Company Profile Retrieval-Augmented Generation (RAG) Large Language Models (LLMs)

Dedication

I dedicate this thesis to my family and all those who supported me on this challenging but rewarding journey. Your constant support and belief in me have helped turn my dreams into reality.

A special thanks to mamá, papá, Juan, Jacqueline, Matthias, Mario, Sebastian, and Melanie.

Abstract

Abstract (English)

This Master’s thesis addresses the challenge of creating comprehensive fact-based ESG (Environmental, Social, and Governance) company profiles, a procedure obstructed by the manual synthesis of diverse, often unstructured, data, and the limitations of standard Large Language Models (LLMs) in ensuring accuracy and traceability. The project proposes and implements a Retrieval-Augmented Generation (RAG) framework designed to automate the initial drafting of ESG profiles while maintaining high factual integrity and source attribution.

The implemented methodology involves the ingestion of external documents (such as sustainability and research reports) and proprietary internal ESG metrics. External documents undergo pre-processing, including robust parsing and chunking, before being vectorized and stored in a knowledge base. The core RAG pipeline dynamically retrieves relevant document pages based on the specific ESG topic being analyzed, incorporates formatted internal data, and uses sophisticated prompt engineering to guide a generative LLM. A crucial post-generation verification layer uses a separate LLM to automatically assess factual accuracy and citation correctness against sources.

Evaluation of the implemented system has shown significant efficiency gains, reducing the time required for analysts to produce a final ESG company profile by approximately 80% compared to traditional manual methods. The automated verification layer has successfully identified inconsistencies and errors, highlighting its value for output reliability. Analysts feedback has confirmed the utility of the drafts and has praised their integrated traceability features (citations, access to sources via the user interface) to facilitate the validation. This architecture is applicable for evidence-based synthesis from heterogeneous data sources.

Keywords: ESG (Environmental, Social, Governance) Company Profile, Retrieval-Augmented Generation (RAG), Large Language Models (LLMs).

Abstract (Spanish)

Este trabajo de fin de máster aborda el desafío de crear perfiles ESG (ambiental, social y gobernanza) de empresas detallados y basados en hechos, un procedimiento dificultado por la síntesis manual de datos heterogéneos, a menudo no estructurados, y las limitaciones de las LLMs estándar para garantizar la precisión y la trazabilidad. El proyecto propone e implementa un marco de RAG diseñado para automatizar la redacción inicial de perfiles ESG manteniendo una alta integridad factual y atribución de las fuentes.

La metodología implementada implica la incorporación de documentos externos (como informes de sostenibilidad e investigación) y métricas ESG internas propietarias. Los documentos externos pasan por un preprocesamiento, incluyendo parsing robusto y división en fragmentos, antes de ser vectorizados y almacenados en una base de conocimiento. El proceso RAG principal obtiene dinámicamente las páginas relevantes de los documentos añadidos, basándose en el tema ESG específico analizado, incorpora los datos internos sobre la empresa en un formato adecuado, y utiliza ingeniería de prompts sofisticada para guiar al LLM generativo. Posteriormente, una capa crucial de verificación post-generación utiliza un LLM separado para evaluar automáticamente la precisión factual y la corrección de las citas en comparación con las fuentes.

La evaluación del sistema implementado ha demostrado ganancias significativas en eficiencia, reduciendo el tiempo requerido para que los analistas produzcan un perfil ESG final de una empresa en aproximadamente un 80% en comparación con los métodos manuales tradicionales. La capa de verificación automatizada ha conseguido identificar exitosamente inconsistencias y errores, destacando su valor para la fiabilidad del resultado. El feedback de los analistas ha confirmado la utilidad del borrador generado por IA y ha elogiado las características de trazabilidad integradas (citas, acceso a fuentes a través de la interfaz de usuario) por facilitar la validación. Esta arquitectura es aplicable para la síntesis basada en evidencia a partir de fuentes de datos heterogéneas.

Palabras clave: Perfil ESG (Ambiental, Social y Gobernanza) de empresa, Generación Aumentada de Recuperación (RAG), Grandes Modelos de Lenguaje (LLMs).

Contents

Abstract	vii
Table of Contents	ix
List of Figures	xiii
List of Tables	xv
1 Introduction	1
1.1 Background	1
1.1.1 Context ESG	1
1.1.2 Context Large Language Models	8
1.2 Problem Statement and Objectives	13
1.2.1 Problem Statement	13
1.2.2 Motivation	13
1.2.3 Objective	14
2 State of the Art	15
2.1 Large Language Models	15
2.1.1 Core LLM Developments	15
2.1.2 Scaling Laws and Efficiency	16
2.1.3 Accessibility and Cost Considerations	18
2.2 Prompt Engineering	19
2.2.1 Key Techniques	19
2.2.2 Error mitigation	20
2.3 Document Attachment and Context Window	21
2.3.1 Context Window	21
2.3.2 Parsing and Feeding Documents	22
2.3.3 Limitations of Large-Context Models	23

2.4	Hallucinations	24
2.4.1	Why LLMs Hallucinate and Why It Matters	24
2.4.2	Strategies to Mitigate Hallucinations	25
2.5	Retrieval-Augmented Generation (RAG)	26
2.5.1	Concept and Motivations	26
2.5.2	Representative RAG Architectures and Tools	27
2.5.3	Types of RAG	31
2.5.4	Evaluation of LLMs for RAG	32
2.5.5	RAG vs. Long-Context Models	33
3	Methodology	37
3.1	Introduction and Architectural Overview	37
3.2	Data Sources and Ingestion	38
3.2.1	Overview of Analyst Interaction and Input Data	38
3.2.2	External Document Corpus	39
3.2.3	Internal ESG Data	39
3.3	Document Preprocessing and Knowledge Base Construction	40
3.3.1	PDF Conversion and Structural Parsing	40
3.3.2	Text Chunking Strategy	41
3.3.3	Vectorization and Storage	41
3.4	Retrieval-Augmented Generation Pipeline	42
3.4.1	Prompt Engineering Strategy	42
3.4.2	Information Retrieval	43
3.4.3	LLM-Based Profile Generation	44
3.5	Post-Generation Correction and Verification Layer	44
3.5.1	Automated Verification using a Dedicated LLM Call	44
3.5.2	Citation Validation	45
3.5.3	Re-generation Loop	45
3.5.4	Structured Output Format	45
3.5.5	Analyst Review and Feedback Mechanism	46
3.6	Evaluation Strategy Outlook	46
4	Implementation	49
4.1	System Architecture	49
4.2	Technology Stack	51
4.3	Implementation of Core Components	52
4.3.1	Data Repository and Loading	52

4.3.2	Preprocessing Pipeline	52
4.3.3	RAG Pipeline Implementation	54
4.3.4	Post-Generation Verification Layer Implementation	55
4.3.5	Structured Output and Saving	56
4.4	User Interface Implementation	56
4.5	Infrastructure and Deployment Considerations	60
4.6	Key Implementation Challenges and Solutions	61
5	Results Analysis	63
5.1	Introduction	63
5.2	Evaluation Methodology	63
5.3	Quantitative Results: Efficiency and Accuracy	64
5.3.1	Efficiency Gains	64
5.3.2	Accuracy and Verification Metrics	66
5.3.3	Comparative Analysis and Discussion of Results	67
5.4	Qualitative Analysis: Output Quality and Usability	69
5.5	Summary of Results	70
6	Conclusion and Future Work	71
6.1	Conclusion	71
6.2	Future Work	72
6.2.1	Infrastructure and Model Enhancements	72
6.2.2	Preprocessing Pipeline Refinements	72
6.2.3	RAG Pipeline Advancements	72
6.2.4	Verification and Quality Improvements	73
6.2.5	Output Enhancements and Application Development	73
6.2.6	Continuous Learning and Feedback Loop	73
	Bibliography	75
	Appendix A: Supplementary Details for State of the Art	89

List of Figures

1.1	The Transformer architecture underlying most LLMs [1].	9
3.1	Workflow Diagram of RAG System.	38
4.1	Implemented System Architecture Diagram.	49
4.2	Main page of the Streamlit UI showing company selection, source document management, and pipeline configuration options.	57
4.3	Document Repository view in the Streamlit UI.	58
4.4	Internal Data page.	58
4.5	ESG Profile Analysis page.	59

List of Tables

5.1	Pipeline Execution Times by Company and PDF Parser	65
5.2	Automated Verification Metrics Summary by Company and PDF Parser	66

1 Introduction

1.1 Background

1.1.1 Context ESG

Environmental, Social and Governance (ESG) refers to a set of criteria used to evaluate a company's performance beyond traditional financial metrics. Environmental factors include the impact and stewardship of the natural environment of a company (e.g., carbon emissions, resource usage); Social factors assess how a company manages relationships with employees, customers, and communities (e.g., labor practices, product safety); and Governance factors consider corporate leadership, ethics, audits, internal controls, and shareholder rights (e.g., board structure, transparency). These ESG aspects have gained prominence as key indicators of long-term corporate sustainability, value, and risk management.

In fact, evaluating ESG factors provides analysts with a broader perspective on potential risks and opportunities beyond what financial statements alone reveal. Mismanagement of environmental or social issues, for example, an industrial accident or a labor scandal, has led to high-profile corporate crises that negatively impacted share prices. In contrast, strong ESG practices can indicate prudent management and resilience. Thus, ESG analysis, sometimes called extrafinancial analysis, complements financial analysis by offering a holistic picture of corporate performance. It enhances confidence in investment decisions by revealing how well a company is positioned in a changing world of resource constraints, evolving regulations, and societal expectations [2].

Today, ESG analysis is considered an essential component of investment decision making, and regulators and investors are increasingly viewing ESG integration as "no longer optional" but rather integral to prudent portfolio management [3]. A growing body of research supports the relevance of ESG factors: numerous studies have found that companies with strong ESG performance often exhibit equal or better financial performance compared to peers [4, 5, 6]. However, the link between sustainability and financial success is complex rather than deterministic. In summary, ESG analysis provides critical context on factors that can affect a firm's

risk profile, value, and environmental or societal impact, complementing traditional financial analysis.

Sustainable Investments

The field of sustainable investments has evolved substantially over the past few decades. Its origins were mainly based on negative screening, an exclusionary approach rooted in ethical or moral considerations. Investors employing negative screening deliberately avoided certain companies or industries deemed objectionable, early examples included divestment from South African apartheid-linked businesses in the 1980s or from tobacco companies in the 1990s [7]. While negative screening remains in use (e.g., many funds still exclude weapons, gambling, or fossil fuel companies), it has fundamental limitations, such as reducing the investable universe and forgoing engagement with excluded firms.

Over time, sustainable investments broadened into more proactive and sophisticated strategies beyond simple exclusion. A key approach today is ESG integration, the explicit incorporation of ESG factors into financial analysis and investment decisions. Rather than merely avoiding certain stocks, ESG integration involves evaluating how ESG risks and opportunities can impact company performance (e.g., how a firm's carbon footprint could affect future regulatory costs or how workforce satisfaction could influence productivity) and adjusting the valuations or portfolio weights accordingly. This approach aims to improve risk-adjusted returns by including material ESG information in the investment process, aligning with the idea that considering sustainability is part of fiduciary duty [7]. In fact, by 2020, the most common sustainable investment strategy globally was ESG integration, surpassing exclusionary screening [8].

Another expanding category is impact investing. Impact investments are characterized by an intentional focus on generating measurable positive social or environmental outcomes alongside financial returns. In practice, this might involve investing in projects such as renewable energy infrastructure, social enterprises, or development finance, where capital is directed to solutions to social challenges. It is important to mention that most impact investments still target the market rate or competitive returns, distinguishing them from pure philanthropy [7]. What sets impact investing apart is the commitment to report and quantify the actual impact achieved. Impact or solution-oriented funds have grown as investors seek to "do well by doing good", channeling capital to areas such as clean technology, affordable housing or healthcare innovation.

A pillar in the evolution of sustainable investment is active ownership (also known as engagement or stewardship). This strategy involves investors using their rights as shareholders to influence corporate behavior on ESG matters. Active ownership can take the form of exercising proxy voting power in line with ESG principles and, perhaps more critically, direct dialogue

with company management or boards to urge improvements in sustainability practices. For example, large asset managers may pressure companies to set climate targets, improve diversity and inclusion, or strengthen governance structures. Engagement is typically a long-term process that aims to encourage companies to address material ESG issues, potentially reducing risks and improving value for shareholders and stakeholders alike. Investment firms are increasingly emphasizing the power of engagement in driving sustainability outcomes, and stewardship codes in various jurisdictions have formalized expectations for investors to actively supervise and engage with investee companies. Active ownership thus complements ESG integration: while integration informs which companies to invest in (and at what weight), engagement focuses on how those companies operate after investment. [9, 10]

Collectively, these strategies, negative screening, integration of ESG, impact investing, and active ownership, often operate in tandem within a sustainable investment framework. Many contemporary sustainable funds employ a combination (for instance, integrating ESG factors into analysis, excluding a few worst-outs, and engaging with holdings). As sustainable investing has become mainstream, assets under management in ESG-aligned strategies have surged during the last decade. This remarkable growth reflects the broadening of investor interest and the normalization of ESG considerations in investment processes around the world. [11]

It should be noted that the path of ESG investment is not uniform across regions. In Europe, regulatory support and stakeholder pressure for sustainability remain robust. The European Union has implemented sweeping sustainable finance regulations to encourage ESG integration and transparency, and European investors generally continue to increase their allocation to sustainable assets. In contrast, the United States has seen a more polarized and shifting landscape. Enthusiasm for ESG investments grew in the 2010s, but a political backlash has emerged in recent years. In particular, since the return of President Trump to office in 2025, there has been a marked increase in skepticism and pushback against ESG in the US market. This is evidenced by a wave of state-level actions: in 2023 alone, more than 150 anti-ESG bills were proposed in 37 states, and 40 anti-ESG laws were enacted in 18 states [12], reflecting efforts to restrict the consideration of ESG factors by public pension plans and other entities. The new administration's deregulatory stance has further cooled federal support for ESG initiatives. Despite these headwinds in the US, global investor interest in sustainable investments continues and many institutions view ESG issues as financially material challenges that must be addressed.

Materiality

A foundational concept in effective ESG analysis is materiality, which refers to the significance of specific ESG issues to a company's financial performance and long-term value, given the company's industry and operating context. Not all ESG factors are equally relevant for every

business; what is material (i.e. likely to impact economic outcomes or be important to stakeholders' decisions) varies by sector and circumstances. For example, climate change and carbon emissions are highly material concerns for an energy company (where future carbon regulations or shifts to renewables directly affect profitability), whereas for a software company, carbon emissions may be relatively minor in impact. In contrast, data privacy and cybersecurity could be a material ESG issue for a tech firm but less so for an oil producer. In other words, each industry has a different ESG risk/opportunity profile. This principle is well recognized by investors and ESG standard-setters: the materiality of ESG issues is industry-specific. Identifying the key ESG factors that materially affect a given business is crucial for a meaningful analysis. [13, 14, 7].

Furthermore, it is essential to consider double materiality, a concept increasingly recognized in ESG analysis [2]. Double materiality broadens the perspective by assessing two dimensions:

1. How ESG issues affect the company's financial performance (financial materiality), and
2. how the company's activities impact society and the environment (environmental/social materiality).

For example, a company's carbon emissions pose a financial risk if regulations impose costs on carbon (financial materiality), and simultaneously these emissions contribute to climate change (impact on the environment). Double materiality acknowledges that stakeholders (investors, regulators, communities) care about both.

In summary, materiality is about prioritization: effective ESG analysis focuses on environmental, social, and governance factors that truly matter for the creation and management of value of an enterprise, which differ between industries and contexts. Keeping materiality front and center ensures that ESG evaluations are decision-useful and economically relevant, rather than a check-the-box exercise on an arbitrary list of issues.

ESG Rating Systems and Methodologies

As ESG investments gained traction, a number of specialized ESG rating agencies and data providers emerged to evaluate companies' sustainability performance. Leading ESG rating providers include firms like MSCI ESG Ratings, Sustainalytics (Morningstar), S&P Global (formerly RobecoSAM for Dow Jones Sustainability Index), Moody's ESG (via V.E), Fitch (via Sustainable Fitch), ISS ESG, and others, as well as smaller specialists and non-profits (e.g., CDP for carbon disclosure, JUST Capital, etc.). Each of these agencies employs their own methodologies to assess companies on a range of ESG criteria and assign scores or ratings (typically numeric scores or letter grades) to reflect overall ESG quality or risk. Common

inputs to ESG ratings include a mix of quantitative metrics (e.g., emissions levels, employee turnover rates, board diversity percentages) and qualitative assessments (e.g., controversies, policies, programs in place). However, methodologies vary widely across providers, leading to divergent ratings for the same company. Each rater makes different judgments about which factors to include (scope), how to measure those factors (metrics), and how to weight them in an overall score. A study by MIT Sloan highlighted this "aggregate confusion", finding that the correlation among major ESG ratings to be around 0.54 on average. In contrast, credit rating agencies (Moody's, S&P) typically agree much more closely (correlation 0.9) [15]. For example, one rating system might heavily weight climate-related metrics, while another focuses more on governance; one may treat controversial incidents as a heavy penalty, while another emphasizes forward-looking commitments. There is also a lack of standardization in data collection, agencies rely on sustainability reports from companies, news sources, questionnaires, and estimates, which can result in inconsistent datasets. This lack of agreement has fueled the debate about the usefulness of aggregated ESG scores.

In contrast, sophisticated investors increasingly recognize the limitations of one-size-fits-all ESG ratings and bypass aggregated scores altogether, using the raw data and underlying metrics from providers to conduct their own analysis. Rather than relying on a single summary score (which might obscure important complexities), large asset managers often ingest data feeds from multiple ESG data sources, covering carbon emissions, board composition, employee satisfaction scores, etc., and then apply proprietary models to evaluate which companies are outperforming on the ESG metrics they deem material. This custom scoring can yield a more tailored view of ESG performance aligned with the investor's strategy or values.

Challenges in ESG Analysis

Key challenges continue to confront ESG analysis even as it becomes more sophisticated. One fundamental challenge is data quality and consistency. The quality of ESG data remains an important concern, because, as mentioned before, a large portion of ESG information is voluntarily disclosed by companies in sustainability reports or questionnaires, without the strict auditing standards applied to financial data. The Economist Intelligence Unit observes that inconsistent standards across the industry "obstruct comparability" and allow companies to cherry-pick which data to highlight [16]. As a result, analysts struggle with collecting and verifying high-quality ESG data. Important metrics, for example, greenhouse gas emissions, might be reported by some firms and not by others, or calculated differently, making apples-to-apples comparison difficult. . Additionally, for many ESG topics (like biodiversity impact or community relations), measuring outcomes is inherently complex, and data can be more qualitative. The absence of universally accepted measurement methods introduces subjectivity

and potential bias into ESG assessments [17].

Another challenge is regulatory inconsistency across jurisdictions. Different countries or regions are moving at different paces in mandating ESG disclosures and standards. The EU, for example, has been proactive with a comprehensive sustainable finance agenda (including the Sustainable Finance Disclosure Regulation for investors and the Corporate Sustainability Reporting Directive for companies) [16], while other markets like the US have no overarching federal ESG mandate and even pockets of resistance. Some countries or states promote ESG consideration (or at least require disclosure of climate risks, for instance), while others actively discourage it (as seen in anti-ESG state laws in the United States). This inconsistency creates challenges for global investors who must navigate different standards, and a portfolio spanning Europe and the US faces uneven data availability and legal obligations. Likewise, companies operating internationally must comply with a mosaic of ESG reporting rules, which complicates the comparability of data. The regulatory divergence can also lead to arbitrage or "forum shopping" (e.g., companies choosing to list in jurisdictions with looser ESG requirements). Until there is greater convergence (perhaps through bodies like the IFRS's new International Sustainability Standards Board), ESG analysts must interpret the data in light of the regulatory context and be cautious when comparing firms across regions. [12]

Greenwashing and credibility issues are another concern in ESG analysis. As ESG becomes trendy, the practice of making misleading or overstated claims about sustainability has become a significant concern. Some companies (and investment funds) have been accused of painting an unjustifiably green picture of their activities to cater to investor demand. This damages trust and can lead to reputational and financial risks if claims are exposed. Surveys indicate that investors are increasingly cautious: for example, the PwC 2023 Global Investor Survey found 94% of investors expressed skepticism about the veracity of companies' sustainability reporting, suspecting that many claims are not adequately supported by evidence. High-profile incidents of greenwashing, such as funds being relabeled or corporations touting "net zero" pledges without concrete plans, have further fueled scrutiny. ESG analysts must therefore apply a critical eye to corporate disclosures and third-party labels, checking for inconsistencies or lack of alignment between words and actions. Ensuring credibility in ESG analysis requires vigilance against such practices and sometimes skepticism toward unverified claims [18, 19].

These challenges, among others, underscore that ESG analysis is not a straightforward task. It requires careful judgment in data interpretation, staying abreast of evolving laws, and a healthy dose of due diligence to validate sustainability claims. Overcoming these hurdles is an active area of development: initiatives are underway to improve ESG data quality (including moves toward mandatory audited sustainability reports in some jurisdictions), to harmonize standards, and to establish clearer definitions to prevent greenwashing. However,

for the time being, ESG analysts and investors must navigate a landscape that is still maturing, with inherent uncertainties and the need for continuous critical evaluation of ESG information.

Role of ESG Analysts

In this complex context, ESG analysts play a crucial role in the investment industry by bridging the gap between raw sustainability information and actionable investment insight. One of the core responsibilities of the ESG analyst is to research and evaluate companies according to relevant environmental, social, and governance criteria, ultimately forming an independent judgment of the ESG performance of a company and its implications for financial risk / return (key responsibilities of the ESG analyst are: research, data analysis, evaluation, stewardship, integration and advisory). This involves a combination of quantitative and qualitative analysis: ESG analysts examine over sustainability reports, regulatory filings, news articles, expert reports, and datasets from ESG rating agencies or data providers. They identify which ESG issues are most material to each company (for example, carbon emissions for an airline, or data privacy for a bank) and assess how the company is managing those issues. Key tasks include analyzing policies and targets (does the company have credible climate goals? diverse leadership? robust labor standards?), performance metrics (trend in emissions, employee turnover, product safety incidents, etc.), and any controversies or incidents (environmental spills, fraud allegations, labor strikes, etc.). ESG analysts often engage directly with company management through questionnaires or meetings to clarify outstanding issues and encourage better disclosure. They then synthesize all this information into ESG research reports and scores or ratings that feed into the investment process, for example, flagging companies with excessive ESG risk or highlighting sustainability leaders with potential competitive advantages.

Crucially, ESG analysts must exercise a significant judgment. Unlike purely quantitative financial metrics, many ESG factors require context and interpretation. For example, an ESG analyst must determine whether ambitious sustainability promises of a company are backed by substance or amount to greenwashing, or how a new regulatory requirement might impact the business model. They also need to contextualize their performance relative to peers and industry norms. This makes the role challenging, but also essential. ESG analysts provide the sophisticated understanding that raw data or third-party scores alone cannot offer. Indeed, many investment firms now incorporate ESG analysts within investment teams or have dedicated ESG research units, underlining the importance of their insights for portfolio construction and risk management.

Advances in technology are beginning to enhance the work of ESG analysts. The rise of big data and artificial intelligence (AI) has opened up new possibilities to tackle information overload in ESG research. Natural language processing and machine learning algorithms can

rapidly scan and analyze massive volumes of text, such as news feeds or corporate filings, to identify ESG-related signals (for example, detecting negative sentiment around a company's labor practices or highlighting any mention of a factory accident). Such AI-driven tools can help analysts avoid missing critical developments and update ESG evaluations in real-time as new information emerges. Likewise, specialized ESG data platforms now use AI to fill data gaps (e.g., estimating a company's emissions from satellite imagery when disclosures are missing) and to harmonize inconsistent data. These technologies promise to improve the efficiency and depth of ESG analysis. However, it is widely acknowledged that AI is a complement to, not a replacement for, human judgment in ESG. ESG issues often involve context, ethics, and stakeholder perspectives that algorithms alone cannot fully comprehend. In an ideal scenario AI could capture all quantifiable aspects of ESG, "but real-world considerations and dynamic contexts demand human oversight". [20]

In practice, AI can handle the heavy lifting of data processing and pattern recognition, whereas human analysts are needed to interpret the results, cross-check for plausibility, and make judgment calls on what is material or how to engage with a company on an issue. For example, if an AI model detects a spike in negative news about a company's supply chain, an ESG analyst would investigate the underlying issue (perhaps a report of child labor at a supplier) and assess its credibility and importance.

In addition, many aspects of the ESG analysis involve subjective and prospective evaluation (such as evaluating a company's culture or the sincerity of its commitment to improvement), which are not easily quantifiable. Human analysts bring contextual intelligence, skepticism, and an understanding of complexity that automated systems lack. They also serve as the link to portfolio managers, translating ESG findings into investment recommendations and engaging in dialogue about trade-offs. In summary, ESG analysts remain indispensable even as data technology progresses, their role is evolving to work alongside AI tools, using them to enhance insight while providing critical thinking and contextual awareness that ensure ESG analysis is meaningful. This synergy between human expertise and artificial intelligence is an emerging frontier. [21, 20, 22]

1.1.2 Context Large Language Models

Introduction

Over the past decade, natural language processing (NLP) has evolved at a remarkable pace. In the early 2010s, many NLP tasks, such as sentiment analysis, topic classification, and named entity recognition, relied on feature engineering and comparatively small datasets. Although these approaches brought measurable improvements in accuracy, they often struggled with

complexities in language and the multifaceted nature of real-world text. The landscape began to shift dramatically with the introduction of deep learning methods, leading to a wave of improvements in tasks like machine translation and question answering.

However, the most significant leap in NLP capabilities emerged with the introduction of the Transformer architecture in 2017 by Vaswani with the paper "Attention Is All You Need" [23] and the subsequent adoption of large-scale pretraining in massive text corpora [24, 25]. Using the self-attention mechanism, Transformers allow models to capture contextual relationships across entire sentences or paragraphs in parallel, overcoming the sequential bottlenecks of previous recurrent neural networks (RNNs). When combined with massive datasets (ranging from billions to trillions of tokens) and scaled to billions of parameters, these Transformer-based architectures gave rise to what we now call Large Language Models (LLMs)[26].

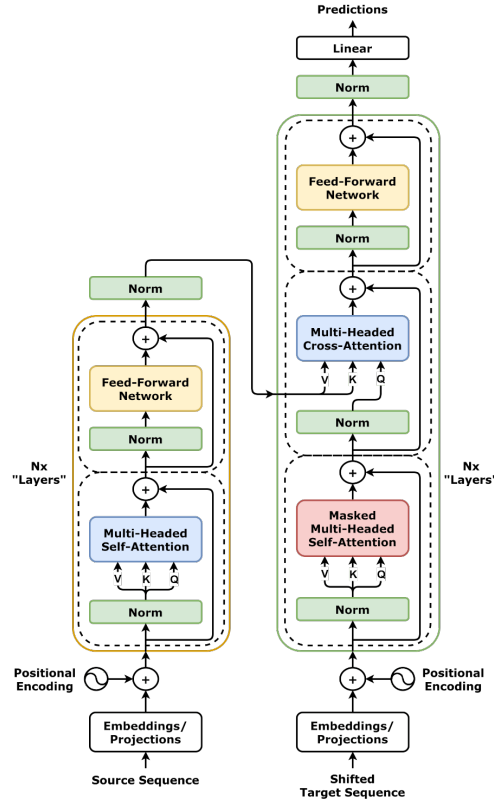


Figure 1.1: The Transformer architecture underlying most LLMs [1].

Figure 1.1 shows a standard Transformer architecture comprising an encoder (left) and a decoder (right), each stacking multi-headed attention and feed-forward layers to contextualize token representations [1]. During training, text is divided into subword tokens and learned via self-supervised objectives. GPT-style models, for example, are trained to predict the next token in a sequence, allowing them to generate coherent text and perform zero-shot task transfer. In particular, the GPT-2 model (1.5 billion parameters) achieved state-of-the-art results

on multiple benchmarks in a zero-shot setting, solely through large-scale generative pretraining [25]. These advancements established the two main paradigms for LLMs: encoder-based (exemplified by BERT, designed for language understanding) and decoder-based (exemplified by GPT, designed for language generation), both underpinned by the Transformer foundation.

Building on these Transformer-driven breakthroughs, the field of NLP evolved at a remarkable pace over the past five years, ultimately giving rise to LLMs—models scaled to billions of parameters and trained on massive text collections. Their ability to generalize from prompts, known as few-shot or zero-shot learning, represented a significant milestone [27]. Models such as Google’s T5 (2019) further unified NLP tasks by casting every problem into a text-to-text framework [28]. More recently, systems such as PaLM (2022) with 540 billion parameters and GPT-4 (2023) introduced emergent abilities that materialize only at scale [29]. Meanwhile, Meta AI’s LLaMA (2023) demonstrated that smaller architectures (7–65B) trained on high-quality data can still match or surpass larger counterparts, spurring numerous domain-specific variants [30].

In 2024–2025, a new wave of improvements further pushed the frontier. OpenAI GPT-4 and 4.5 introduced more intuitive conversational abilities, a massive context window (up to 128,000 tokens), and significantly reduced hallucination rates (factual error rate dropped from 62% in GPT-4 to 37%) [31]. Google’s Gemini (2024) broke new ground in understanding long contexts, reportedly handling up to 1–2 million tokens [32]. Google also launched NotebookLM, an AI research assistant that ingests documents and provides cited answers, while Anthropic’s Claude models now support context windows of 100K–200K tokens, and emerging systems like DeepSeek achieve state-of-the-art performance at lower computational costs [33]. Advances in training, such as reinforcement learning with human feedback (RLHF) and novel optimizations (e.g., OpenAI ”o1” and ”o3” models), and architectures like the advanced reasoning have further improved model efficiency and reliability [34, 35].

Applications of LLMs

LLMs excel at core NLP tasks and have unlocked new possibilities, and a key point is that many of these abilities can be attained without explicit task-specific training by prompting a sufficiently large model. Such flexibility makes LLMs a unified solution for a spectrum of language tasks that previously required separate specialized models, and it has led to their adoption in many industries. In healthcare, LLMs are used to analyze medical records, provide conversational responses to patient queries, and even assist in diagnostic reasoning by synthesizing medical literature. In the financial domain, LLMs power customer service chatbots and help parse financial reports or news to inform investment decisions. The legal industry employs LLMs to summarize legal documents, perform contract analysis, and conduct legal research

with natural language queries. In education, LLMs act as virtual tutors, answering student questions and generating explanations or quizzes. More generally, since the debut of ChatGPT in late 2022, LLM-powered systems have been integrated into customer support and expert advisory tools, literature search engines, and content creation pipelines. For example, organizations have used LLMs to draft parts of reports and communications, with human oversight, to increase productivity. The wide use of LLMs across these fields underscores their role as foundational AI tools capable of handling unstructured language data at scale [36].

Recent developments have led to the integration of LLMs in ESG analysis, enabling more efficient data extraction, monitoring, and compliance assessment. ESG rating providers such as MSCI leverage AI to extract sustainability-related data from diverse unstructured sources, including news articles, industry publications, and company reports. This improves both the timeliness and the objectivity of their assessments [37]. Similarly, Sustainalytics employs advanced NLP techniques for automated controversy monitoring, scanning thousands of media sources to detect potential ESG issues in real time [38]. These applications demonstrate how LLM-powered solutions can efficiently process and synthesize vast amounts of ESG data, uncovering trends and discrepancies that inform investment decisions.

Another emerging application is the automated compliance with regulatory regulations. LLMs can extract relevant mentions of activities or investments from reports, map them to regulatory taxonomies, and flag compliance gaps.

Lastly, it is relevant to mention their use in greenwashing detection, identifying when companies make misleading sustainability claims. Researchers have fine-tuned models such as ClimateBERT to identify potential greenwashing language in corporate reports and media, achieving approximately 86% accuracy in their preliminary study [39].

Challenges and Limitations of LLMs

Despite their impressive capabilities, LLMs face several challenges and limitations that are especially pertinent when applying them to ESG data and other high-stakes domains.

One of the most important is the hallucination rate. LLMs can produce hallucinations, which means text that is factually incorrect, non-sensical, or not supported by any source. In the context of ESG, a hallucinating model might invent a metric or claim about a company that is not in the input data. Studies categorize hallucinations as intrinsic (contradicting known information) or extrinsic (statements that sound plausible but cannot be verified). High-profile LLMs are known to sometimes state incorrect facts with great confidence. This is problematic in ESG analysis, where accuracy is crucial, a hallucinated statement about a company's emissions or controversies could mislead assessments. The risk is exacerbated by the complexity of ESG data, where the model might not fully understand the context and

fill gaps with its learned priors. Reducing hallucination remains an open challenge: even the latest LLM models will occasionally produce false information or cite non-existent reports. As discussed in the literature, LLMs may produce hallucinations due to the limits of their parametric knowledge, and additional methods are needed to ensure factuality [26] [40].

Another limitation is that LLMs normally do not inherently provide sources or citations for the statements they generate (Lack of traceability). The generation process is a black-box use of learned parameters, so when an LLM produces a claim it is usually unclear what training data or input led to that claim. This lack of traceability undermines trust, as users cannot easily verify the origin of the information. In addition, they typically only cite content from retrieval (external documents) and have no way to attribute facts that come from the internal knowledge of the model. In ESG analysis, this opaqueness is a serious concern. Without sources, generated output cannot be used in regulatory or assurance processes. The inability to explain or justify an output makes it hard to integrate LLMs into workflows that require accountability. In general, the "black box" nature of LLM output is a barrier to their use in compliance-focused applications [41].

The context window constraints can also be an important challenge; it is the maximum amount of text LLMs can process in a single query or prompt. Although original Transformer models were typically limited to a few thousand tokens of context, newer LLMs can handle orders of magnitude more. Anthropic's Claude 2 already offered a 100K token window in 2023, and its Claude-Opus model pushed this to an industry-first 200K tokens in 2024. In fact, Claude 3 models are reported to accept inputs up to 1 million tokens (for specialized use cases) with near-perfect recall on long documents [42]. Similarly, Google's Gemini 1.5 Pro introduced a 2 million token context window – the longest of any large model – allowing it to ingest very large collections of text or even book-length inputs [32]. OpenAI has also extended GPT-4's capabilities with variants like GPT-4 128k, enabling it to handle long conversations or documents (e.g., large legal contracts or research papers) without running out of context. While these advancements significantly reduce context-related limitations, access to such models remains a challenge due to availability and cost constraints. As a result, they may not fully resolve issues that arise when multiple documents must be combined for complex analyses, such as in ESG profiling. In these cases, alternative strategies like retrieval-augmented generation (RAG) remain essential [26] [40].

Furthermore, LLMs can exhibit biases emerging from their training data or design, raising ethical concerns in sensitive applications such as ESG assessment. Data biases in training corpora (which might over-represent certain viewpoints, geographies, or demographics) can lead to skewed outputs. For example, if an LLM's training data on corporate social responsibility comes mostly from Western large-cap companies, the model's "opinions" or summaries might

underrepresent perspectives relevant to emerging markets or small enterprises. Bias can also manifest in the form of stereotypes or unjustified associations present in the model’s answers. In ESG analysis, this might translate into unwarranted positive or negative prejudices. [43] [26] [44].

To address these issues, researchers and practitioners have developed several mitigation strategies. Some of the most important are RAG, which combines LLMs with external knowledge retrieval systems, grounding responses in up-to-date, verifiable content [34].

1.2 Problem Statement and Objectives

1.2.1 Problem Statement

As seen in the preceding section (1.1.1), in today’s investment landscape, ESG analysis has become increasingly essential for informed investing. Regulators and institutional investors now view ESG integration as a core component of prudent portfolio management, driving demand for accurate and timely ESG reports. However, the current ESG evaluation process remains labor-intensive: sustainability analysts must manually sift through fragmented, often multilingual data from sources such as PDFs, HTML reports, and CSV files, spending significant time on data aggregation and verification. The combination of complexity and heterogeneity of these sources further increases the risk of errors, inconsistencies, and incomplete coverage.

Recent advances in LLMs show promise for accelerating information extraction. Nevertheless, standard LLM solutions struggle with hallucinations, inconsistent output structure, and lack of source attribution. In high-stakes ESG contexts, where data must be auditable and grounded in verifiable evidence, such shortcomings make conventional LLM-based approaches unsuitable. Essentially, a more reliable system that can both automate the synthesis of ESG data and ensure factual correctness is in clear demand.

To address these gaps, this thesis proposes a RAG framework that anchors LLM output in trusted data sources, thereby minimizing hallucinations and offering complete source traceability. By focusing on factual accuracy, operational efficiency, and analyst-centric design, the project seeks to significantly reduce manual workloads and expand ESG coverage without compromising on quality.

1.2.2 Motivation

The impetus for this project comes from two converging factors: escalating costs and complexity in ESG analysis, and the transformative potential of AI-driven reporting.

First, as demand for ESG reports continues to rise, organizations face ballooning expenses in analyst hours and data licensing fees. Manual processes are not scalable, particularly for midsize financial institutions or multinational firms facing conflicting regional regulations. This intensifies the need for automating preliminary data synthesis without compromising on accuracy or compliance.

Second, the development of LLMs, and in particular RAG techniques, presents an opportunity to revolutionize ESG reporting. By anchoring model outputs in verifiable sources, RAG methods significantly reduce hallucinations and provide transparent source attribution. As a result, analysts can focus on interpretation and strategy rather than repetitive data collection and validation. This shift not only improves coverage but also enhances decision-making through more robust and timely ESG profiles.

Beyond the ESG domain, another key motivation is the scalability and adaptability of this approach, making it applicable to any scenario requiring the transformation of unstructured or semistructured data into reliable, evidence-based reports.

1.2.3 Objective

The primary objective of this thesis is to design and implement a production-grade RAG system capable of generating accurate, structured, and evidence-based ESG company profiles with minimal hallucinations and complete source attribution, reducing the manual effort required from sustainability analysts.

The secondary objectives are the following.

1. **Modularity:** Develop the solution with modular components (retrieval database, scoring models, templating), allowing independent upgrades or replacements.
2. **Efficiency and Accuracy:** Automate the initial data collection and drafting process to reduce ESG profile creation time while achieving high factual accuracy.
3. **Feedback Integration:** Incorporate an analyst feedback loop to continuously refine system outputs and collect data for future enhancements.

2 State of the Art

This chapter surveys the state-of-the-art in AI-driven techniques for ESG company profiling, with a particular emphasis on recent advances in LLMs and RAG. Given that ESG analysis involves the synthesis of a wide range of unstructured textual data, from corporate disclosures to news articles, these cutting-edge methods offer strong potential to enhance both the speed and accuracy in the creation of ESG company profiles.

Recent research studies underscore the promise of these models in producing coherent and on-demand analyzes, but also highlight challenges such as hallucinations, citation gaps, and the need for context-aware prompting [45, 46, 47, 48, 49]. Consequently, this chapter reviews the technical capabilities and limitations of modern LLMs, examines emerging strategies for grounding AI-generated output in verifiable sources (RAG), and assesses their implications for high-stakes ESG domains. By clarifying where these techniques succeed and where caution is warranted, we provide the technological foundation upon which our subsequent research and experiments will build.

2.1 Large Language Models

The landscape of LLMs has rapidly evolved, with 2024-2025 bringing new models that are more capable, efficient, and accessible than ever. Leading AI companies, including OpenAI, Anthropic, Google/DeepMind, Meta, Microsoft, Alibaba, and emerging players like DeepSeek and Mistral, have pushed the state-of-the-art in different directions. This analysis reviews the core developments of these LLMs and discusses their properties for future implementations.

2.1.1 Core LLM Developments

The period spanning late 2024 and early 2025 witnessed continued rapid advancements in LLMs, with major AI research laboratories and emerging players alike releasing significant updates or entirely new model families. These developments have consistently pushed the boundaries of LLM capabilities, often including both large-scale flagship models and more

efficient, specialized, or open-source alternatives [50, 51, 52, 53].

Key trends observed across these new releases include improved natural language understanding and generation, improved reasoning abilities, reduced hallucination rates, and expanded context windows, sometimes reaching hundreds of thousands or even millions of tokens. Multimodality (handling text, image, and audio) has become increasingly prevalent, and a notable divergence persists between powerful proprietary models offered via APIs (such as those from OpenAI, Anthropic, Google’s Gemini line, and xAI) and a growing ecosystem of high-performing open-source models (from entities like Meta, Google’s Gemma line, DeepSeek, Mistral AI, Alibaba, and Microsoft’s Phi series). These open source alternatives often prioritize efficiency, customization, and on-premise deployment, providing viable options for organizations with specific data privacy or infrastructure requirements. The choice of model for tasks such as ESG profiling, which demands accurate analysis of large text volumes, involves careful consideration of these evolving capabilities, access models, and cost implications.

A detailed overview of prominent LLM models released by key industry players, including their specific features, characteristics, and typical use cases as of early 2025, is provided in the Appendix 6.2.6. This appendix elaborates on the offerings from companies such as OpenAI, Anthropic, Google, Meta, DeepSeek, Mistral AI, Alibaba, Microsoft, and xAI.

2.1.2 Scaling Laws and Efficiency

The rapid advancement of LLMs has prompted a critical reassessment of traditional scaling laws in artificial intelligence. Early scaling laws, such as those articulated by Kaplan et al. (2020)[54], posited a straightforward relationship between model size, training data, and performance, suggesting that larger models trained on more data would consistently yield superior results. This principle motivated the dramatic expansion of the LLM scale. The Chinchilla law (DeepMind, 2022) [55]) introduced a significant refinement, highlighting the existence of an optimal balance between model size and training data for a given computational budget. However, contemporary research increasingly emphasizes the concept of efficient scaling, in order to maximize model capabilities while minimizing resource expenditure. By 2025, the landscape has evolved further as researchers have identified the practical limitations of brute-force scaling. Although the relationship between model performance and scale remains valid, the industry has shifted toward multidimensional approaches to efficiency rather than relying solely on parameter count increases, leading to the exploration of innovative techniques that challenge conventional scaling paradigms. [56, 57]

This modern understanding of LLM scaling encompasses multiple interacting dimensions beyond a mere parameter count. Key areas of innovation include:

- **Hyperparameter Scaling Laws:** Discovering universal relationships to optimize learning rates and batch sizes on model scales, such as the "Step Law" [56].
- **Advanced Training Methodologies:** Employing techniques like Reinforcement Learning from Human Feedback (RLHF), curriculum learning, and sophisticated post-training optimizations (fine tuning, instruction tuning) to enhance model capabilities without proportional size increases [58].
- **Data Quality and Quantity Optimization:** Recognizing the critical role of high-quality, extensive training corpora, with research suggesting that performance benefits can extend further than previously assumed with sufficient data quality [59, 56].
- **Architectural Efficiencies:** Developing novel architectures like Mixture-of-Experts (MoE) models [60, 61] and leveraging model distillation techniques [62, 63] to achieve superior performance-to-cost ratios or create compact yet capable models.
- **Precision Reduction:** Utilizing quantization techniques to reduce numeric precision (e.g., to 4-bit or 8-bit representations), thereby decreasing memory and computational demands while largely preserving model capabilities [64, 65].
- **Inference-Time Compute Scaling:** Dynamically allocating additional computation at inference time through methods like recurrent thought processes or self-consistency decoding, allowing models to "think harder" on demand [66, 59].

A more detailed exploration of each of these scaling dimensions is provided in the Appendix 6.2.6.

The understanding of LLM scaling in 2025 has evolved far beyond the one-dimensional focus on parameter count that characterized earlier research. Today's scaling paradigm encompasses multiple interacting dimensions, as described: parameter count, data volume and quality, hyperparameter optimization, architectural choices (including sparsity), precision, distillation strategies, or inference-time computation allocation.

The practical implications for LLM deployment are profound. Model selection now requires consideration of these multiple dimensions and their interactions, rather than simplistic comparisons of parameter counts. A well-designed 34B parameter model with optimized training data, efficient architecture, and smart inference-time strategies may significantly outperform a vanilla 100B model lacking these refinements.

For organizations constrained by computational budgets, these advances offer particularly valuable pathways to state-of-the-art capabilities. The multi-faceted nature of modern scaling approaches means that innovation can occur along multiple dimensions simultaneously, with

advances in one area often complementing progress in others. As raw parameter scaling faces increasing economic and environmental constraints, research into these efficiency-focused scaling dimensions will likely intensify, driving continued capability improvements despite plateauing model sizes.

This evolving understanding of scaling represents not just a technical advance but a philosophical shift in how artificial intelligence development is conceptualized, moving from brute-force approaches toward increasingly sophisticated optimization of existing resources.

2.1.3 Accessibility and Cost Considerations

The practical application of LLMs, particularly in resource-sensitive domains such as ESG profiling, necessitates careful consideration of deployment strategies. The fundamental choice lies between local execution (on personal computers, on-premises servers, or edge devices) and remote execution (via cloud-based APIs or services). Each approach presents distinct trade-offs regarding accessibility, cost, data privacy, scalability, and customization.

Local execution offers enhanced data privacy, potential long-term cost savings for high-volume use, full model customization, and offline functionality. However, it requires significant upfront hardware investment (especially GPUs with ample VRAM), entails increased maintenance overhead, and can present scalability challenges [67, 68]. A rich ecosystem of tools and frameworks has emerged to simplify local LLM deployment.

Remote execution, on the other hand, provides simplified infrastructure management, access to the most powerful state-of-the-art models, robust scalability, and lower upfront costs due to pay-as-you-go pricing. The drawbacks include variable usage-based costs that can escalate, data privacy concerns associated with external APIs, dependency on network connectivity, and limited direct control over model configurations. Leading cloud providers and specialized API vendors offer diverse platforms for remote LLM access.

More details on specific advantages, disadvantages, common tools (e.g., Hugging Face Transformers, Llama.cpp, Ollama, LangChain for local; Azure AI Studio, Google Vertex AI, AWS SageMaker/Bedrock, and direct API providers for remote) and AI-enhanced development platforms are discussed in Appendix 6.2.6.

Ultimately, the optimal approach to LLM accessibility and cost for ESG profiling is context dependent, requiring careful consideration of organizational priorities, technical capabilities, budget constraints, and data governance policies. By strategically evaluating trade-offs and leveraging the various tools and platforms available, organizations can effectively harness the power of LLM to enhance their ESG analysis capabilities in a responsible and sustainable manner.

2.2 Prompt Engineering

Prompt engineering, the craft of designing inputs to guide LLMs to produce the desired output, remains a critical skill, even as models become more capable. A well-designed prompt can transform a vague response into a precise and detailed answer by clearly defining tasks, providing context, and setting constraints. This is particularly important for pre-trained models not tailored to specific tasks, where prompts act as "prompt programming" to condition the model on the fly.

Given the diversity in model training, the optimal prompt may vary slightly, although the core principles remain stable. In particular, LLM developers such as OpenAI [69], Google [70], Anthropic [71], Llama [72], or Mistral AI [73] offer tailored recommendations, reflecting the evolving landscape.

The field of prompt engineering continues to evolve rapidly as new techniques emerge and existing approaches are refined. Based on recent comprehensive surveys and research studies, this section presents an updated view of key prompt engineering techniques and error minimization strategies used with LLMs.

2.2.1 Key Techniques

Key prompt engineering techniques, as surveyed in recent studies [74, 75], can be broadly classified into several categories. These approaches aim to elicit more accurate, relevant, and structured responses from LLMs by carefully crafting the input or "prompt." The primary categories include:

- **Zero-Shot Prompting:** Instructing an LLM to perform a task without providing explicit examples, relying on the model's pre-trained knowledge and the clarity of the instruction.
- **Few-Shot Prompting:** Providing the LLM with a small number of examples (shots) of the desired input-output behavior before presenting the actual task, guiding the model through demonstration.
- **Thought Generation (e.g., Chain-of-Thought - CoT):** Encouraging the LLM to generate intermediate reasoning steps or a "chain of thought" before arriving at a final answer, which often improves performance on complex reasoning tasks.
- **Ensembling:** Combining multiple prompts or the outputs generated from multiple prompts to improve robustness, reduce variability, or improve overall response quality.

- **Self-Criticism:** Designing prompts that instruct the model to evaluate, verify, or refine its own generated responses, with the aim of improving reliability and factual accuracy.
- **Decomposition:** Breaking down complex problems or tasks into smaller, more manageable subtasks that the LLM can address sequentially or in a structured way.

Each of these categories encompasses a variety of specific sub-techniques and nuanced approaches. A more detailed breakdown of these techniques, including examples and further sub-classifications, is provided in the Appendix [6.2.6](#).

2.2.2 Error mitigation

Effective prompt engineering is essential not only for optimizing the performance of LLMs but also for minimizing errors in their outputs. By implementing a variety of strategies, users can address prevalent issues such as misinterpretations, hallucinations, inconsistencies, and security vulnerabilities. These error mitigation techniques improve the accuracy, reliability, and trustworthiness of LLM responses in a wide range of applications.

A key strategy for reducing errors is to provide clear, specific instructions that minimize ambiguity. By explicitly defining the task, setting constraints, and specifying the desired output format, such as structured, users can guide the model toward more accurate and complete outputs. This reduces the likelihood of omissions and misinterpretations. Moreover, grounded generation techniques help anchor model responses in verifiable data. For example, methods such as requiring source citations or using RAG reduce the risk of hallucinations by ensuring outputs are factually supported, significantly improving reliability.

Together, with other strategies mentioned in the previous section, it forms a robust framework for minimizing errors through prompt engineering. For example, combining structured reasoning, robust input handling, consistency checks, and clear examples, users can significantly enhance the reliability of LLM outputs.

In conclusion, prompt engineering encompasses a broad spectrum of techniques aimed at guiding large language models to produce accurate and reliable outputs. Methods such as zero-shot and few-shot prompting, chain-of-thought reasoning, ensembling, self-criticism, and decomposition each provide distinct advantages tailored to specific tasks and contexts. These approaches collectively improve the model's ability to handle a wide range of applications, from simple queries to complex problem solving scenarios.

As the field of prompt engineering continues to advance, the systematic integration of these techniques remains crucial for unlocking the full potential of large language models. However, new techniques such as automatic prompt engineering and prompt compression are pushing the

boundaries and suggesting a future where prompt engineering becomes more automated, reducing human effort while maintaining high performance, especially for complex, data-intensive tasks.

2.3 Document Attachment and Context Window

ESG profiling often requires an LLM that digests large volumes of text, for example, sustainability reports, regulatory filings, news articles, etc. Modern models have shown significant improvements in handling long texts, both through extended context windows and clever processing techniques. This section examines how to feed large documents or datasets into LLMs, parsing and inputting document methods, and how recent model developments enable much longer context lengths.

2.3.1 Context Window

The context window represents the maximum amount of text an LLM can process in a single inference, essentially determining how much information the model can "see" and reason about simultaneously. Modern LLMs such as Meta's LLaMA3 (128 K tokens), Anthropic's Claude 3 (200 K tokens) and Google's Gemini 1.5 Pro (2 M tokens) or Gemini 2 (1 M tokens) have dramatically expanded context windows, allowing the analysis of complete documents or datasets in a single query [76, 77, 78, 79]. However, traditional Transformer architectures face inherent challenges with long contexts due to the quadratic computational complexity $O(n^2)$ of self-attention mechanisms. For example, processing a 100 K token document requires 10 billion attention operations, which pose significant memory and latency constraints. [80]

To address these computational challenges, researchers have developed several innovative approaches.

- **Optimized Memory Techniques:** Strategies such as FlashAttention, activation checkpointing, and advanced quantization methods that reduce GPU memory usage and computational demands [81, 82].
- **Linear and Sparse Attention Mechanisms:** Attention variants (e.g., Transformer-XL, Longformer) that scale more linearly with sequence length, or sparse attention patterns (e.g., Block-Sparse, sliding-window) that selectively focus on relevant token relationships [83, 84].
- **Architectural Innovations:** The use of Mixture-of-Experts (MoE) architectures, hierarchical attention, and enhanced positional encoding schemes (such as RoPE) to better

manage and reason over long contexts [85, 86, 87].

A more detailed discussion of these techniques for managing long contexts is provided in the Appendix 6.2.6.

These advances collectively enable the processing of complete reports while maintaining computational feasibility; however, each approach involves different trade-offs between efficiency, performance, and implementation complexity. It is important to understand that even the most modern models may not utilize their entire context window with perfect recall for all tasks, and can still exhibit "lost in the middle" phenomena or hallucinations when trying to retrieve specific information from very long inputs.

2.3.2 Parsing and Feeding Documents

Before LLMs can analyze documents, these documents must be parsed and transformed into a format that the models can process. This involves extracting text and structured data from various file types such as PDFs, Excel sheets, CSV files, or databases. [88]

For documents, converting PDFs to intermediate formats, such as HTML, can preserve more structural information than direct text extraction. However, the optimal approach depends on the specific characteristics and analysis requirements of the document.

Basic text extraction from PDFs employs libraries such as PyPDF2, pdfplumber, or pdf2text to convert document content into plain text. Although straightforward, these methods often lose structural information that may be critical, such as table relationships or hierarchical headings.

Many documents, particularly older reports or scanned materials, require Optical Character Recognition (OCR) to convert image-based content to machine-readable text. Modern OCR systems leverage multimodal models to effectively extract information from both textual and visual elements, such as charts and diagrams commonly found in reports.

Advanced document processing tools now incorporate layout analysis capabilities that preserve document structure during parsing, thanks to their multimodal capabilities. Systems like LlamaIndex's LlamaParse, LangChain or Unstructured.io can identify and maintain structural elements such as headers, tables, and sections, enabling more sophisticated processing of complex documentation. [89, 90, 91, 92]

Post-parsing techniques, including data cleaning strategies such as artifact removal, formatting standardization, and OCR error correction, improve input quality. Chunking strategies divide documents into semantically meaningful segments to effectively manage context limits. Additionally, hierarchical document representation methods independently process these

chunks before synthesizing them into a unified analysis. These methods also involve preserving important metadata, such as publication date, authorship, and document structure, and generating preliminary summaries for individual sections. In doing so, the models establish an initial high-level overview, facilitating detailed subsequent analysis.

RAG further enhances document processing by dynamically retrieving relevant external information during inference. This approach will be discussed in greater detail in Section 3 of this State-of-the-Art chapter.

2.3.3 Limitations of Large-Context Models

Despite advances in context-window technology, several significant limitations affect the application of LLMs to large context-window tasks.

Cost and Speed

Processing extensive documentation with large-context models involves substantial computational overhead, resulting in significant cost and latency implications. As the context length increases, both the inference time and API costs increase proportionally, creating practical constraints for real-time analysis applications. [93]

Attention Dilution

When processing very large documents, models often experience a phenomenon known as *attention dilution*, where critical information is masked by less relevant content. This effect may result in the model overlooking key metrics or misinterpreting essential points within extensive reports.

Research studies have shown that model performance tends to degrade as context length surpasses certain thresholds, exhibiting diminishing returns with each additional token. This performance degradation is particularly pronounced in reasoning tasks, where synthesis of information from distant sections of a document becomes increasingly challenging. [94]

Input Formatting and Errors

Large documents often contain complex formatting elements that convey critical information, including tables of environmental metrics, hierarchical organizational structures, and specialized notation. Current models lack robust mechanisms for understanding document structure unless it is explicitly encoded in the input text through formatting cues like Markdown or XML.

Lack of Long-Range Reasoning Training

Another fundamental limitation of current models is their relatively weak capability for long-range reasoning in extended contexts. Although technically able to process long input, models often struggle to draw connections between widely separated sections of a document or to maintain consistent reasoning across thousands of tokens. [94]

In conclusion, the challenges and solutions associated with document attachment and context window management represent critical considerations for AI-assisted ESG profiling. Despite the fact that recent advances in context window technology have dramatically expanded LLMs' ability to process lengthy documentation, significant limitations remain in terms of cost, attention management, and long-range reasoning capabilities. Nonetheless, as these technologies continue to evolve, the capabilities for automated document analysis will likely expand, enabling increasingly sophisticated and reliable analysis.

2.4 Hallucinations

One significant challenge in leveraging LLMs for factual domains such as ESG company profiling is the occurrence of hallucinations. This refers to instances where the model generates information or details that are not grounded in the provided data and are factually incorrect. Hallucinations can manifest in varying degrees, from minor inaccuracies like misstating a numerical value to more critical errors such as inventing nonexistent policies or events. In high-stakes applications like ESG analysis, the presence of hallucinations is particularly problematic, as decisions could be based on spurious information, undermining the reliability of AI-driven insights.

Complementary to the issue of hallucination is source attribution, which is the ability to reliably trace the origin of the model statements back to verifiable sources or datasets. The importance of source attribution is paramount to ensure transparency and build trust in AI-generated output. This section explores the underlying causes of hallucinations and examines various strategies developed to mitigate these issues, emphasizing how techniques like source provision and retrieval mechanisms contribute to enhancing the trustworthiness of LLMs. [95, 96]

2.4.1 Why LLMs Hallucinate and Why It Matters

LLMs hallucinate primarily because they are trained to generate statistically plausible text sequences rather than to act as knowledge repositories or fact checkers. This process enables

them to generate coherent and contextually relevant text, but it does not inherently equip them with a mechanism to discern truth from falsehood or to verify factual accuracy against an external knowledge base [97]. Even when provided with documents or extensive text as a context, LLMs are not always adept at strictly adhering to the information contained within. They might still generate responses that, while seemingly related to the input context, introduce fabricated details or deviate from the factual content of the source material. This tendency to invent information, rather than strictly extracting and synthesizing the provided text, is a core aspect of why hallucinations occur, even in retrieval-augmented setups [98].

In the context of ESG analysis, the implications of hallucinations are particularly significant. ESG assessments require high levels of accuracy and reliability, as they inform investment decisions, regulatory compliance, and stakeholder trust. Inaccurate ESG profiles due to hallucinations could lead to misinformed investment strategies, flawed risk assessments, and reputational damage. Therefore, mitigating hallucinations and ensuring source attribution are critical for the responsible and effective application of LLMs in ESG company profiling and similar high-stakes domains.

2.4.2 Strategies to Mitigate Hallucinations

Researchers and practitioners have developed multiple approaches to reduce hallucinations [98, 99, 100, 101, 97]. These strategies aim to improve the factual accuracy and reliability of the generated text. Key categories of mitigation techniques include the following.

- **Prompting and Instructions:** Carefully crafting prompts to explicitly guide the LLM, instruct it to avoid guessing, strictly utilize the provided context or even engage in self-verification steps [102].
- **Retrieval-Augmented Generation (RAG):** Grounding the LLM's generation process in verifiable data retrieved from external knowledge sources, thereby reducing reliance on parametric memory, which can be a source of fabrications.
- **Fine-Tuning and Domain Adaptation:** Adapting LLMs to specific domains or tasks by training them on relevant, high-quality datasets, potentially teaching them to indicate uncertainty or verify responses against trusted knowledge, although this can be resource-intensive [99].
- **Tools and External Checks:** Integrating external automated verification tools or secondary LLMs to cross-reference generated statements against trusted sources, flagging or correcting inaccuracies post-generation.

- **Controlled Generation and Calibration:** Employing experimental techniques to modify the LLM’s generation process itself, such as calibration to improve uncertainty expression, or constrained decoding to prevent certain types of unsupported outputs [96].

A more detailed explanation of each of these strategies is provided in Appendix 6.2.6.

In conclusion, hallucination remains a significant and unresolved challenge in the broader field of LLMs. It is arguably one of the most critical obstacles to achieving truly trustworthy LLMs output, especially in sensitive domains. Currently, the combination of RAG, meticulous prompt engineering, and robust answer verification processes represents the most pragmatic and effective approach for many real-world applications. This multifaceted strategy essentially outsources the factual recall to a reliable external database or knowledge source, while leveraging the strengths of the LLM in language understanding, reasoning, and synthesis to process and interpret the retrieved information.

2.5 Retrieval-Augmented Generation (RAG)

This section provides a comprehensive analysis of the RAG architecture, previously established throughout the thesis, and describes how retrieving relevant information and integrating it into an LLM can significantly improve factual accuracy.

2.5.1 Concept and Motivations

RAG can be seen as giving an LLM an open book during its exam. Instead of relying solely on the information stored in its weights, the LLM is provided with relevant text from an external repository at query time, and it uses that text to compose its answer. The paradigm was popularized by a 2020 Facebook AI paper titled "Retrieval-Augmented Generation for Knowledge-Intensive NLP" [103] which found that even a smaller generative model could outperform a much larger one on knowledge-heavy tasks if it had access to a retrieval mechanism. The motivation is clear; LLMs find it challenging to deal with current or specialized information that was either absent or minimally represented in their training data set. By augmenting them with a retriever, detailed knowledge as needed can be injected.

A typical RAG system operates through a sequence of steps. Initially, an incoming query is processed by a retriever, a component that functions similar to a search engine or vector similarity lookup. This retriever identifies and extracts documents or snippets that are semantically related to the query. These retrieved snippets are then combined with the original query, and the composite text is fed into the LLM. The intention is for the LLM to generate an answer

that is informed by the retrieved evidence, ideally quoting or summarizing relevant portions of the snippets. In this manner, the generated answer is augmented by knowledge that the model might not inherently possess. Essentially, the generative model transitions into a reasoning and language engine operating on top of a dynamically updated knowledge base. [104, 105, 48]

For applications like ESG profiling, the advantages of RAG are particularly compelling. An ESG-focused knowledge base can encompass a wide range of documents, including company disclosures, news articles, or academic research reports detailing industry-specific risks. When generating an ESG profile for a particular company, the system can retrieve precise data points, such as "the company's latest emissions data" and "a news story about a recent scandal" and provide these to the LLM to create a unified narrative. Without RAG, the model might either fail to recall these specific details or, as previously discussed, hallucinate similar but inaccurate information. RAG ensures that the output remains firmly grounded in verifiable data.

Another critical motivation for RAG is currency; LLMs, by nature of their training process, possess knowledge that is inherently time-lagged, with training datasets often having a cutoff date (e.g., 2021 or 2022 data). RAG allows for the incorporation of the most recent information by simply updating the indexed document repository. This adaptability is far more efficient and practical than frequently retraining the entire LLM [104]. Furthermore, RAG enables the utilization of local models in scenarios where internet connectivity is limited or data privacy is paramount.

Finally, RAG offers enhanced controllability and transparency. Developers have direct control over the sources accessed by the model, enabling them to curate a knowledge base comprised solely of authoritative reports and reputable news sources, rather than relying on potentially unreliable or biased internet content, thereby ensuring input quality. As mentioned, RAG systems can be designed to include citations in the output, directly linking assertions back to their source snippets, significantly enhancing the transparency and verifiability of the generated responses. These inherent benefits explain why RAG has become a foundational approach for applications ranging from question answering systems and virtual assistants operating on private datasets to any scenario where factual correctness and source traceability are critical requirements.

2.5.2 Representative RAG Architectures and Tools

A typical RAG pipeline is composed of several interacting components, designed to efficiently retrieve and utilize external knowledge to augment LLM responses. [104, 105, 48]

Knowledge Repository

The knowledge repository serves as the external source of information that the RAG system queries. This repository can take various forms, including a collection of documents, a structured database, or a knowledge graph. For effective implementation of RAG, the repository is typically pre-processed using techniques such as text chunking, segmenting documents into fragments of approximately 256-512 tokens, and then converting each chunk into a dense vector embedding using embedding models such as Hugging Face Transformers embedding [106]; popular choices include *text-embedding-ada-002*, *text-embedding-3-large*, *bge-large-en-v1.5*, or models from the *SentenceTransformers* library. The selection of the embedding model plays a crucial role, as it directly affects the quality of the semantic representation and, consequently, the effectiveness of the retrieval process. The resulting embedding vectors, which numerically capture the semantic content of each fragment, are then stored in a vector database to enable fast and semantically relevant retrieval.

The implementation of the knowledge repository and vector database can be carried out locally using open source libraries such as FAISS (Facebook AI Similarity Search) [107], Chroma [108], or LanceDB, which are suitable for development and experimentation on local machines. For production environments or large-scale applications, cloud-managed vector database services such as Pinecone [109], Weaviate [110], Chroma [108], or Qdrant [111] offer scalable and robust solutions, capable of handling large volumes of embeddings while simplifying infrastructure management [112].

In order to improve both the quality of the data and the effectiveness of retrieval, it is essential to pass documents through a cleaning process before chunking or embedding them. As discussed in Section 2.3.2 (*Parsing and Feeding Documents*), these steps may include parsing documents, cleaning text by removing artifacts and standardizing formats, implementing fragmentation strategies to segment documents semantically, and extracting relevant metadata to enrich context [113].

Lastly, frameworks such as LangChain [114], LlamaIndex [115] or Haystack [116] provide abstractions and tools to streamline the integration with various vector databases, embedding models and document loaders, simplifying the development of RAG pipelines. [117]

Retriever

The retriever component is responsible for identifying and fetching relevant information from the knowledge repository in response to a user query. When a query is received, it is first transformed into an embedding vector using the same embedding model used to index the documents in the vector database. This query embedding is then used to perform a similarity

search against the vector index to find the most semantically similar document chunks. The number of retrieved snippets can be adjusted based on the application requirements and the constraints of the context window of the LLM; typically, retrieving the top K (e.g., the top 3, 5 or 10) snippets [48].

Although vector similarity search is the predominant retrieval method in RAG systems, keyword-based search can be used as an alternative or complementary approach. Keyword search, using techniques such as TF-IDF or BM25, can be effective in retrieving documents based on lexical matching, which can be beneficial when queries are keyword-driven or when seeking a hybrid retrieval strategy that combines semantic and lexical relevance [105]. The prompt itself significantly influences both the retrieval and answer generation stages. Well-designed prompts guide the retriever to focus on the most relevant aspects of the query and ensure that the retrieved context is pertinent to the desired answer. Specific prompting strategies, such as query expansion and decomposition, can further enhance retrieval accuracy and relevance [117].

Beyond traditional text-based retrieval, more advanced variants of RAG have emerged, such as GraphRAG, which use structured knowledge representations to improve retrieval quality. In this approach, information is organized in the form of knowledge graphs, which consist of entities and their relationships, allowing more context-aware and precise retrieval through graph traversal. GraphRAG is particularly useful for complex queries that require reasoning about structured knowledge and relationships between entities [118]. The models treat the query and the search on the vector database differently. The query is used primarily to guide the retrieval process, while the retrieved context is used to inform the generation process. The retriever focuses on semantic similarity between the query and document chunks, whereas the LLM leverages the retrieved context to generate a coherent and contextually relevant answer.

Prompt Assembly

The prompt assembly stage involves constructing the input prompt that will be fed to the LLM, incorporating the retrieved snippets from the knowledge repository. The goal is to create a prompt that effectively instructs the LLM to utilize the provided context to answer the user's question accurately and reliably. A common prompt structure involves a template that includes instructions for the LLM, the user's question, and the retrieved excerpts. The prompt template can be customized to suit different models and frameworks.

Frameworks like LangChain and LlamaIndex provide tools and abstractions to manage prompt templates and to dynamically build prompts. LangChain, for example, offers PromptTemplate classes that allow for defining prompt structures with placeholders for variables like question and context, simplifying the prompt assembly process. LlamaIndex provides similar

prompt management capabilities and allows for creating more complex prompt structures tailored to different retrieval and generation scenarios. When designing prompt templates, it is crucial to consider whether the prompt for searching the vector database (retrieval prompt) should be the same as the prompt used for the LLM (generation prompt). In many cases, the retrieval prompt is a simplified version of the user query, focused on capturing the core semantic intent for effective document retrieval. The generation prompt, on the other hand, is more elaborate, including instructions, context, and the full user question to guide the LLM's answer generation process. [117]

Generation

In the generation stage, the LLM processes the assembled prompt, which includes the user's question and the retrieved context, to generate a final answer. The LLM leverages its language understanding and reasoning capabilities to synthesize information from the retrieved snippets and formulate a coherent and contextually relevant response. Ideally, the LLM should primarily rely on the information provided in the retrieved context to construct its answer, minimizing reliance on its parametric memory and reducing the risk of hallucinations. The choice of LLM can impact the effectiveness of the RAG system. Although many LLMs can be used in RAG pipelines, some models may be better suited for this task due to their architecture, training, or fine-tuning. Open-source models that are instruction tuned and designed to follow instructions closely, such as Meta's Llama series or Mistral AI's models, tend to perform well in RAG setups as they are more responsive to prompt guidance and context. The generation process in RAG aims to leverage the LLM's strengths in language articulation and reasoning while offloading the factual information retrieval to the retriever and knowledge repository. This division of labor improves the accuracy, reliability, and transparency of the responses generated.

Post-processing

Post-processing is an optional but valuable stage in the RAG pipeline that can further enhance the quality and trustworthiness of the generated answers. One common post-processing technique is citation augmentation, where the generated answer is augmented with citations that link specific pieces of information back to their source snippets. This can be achieved by tracking which parts of the generated text are derived from which retrieved snippets and then appending footnote numbers or in-text citations accordingly [119, 105].

Some RAG systems, such as those built with LlamaIndex or Langchain, offer built-in citation capabilities, automatically adding citations to the generated text. For systems without native citation support, post-processing techniques can be implemented to identify source attribution. Simple methods, such as string matching quotes from the generated answer to the retrieved

snippets, or more advanced methods using semantic similarity or attention mechanisms, can be used to determine the source of each piece of information.

Beyond citation, post-processing can also involve answer verification. This step aims to further validate the accuracy and factual correctness of the generated answer, even after the RAG has grounded the response in the retrieved context. One approach is to use another LLM, or a smaller verification model, to check the consistency and factual alignment of the generated answer with the retrieved sources. This verification step can help identify and flag potential inaccuracies or hallucinations that may have slipped through the RAG process. By incorporating post-processing techniques like citation and verification, RAG systems can not only provide more accurate and reliable answers, but also enhance transparency and user trust by clearly indicating the sources of information and validating the factual integrity of the generated content. [120]

2.5.3 Types of RAG

In addition to the representative RAG architecture outlined, various specialized RAG approaches have emerged, each tailored to address specific challenges or enhance performance in different contexts. These can be broadly categorized according to their complexity and the sophistication of their retrieval and generation mechanisms. [121, 105]

Beyond the foundational "Naïve RAG" architecture, which involves a straightforward pipeline of query embedding, document retrieval, and LLM generation, key categories of these evolving RAG types include:

- **Advanced RAG:** Incorporates sophisticated methods for query optimization, intelligent chunking/indexing, re-ranking of retrieved results, and enhanced prompt engineering.
- **Modular RAG:** Emphasizes flexibility through distinct, interchangeable modules for components such as retrieval, context enrichment, generation strategies, and output processing.
- **Adaptive RAG:** Dynamically adjusts retrieval and generation strategies based on input query characteristics, retrieved information quality, or LLM confidence.
- **Corrective RAG:** Focuses on enhancing factual accuracy by integrating mechanisms for error detection, fact verification, and self-correction loops.
- **Self-RAG:** Trains the generative model itself to decide when and what to retrieve and how to integrate this information, making the LLM an active participant in the retrieval process.

- **Graph RAG:** Utilizes knowledge graphs as the primary knowledge source, enabling retrieval through graph traversal and reasoning over structured entity relationships.
- **Hybrid RAG:** Combines different retrieval methods (e.g., vector and keyword search) or knowledge sources (e.g., text and knowledge graphs) to leverage their complementary strengths.
- **Agentic RAG:** Integrates LLMs with agent-like functionalities for complex, interactive information seeking, planning, and interaction with external tools or APIs [122].

A detailed description of each of these RAG types, outlining their specific mechanisms and objectives, is provided in Appendix 6.2.6.

These diverse types of RAG architecture illustrate the ongoing evolution and refinement of retrieval-augmented generation techniques. The choice of RAG type depends on the specific application requirements, the complexity of queries, the nature of the knowledge base, and the desired level of accuracy, efficiency, and interactivity.

2.5.4 Evaluation of LLMs for RAG

Evaluating RAG systems is crucial to ensure their effectiveness in generating accurate and relevant responses and efficiency in their operation. A comprehensive evaluation strategy must consider both the quality of the outputs generated and the resources required to produce them. This section outlines key evaluation metrics and practical approaches for measuring RAG systems, focusing on both effectiveness and efficiency.

Key dimensions for evaluating the effectiveness of RAG systems include [105]:

- **Query-Context Relevance:** Assessing the alignment between the retrieved context and the user’s query, as the quality of the retrieved information is foundational. Metrics often focus on precision and recall of relevant documents (e.g., from frameworks like KILT, ARES, RAGAS, TruLens).
- **Context-Answer Coherence (Faithfulness):** Ensuring the generated answer is consistent with and factually supported by the retrieved information, minimizing hallucinations. This involves metrics like answer faithfulness, groundedness, and specific citation-focused scores (e.g., from CRUD, RECALL, RECLAIM, LongBench-Cite, ALCE).
- **Query-Answer Accuracy and Relevance:** Measuring how well the final generated response directly and correctly addresses the user’s query, including aspects like hallucination rates, missing information, and task-specific performance (e.g., from CRAG, RAGAS, ARES, TruLens, CRUD/RAGQuestEval, KILT, RGB).

In addition to effectiveness, the efficiency of RAG systems is critical for practical deployment. Key efficiency metrics focus on performance and resource consumption [105]:

- **Latency:** The end-to-end response time from query input to answer output.
- **Throughput:** The number of queries the system can process per unit of time.
- **Resource Utilization:** The computational resources (CPU/GPU, memory) consumed during operation.

A detailed breakdown of specific metrics, benchmarks, and frameworks used for evaluating these aspects of RAG systems is provided in Appendix 6.2.6.

By comprehensively evaluating both the effectiveness and efficiency of RAG systems using the metrics and practical approaches outlined above, developers can build robust, reliable, and high-performance RAG-based applications for various domains. Continuous evaluation and iterative refinement are key to optimizing RAG systems for real-world deployment.

2.5.5 RAG vs. Long-Context Models

As context window capacities of LLMs have expanded dramatically, reaching lengths of 100k to 2M tokens, a pertinent question arises: does RAG remain necessary, or can we simply input all relevant documents directly into the prompt of a long-context model? Although long-context models offer the advantage of processing large amounts of text in a single pass, RAG and long-context approaches present distinct trade-offs in terms of efficiency, relevance, maintainability, and overall performance, as briefly discussed in Section 2.3.1 (*Context Window*). [47, 105]

Efficiency

RAG generally exhibits greater computational efficiency compared to long-context methods. Streamlining the process by selectively retrieving only the most pertinent information, significantly reducing the amount of text that the LLM must process for each query. Consider a scenario with a knowledge corpus spanning 1,000 pages; a purely long-context approach would necessitate feeding a substantial portion, if not all, of this corpus into the model's context window, potentially exceeding context limits or incurring prohibitively high computational costs due to the extensive token processing. In contrast, RAG retrieves only a small subset of highly relevant paragraphs, perhaps a few hundred tokens, directly relevant to the specific query. The LLM then operates only in this focused context, leading to substantial reductions in computational expense and processing time, potentially by orders of magnitude. This efficiency gain

is analogous to consulting only the relevant pages of a book to answer a question rather than reading an entire library.

Relevance and Focus

Long-context models, while capable of ingesting large volumes of text, rely on their internal attention mechanisms to discern relevant information within the extensive input. Although effective to some extent, this approach can be susceptible to attention dilution, where critical details become obscured within a large amount of less relevant content. RAG, on the contrary, prioritizes relevance by performing explicit information retrieval before feeding context to the LLM. A well-tuned retriever effectively filters out noise and extracts highly pertinent snippets, simplifying the LLM's task by providing a focused and relevant context. This pre-filtering can enhance the quality of the generated answers by ensuring that the LLM primarily attends to the most important information. However, RAG introduces a potential point of failure: If the retriever fails to identify and retrieve a crucial document, the LLM will lack that information, potentially leading to incomplete or inaccurate answers. Long-context models, in theory, avoid this specific failure mode as all information is present in the context (assuming that it fits within the context window). However, in practice, if the context becomes too long, the model might struggle to effectively process and utilize all the information, potentially overlooking key details in the noise.

Maintenance and Updatability

RAG offers significant advantages in terms of knowledge base maintenance and updatability. In a long-context approach, preparing the prompt still requires a retrieval-like step to select relevant documents for inclusion in the context window, often performed manually or through ad-hoc methods. RAG formalizes and automates this process through a dedicated retriever component. The knowledge repository underlying an RAG system, often a vector database or search index, can be continuously updated with new documents and revisions to existing ones without requiring any changes to the LLM itself. This decoupling of knowledge and reasoning components is particularly valuable in dynamic enterprise environments where information is frequently updated. In contrast, manually curating and updating prompts with relevant documents for a long-context approach is less systematic and becomes increasingly cumbersome as the knowledge base evolves.

Combining Approaches

It is important to note that the RAG and long-context approaches are not mutually exclusive and can be effectively combined. Hybrid architectures can take advantage of the strengths of both methodologies. For instance, with a long-context model capable of processing 100k

tokens, a RAG system could be used to retrieve a larger set of relevant documents, say the top 20, and all of these retrieved documents, if their combined token count is within the context limit (e.g., 80k tokens), can be included in the prompt. This hybrid strategy aims to ensure broad coverage of relevant information while still benefiting from the focused retrieval of RAG. In scenarios with smaller context windows (e.g., 4k or 8k tokens), RAG might retrieve only a few highly selective documents, whereas larger context windows enable the inclusion of a more comprehensive set of retrieved documents. However, even with very large context windows, a retrieval step remains crucial to select the most pertinent documents from a potentially vast knowledge base, as even 100k or 2M tokens are insufficient to encompass an entire enterprise knowledge corpus.

In summary, RAG represents a strategy for leveraging knowledge on demand and selectively, offering enhanced efficiency and often improved effectiveness compared to indiscriminately feeding entire knowledge bases into LLMs. Especially for applications like ESG profiling, where queries are diverse and require specific types of information (e.g., carbon emissions data, social policies), RAG excels at fetching just the relevant sections of ESG reports or news articles for each query. Recognizing the limitations of RAG, including possible retriever inaccuracies, its benefits in terms of efficiency, relevance, maintainability, and transparency render it an attractive option for the project and many other cutting-edge systems.

3 Methodology

3.1 Introduction and Architectural Overview

Building upon the challenges identified in the preceding chapters, namely the labor-intensive nature of manual development of ESG company profiles and the inherent limitations of standard LLMs regarding factuality and traceability, this chapter outlines the methodology that was designed to address these issues.

As established in the State of the Art (Chapter 2), while LLMs possess remarkable language understanding and generation capabilities, their application in high-stakes domains such as ESG requires careful mitigation of risks such as hallucination and lack of source attribution. Consequently, this project adopted a RAG approach, which was selected for its proven ability to ground the LLM output in verifiable external knowledge sources, thus enhancing the factual accuracy and enabling transparent citation.

The overview of the general architecture of the proposed system is conceptually illustrated in Figure 3.1. The system was designed with a workflow that begins with analyst input (company identification and document provision), proceeds through automated data ingestion, pre-processing, RAG-based synthesis, and culminates in a final verifiable ESG profile draft ready for analyst review. The system was designed to integrate both external documents and proprietary internal ESG metrics, using an LLM to synthesize these disparate data streams into a coherent narrative, with the generated output explicitly citing its sources.

This chapter details each stage of this methodology. Section 3.2 describes the input data required by the system. Section 3.3 explains the document pre-processing steps and the construction of the vectorized knowledge base. Section 3.4 details the core RAG pipeline, including prompt engineering, information retrieval, and profile generation. Section 3.5 details the post-generation verification mechanisms. Section 4.2 provides an overview of the potential technologies considered for implementation, and Section 3.6 briefly outlines the planned evaluation strategy.

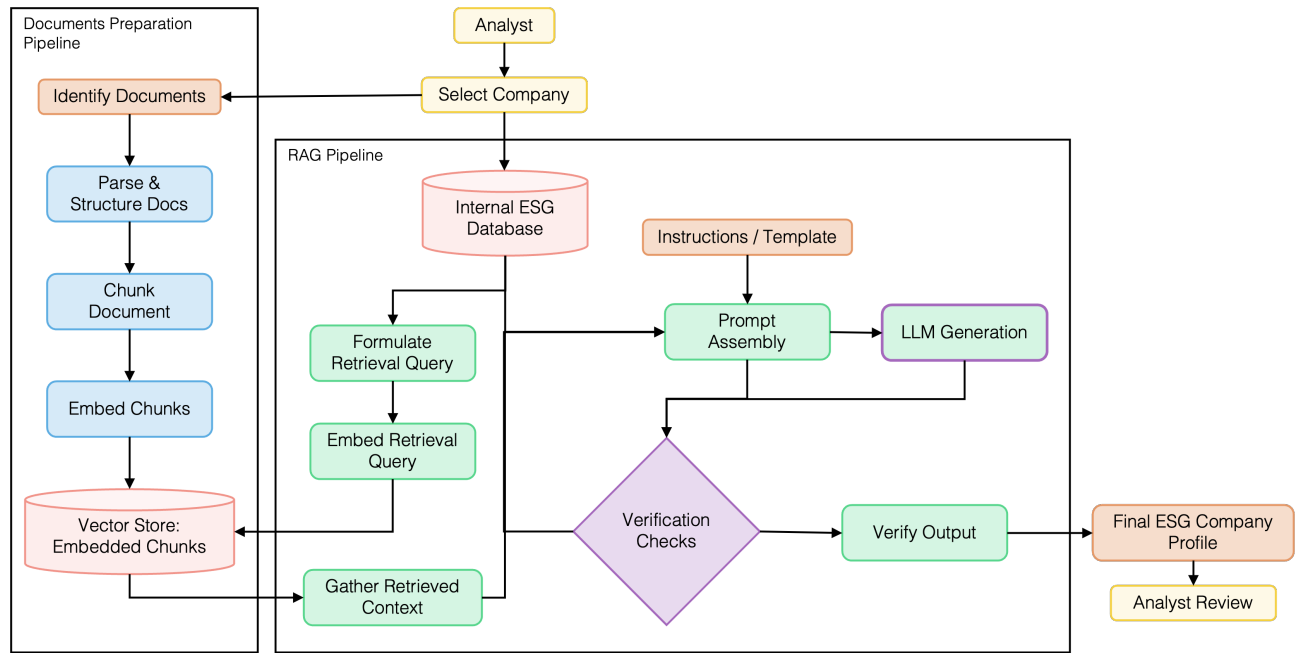


Figure 3.1: Workflow Diagram of RAG System.

Although this chapter details the core methodological steps and key parameters used for the presented results, a comprehensive list of all tools, specific library versions, alternative configurations explored during development, and full implementation details, including precise parameters for the example runs discussed in the Results Analysis chapter 5, can be found in the Implementation chapter 4 and the accompanying code repository [123].

3.2 Data Sources and Ingestion

The foundation of the AI-assisted ESG profiling system was built upon the effective ingestion and integration of diverse data sources, combining external reports with proprietary internal metrics.

3.2.1 Overview of Analyst Interaction and Input Data

The process was designed to begin with the sustainability analyst interacting with the system interface. The primary inputs required from the analyst were the following:

1. **Company Identification:** The analyst specifies the target company, typically using a unique identifier or company name.

2. **Document Provision:** The analyst either uploads new relevant documents (in PDF format) for the specified company or selects from documents already present in the system's repository. The system was designed to detect if specific documents for a given company have already been processed, thereby avoiding redundant pre-processing and leveraging existing vectorized data when available.

3.2.2 External Document Corpus

The system was designed to process a range of external documents that formed the core textual basis for ESG analysis. These typically included:

- **Company Sustainability Reports:** Official reports published by the company detailing its ESG initiatives, performance metrics, policies, and targets. These are often the primary source of company-specific ESG information.
- **MSCI ESG Research Reports:** Reports from third-party data provider MSCI, offering standardized ESG ratings, data points, and analysis based on their proprietary methodology.
- **RepRisk Reports:** Reports from RepRisk, another third-party provider, that focus on ESG and business conduct risks identified through media screening and stakeholder sources, often highlighting controversies.

These documents generally contain a wealth of unstructured and semi-structured information crucial for building a comprehensive ESG profile.

3.2.3 Internal ESG Data

A key component of this methodology was the integration of proprietary internal ESG data maintained by the firm's analysts. This ESG data offered a detailed assessment of the company's sustainability performance based on the firm's specific methodology. This internal data included:

- **Key Issue Scores:** Quantitative scores assigned to the company based on specific ESG Key Issues (KIs) identified as significant based on its particular GICS sub-industry (e.g., 'Human Capital Development', 'Privacy & Data Security', 'Board Independence' or 'Business Ethics').
- **Subindustry Peer Data:** Statistical context for each Key Issue score, including minimum, maximum, median, and quartile values for the company's subindustry peer group. This allows for a relative performance assessment.

- **Controversy Metrics:** Multiplier value that reflects the impact of controversies on the overall scores of the E, S and G pillars.
- **ESG Rating Methodology:** Definitions, weights, and calculation logic used internally to derive pillar scores (E, S, G) and the final company and industry ratings. This includes the formulas for combining Key Issues, applying multipliers, and determining the final ESG score. Details regarding the industry rating calculation and the Inclusion Threshold logic, which determines the best-in-class rating status based on industry positioning, are also part of this internal data.
- **Exclusion Criteria:** Information regarding the company’s status relative to defined exclusion criteria based on controversial business activities.

This internal data was designed to be retrieved from an internal database based on the selected company and be directly incorporated into the LLM prompt during the generation phase, ensuring that the AI’s output aligns with the specific requirements and metrics of the internal ESG framework.

3.3 Document Preprocessing and Knowledge Base Construction

To enable effective retrieval and comprehension by the LLM, the raw input documents underwent a multi-stage pre-processing pipeline before being stored in a knowledge base.

3.3.1 PDF Conversion and Structural Parsing

Given that primary input documents are PDFs, the first step involved converting them into a format more suitable for automated analysis while preserving crucial structural information. PDFs, especially complex reports, often contain layouts, headings, tables, lists, and figures that convey meaning. Simple text extraction often loses this context. Therefore, robust conversion techniques were used to transform PDFs into structured formats such as Markdown. This approach aimed to retain the hierarchical structure (e.g., #, ## tags corresponding to sections) or tabular data which can significantly aid downstream processing and semantic understanding. Critically, during this conversion process, metadata linking the extracted text content back to its original page number or location within the source PDF was preserved, as this is essential for accurate citation later.

3.3.2 Text Chunking Strategy

LLMs operate with finite context windows, and processing entire reports simultaneously is often infeasible or inefficient (due to cost and potential performance degradation, as discussed in the State of the Art (Chapter 2)). Furthermore, retrieval is typically performed on smaller, focused segments of text rather than entire large documents. Therefore, following the parsing process, the document content was segmented into smaller chunks.

The chosen chunking strategy for this methodology was based on page boundaries. Each page of the source PDF document was treated as a distinct chunk. This approach, while simpler than semantic or structural chunking, was adopted for several practical and domain-specific reasons:

- **Simplicity and Implementation Efficiency:** Page-based chunking is straightforward to implement across diverse document layouts and parsing outputs, reducing complexity in the pre-processing pipeline.
- **Alignment with Document Structure:** Corporate reports and regulatory filings are often designed with content logically segmented on a page-by-page basis, with authors attempting to present coherent information units within these boundaries.
- **Enhanced Citation Accuracy and Usability:** Dividing by page directly corresponds to the familiar and essential method of citing source material by page number. This makes the AI-generated citations intuitive and easily verifiable by a human analyst against the original PDF documents.
- **Effective Retrieval Granularity:** While information on a topic may span multiple pages, retrieving relevant individual pages allows the LLM to access concentrated bursts of context directly tied to specific locations in the source. Retrieving multiple relevant pages implicitly provides broader context around key mentions.

Each resulting page-chunk retained its original metadata, specifically the source document path and the corresponding page number from the original PDF. This strategy created contextually relevant units of information directly tied to the source document's pagination, suitable for embedding and retrieval while preserving the crucial link for accurate citation.

3.3.3 Vectorization and Storage

The text content of each page-based chunk, generated from the parsed documents, was then transformed into high-dimensional numerical vectors, known as embeddings. This process,

performed using a pre-trained sentence transformer model (e.g., BAAI/bge-large-en-v1.5 producing embeddings of 1,024 dimensions), captured the semantic meaning of the text contained within each page, allowing for similarity-based retrieval.

These embedding vectors, along with the corresponding raw page-chunk text and its essential metadata (such as the source document and page number), were stored in a vector database (e.g., LanceDB). Thus, it served as the searchable knowledge base containing the parsed and embedded content of the external documents, indexed page by page.

3.4 Retrieval-Augmented Generation Pipeline

The core of the system lay in the RAG pipeline, which dynamically retrieved relevant information from the knowledge base and used it, along with internal data, to guide the LLM in generating the ESG profile draft.

3.4.1 Prompt Engineering Strategy

Effective prompting is crucial for guiding LLMs at various stages of the pipeline. Three distinct prompts were designed and utilized in the implemented system:

1. **Retrieval Prompt:** This prompt was derived from the specific key issues of the ESG profile being generated, often augmented with relevant keywords loaded from a knowledge file. It was converted into an embedding and used to query the vector database to find the most relevant document chunks. The design emphasized clarity and keyword relevance to optimize semantic search results.
2. **Core Generation Prompt:** It was the most complex prompt, assembled dynamically before being sent to the final LLM. It was designed to orchestrate the synthesis of information and included several key components:
 - **Role Instruction and Tone:** Defining the LLM's persona and the desired tone of the output text (e.g., Senior ESG Investment Analyst, neutral/formal tone).
 - **Task Definition:** Clearly stating the specific task (e.g., Generate a SWOT analysis for a particular Key Issue).
 - **Formatting and Style Instructions:** Specifying the desired output structure (e.g., SWOT format), tone, and referencing the example output format.
 - **Citation Requirement:** Explicitly instructing the LLM to cite the source for every piece of information, indicating whether it comes from a specific document

chunk (with identifier and page number, as extracted from the metadata during pre-processing and included in the retrieved context) or from the provided "internal data."

- **Internal ESG Data:** Structured internal data described in Section 3.2.3 (Key Issue scores, peer stats, controversy data, rating definitions), formatted into a clear text string.
- **Methodology Context:** Including summaries of the internal rating methodology and exclusion criteria to ensure the LLM understands the framework it is operating within.
- **Retrieved Context:** Incorporating the top-k relevant chunk text retrieved from the vector database based on the retrieval prompt. This text includes the original page numbers from the source documents for traceability.
- **Few-Shot Examples:** Providing one or more examples of desired output format and content style for a specific Key Issue (loaded from a knowledge file).
- **Negative Constraint:** Instructing the model not to invent information and to state explicitly when information is unavailable in the provided context.

3. **Verification Prompt:** Used in the post-generation phase, this prompt instructed a potentially separate LLM to act as a fact-checker, evaluating the preceding generated text against the exact same internal data and retrieved context that were provided to the generation LLM. It required the verification LLM to output a structured JSON assessment, including scores for faithfulness and citation correctness for each individual statement (bullet point) in the generated text.

3.4.2 Information Retrieval

Once a specific section of the ESG profile needs to be generated (corresponding to a Key Issue), the corresponding retrieval prompt was formulated based on the Key Issue name, description, and associated keywords. This text prompt was then embedded using the configured embedding model. The resulting query embedding was used to perform a similarity search against the vector index created in Section 3.3.3.

The system retrieved the top k most semantically similar chunks from the vector database. The value of k was a configurable parameter (e.g., $k=15$) that balances the breadth of information provided to the LLM against the limitations of the context window and the potential for introducing irrelevant noise. The retrieval process focused on fetching the chunk content and its associated metadata (including the source document name and page number) to ensure

traceability. The selection of the optimal value for k and strategies for dynamic retrieval based on context window size and retrieval scores were crucial aspects of the RAG design.

3.4.3 LLM-Based Profile Generation

The assembled core generation prompt, containing instructions, the query reformulated for generation, internal data, methodology context, and the retrieved chunk text, was then passed to the main generative LLM. For this project, the system implemented was designed to be flexible, supporting different LLM providers and models, with a focus on capable open-source options or accessible APIs (e.g., Azure GPT-4o-mini).

The LLM was expected to synthesize the information provided, drawing connections between the qualitative statements in the retrieved chunk text and the quantitative metrics from the internal data, to generate the analysis text for the specific Key Issue. Adhering to the prompt's instructions, the LLM structured the output according to the desired SWOT format and crucially included citations for factual claims, attributing them either to the specific source document chunk (using the metadata) or labeling them as derived from the provided "internal data".

3.5 Post-Generation Correction and Verification Layer

Even with RAG and careful prompting, LLM outputs may occasionally contain subtle inconsistencies, inaccuracies, or hallucinated citations. To enhance the reliability of the generated draft profile, a post-generation verification layer was incorporated into the methodology. This layer served as an automated quality check before the output reaches the human analyst and could optionally trigger regeneration attempts.

3.5.1 Automated Verification using a Dedicated LLM Call

Immediately after a section of the profile is generated for a specific Key Issue, a dedicated verification step was executed. This involved a separate call to an LLM, potentially a smaller or specifically tuned model optimized for fact-checking. This "verification LLM" was provided with:

- The exact same internal data and retrieved external context that the generation LLM received.
- The text that the generation LLM just produced for that Key Issue.

- A detailed instruction prompt (3.4.1) asking it to evaluate the generated text, statement by statement (bullet by bullet).

The verification LLM (e.g., Azure GPT-4o-mini) was explicitly instructed to output a structured JSON object containing an assessment for each bullet point in the generated text. This assessment included a "faithfulness score" (rating how well the statement is supported by the provided context) and a "citation correctness" flag (checking if the cited source actually contains the supporting information). An "overall confidence score" for the section was calculated based on the average faithfulness score of its bullets.

3.5.2 Citation Validation

As part of the automated verification process, the verification LLM evaluated the precision of the citations. It checked if each [Document name: Page number] citation pointed to content within the retrieved external context snippets, and if [Internal Data] citations aligned with the provided internal metrics. Although this was performed by the verification LLM itself based on the prompt instructions, the structure of the verification output (JSON with per-bullet checks) allowed for programmatic validation of citation accuracy.

3.5.3 Re-generation Loop

The methodology incorporated an optional re-generation loop based on the verification results. If the "overall confidence score" returned by the verification LLM for a section fell below a predefined verification threshold (e.g., a threshold of 3.5 out of 5), the system could attempt to re-generate the section. The generation LLM was invoked again for the same Key Issue, using the same inputs (internal data, retrieved context), but potentially with slightly adjusted parameters or instructions. The results (generated text, verification details, retrieved sources) of each attempt were stored.

3.5.4 Structured Output Format

The system was designed to present the generated ESG profile draft to the analyst in a clear, structured format, closely resembling the target output style of the examples used in the prompt (e.g., distinct SWOT sections for each Key Issue). Crucially, all citations were designed to be clearly displayed alongside the corresponding statements, allowing the analyst to easily trace the information back to its source. The output for each Key Issue included the text from all generation attempts and their corresponding verification results, providing transparency into the pipeline's performance and decision-making (e.g., why a re-generation was triggered or why

the final version might still have a lower score). This structured and transparent output format was considered fundamental to building trust and facilitating efficient review.

3.5.5 Analyst Review and Feedback Mechanism

Despite the automation and verification layers, the generated profile was considered a draft. The sustainability analyst was intended to perform the final review, applying their domain expertise and critical judgment to:

- Validate the factual accuracy and complexity of the generated text, particularly for statements flagged by the verification layer or where the overall confidence score is low.
- Correct any remaining errors, inconsistencies, or awkward phrasing.
- Add insights or interpretations that the AI may have missed.
- Ensure that the profile aligns with the overall investment narrative or specific requirements.

The system was designed to provide tools for the analyst to easily edit and annotate the draft. Importantly, this methodology included a feedback loop: analyst edits and corrections can be captured and stored. This feedback data was considered invaluable for future system improvements.

This human-in-the-loop approach was intended to ensure that the final ESG profile maintained high quality while leveraging AI for efficiency, positioning the system as an assistant rather than a replacement for the analyst.

3.6 Evaluation Strategy Outlook

A rigorous evaluation was considered essential to assess the effectiveness and efficiency of the proposed AI-assisted ESG profiling system. Although detailed evaluation results will be presented in a subsequent chapter, the planned strategy involved measuring performance in several key dimensions.

- **Factual Correctness:** Assessing the accuracy of statements generated by the LLM against the source documents and internal data. This is quantitatively measured by the "faithfulness score" from the automated verification step and qualitatively assessed during analyst review.

- **Citation Accuracy:** Evaluating whether citations correctly point to supporting evidence in the source material. This is quantitatively measured by the "citation correctness" flag from the automated verification step and qualitatively assessed during analyst review.
- **Coverage and Completeness:** Measuring how well the generated profile covers the required Key Issues and relevant aspects based on the input data, qualitatively evaluated by analysts.
- **Coherence and Relevance:** Assessing the logical flow, readability, and relevance of the generated text to the specific context of ESG and the key issue, qualitatively assessed by analysts. The "relevance" flag from the automated verification provides a quantitative indicator at the bullet level.
- **Efficiency Gains:** Quantifying the reduction in time and effort required for analysts to produce a comprehensive ESG profile using the system compared to the traditional manual process, measured through time-tracking and analyst feedback.
- **Analyst Usability and Validation:** Gathering qualitative feedback from sustainability analysts on their experience using the application, the value of the generated output as a draft, and the utility of features such as source traceability.

Although there are standard RAG evaluation frameworks (such as RAGAS, ARES, or TruLens) and provide valuable metrics for components such as retrieval accuracy and answer faithfulness, this project adopted a more tailored evaluation methodology. This decision was driven by the highly specific nature of the task (generating structured SWOT analysis for predefined ESG Key Issues, integrating distinct internal and external data sources) and the critical requirement for qualitative validation by domain experts (sustainability analysts) who represent the end-users. Therefore, the evaluation focused on metrics directly relevant to the project's goals and incorporated expert human judgment alongside automated checks derived from the implemented verification layer.

Methodologies and metrics from established RAG evaluation frameworks were adapted (e.g., faithfulness, citation correctness) where applicable and integrated into the custom evaluation process, complemented by qualitative analyst feedback and quantitative time-tracking to provide a comprehensive assessment of the system's performance and value in a real-world context.

4 Implementation

This chapter details the practical realization of the RAG methodology outlined in Chapter 3. It describes the system's architecture, the specific technologies selected for each component, and how these components were integrated, and the challenges encountered during development. The aim was to provide a comprehensive overview of the implemented system, bridging the gap between the theoretical methodology and its functional application for AI-assisted ESG profiling. The complete source code, including all scripts, data, outputs, configuration files, and instructions for reproducing the environment, is available in the project's GitHub repository [123].

4.1 System Architecture

The implemented system followed a modular architecture designed to support flexibility and maintainability. Figure 4.1 illustrates the key components and their interactions.

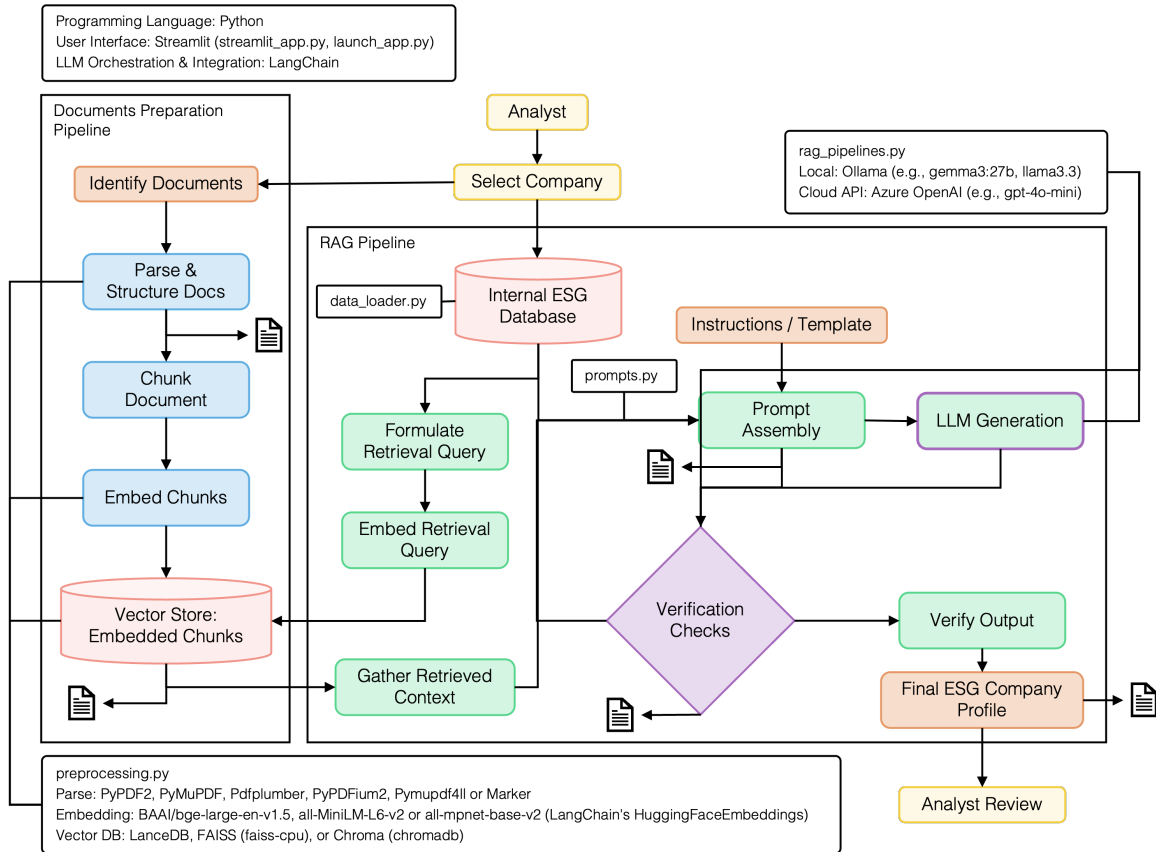


Figure 4.1: Implemented System Architecture Diagram.

The architecture was orchestrated primarily within the "streamlit_app.py" script and the core logic in "rag_pipeline.py", managing the overall workflow, and reacting to user input from the Streamlit User Interface.

Data enters the system from two main sources: External Documents (PDFs) and Internal ESG Data, both stored within the local Data Repository. The Documents Preparation Pipeline encompasses the pre-processing steps handled by the "preprocessing.py" module. This module reads PDFs from the Data Repository, utilizes a PDF Parser to extract text and structure, interacts with an Embedding Model to create vector representations of the document content, and builds/manages the Vector Store where these embeddings and original text/metadata chunks are stored.

The RAG Pipeline component within the Pipeline Orchestrator retrieves relevant information from the Vector Store. It also integrates Internal ESG Data loaded via "data_loader.py" and uses predefined Prompt Templates from "prompts.py". These inputs are combined during Prompt Assembly to form the final query sent to the Generation LLM (supporting different backend services like Ollama or Azure). The LLM produces the generated text output.

An optional Verification Module is integrated downstream of the Generation LLM. This module, also within "rag_pipeline.py", utilizes a Verification LLM (potentially distinct from the generation model, accessed via the same LLM Provider Gateway) and specific verification prompts to perform Verification Checks on the generated output against the original source context. Depending on the verification result, the Pipeline Orchestrator may trigger a re-generation attempt.

The final validated or latest generated text forms the Final ESG Company Profile, which is saved back to the Data Repository and presented to the user via the Streamlit User Interface for Analyst Review.

4.2 Technology Stack

The project was primarily implemented in Python, using a combination of open-source libraries and cloud service APIs. The key technologies used were the following:

- **Programming Language:** Python 3.9+
- **User Interface:** Streamlit for building the interactive web application.
- **LLM Orchestration & Integration:** LangChain framework to manage document loading, text splitting, embedding, vector store interaction, and chaining LLM calls.
- **PDF Parsing:** Multiple libraries were integrated and evaluated, including PyPDF2, PyMuPDF, pdfplumber, PyPDFium2, pymupdf4llm, and Marker for extracting text and structural information from PDFs.
- **Embedding Models:** Hugging Face sentence-transformers library (integrated and evaluated models like BAAI/bge-large-en-v1.5, all-mpnet-base-v2, all-MiniLM-L6-v2) integrated via LangChain's HuggingFaceEmbeddings.
- **Vector Databases:** Local vector store implementations tested and supported include LanceDB, FAISS (faiss-cpu), and Chroma (chromadb). LanceDB was favored for its ease of use and versioning capabilities during development.
- **Large Language Models:**
 - **Local:** Ollama for running models locally (e.g., gemma3:27b or llama3.3), integrated via langchain-ollama.
 - **Cloud API:** Azure OpenAI Service to access models hosted on Azure (e.g., gpt-4o-mini), integrated via langchain-openai.
- **Dependency Management:** Conda for environment management and Pip for package installation ("requirements.txt").

4.3 Implementation of Core Components

4.3.1 Data Repository and Loading

The system operates on a local file-based data repository, structured under the base "data/" directory. The external PDF documents for each company are organized into dedicated sub-directories, following the pattern "data/documents/<issuer_id>". Structured internal ESG data are stored in JSON format within the corresponding issuer directories "data/metrics/<issuer_id>/KEY_ISSUE_BREAKDOWN.json". And the static reference files are located in the main "data/" directory, such as GICS definitions (GICS_DEFINITIONS.csv), proprietary ESG ratings (ESG_RATING_GICS_COUNTRY.csv), Key Issue examples for prompting (KI_EXAMPLES.json), and Key Issue keywords for retrieval (KI_KEY_WORDS.json).

The "data_loader.py" module contains functions to read these files using the standard Python file I/O and the Pandas library for CSV files. It is responsible for parsing the internal JSON data and structuring them into Python dictionaries and lists for convenient access by the RAG pipeline during prompt assembly. The company selection feature in the Streamlit UI utilizes the "data_loader.py" functions to load initial company information and list available documents and metrics based on the existing directory structure.

4.3.2 Preprocessing Pipeline

The pre-processing module ("preprocessing.py") is responsible for transforming raw PDF documents into a format suitable for vectorization and retrieval.

PDF Parsing

A significant portion of the implementation effort involved the integration and evaluation of various PDF parsing libraries. Different parsers offer trade-offs between speed, accuracy, and the ability to preserve document structure (like headings, tables, etc., which is particularly relevant for generating Markdown). The system includes wrappers around PyPDF2, PyMuPDF, pdfplumber, PyPDFium2 (primarily for standard text extraction) and direct integration with pymupdf4llm and Marker for generating structured Markdown output. The Marker library, although requiring additional model dependencies and sequential processing per document, proved effective at converting complex reports into structured Markdown, which can enhance the context provided to the LLM. Configuration options in the Streamlit UI allow the user to select the desired parser, enabling evaluation of their performance and output quality on specific document types. Parsed content for each file, accurately tagged with original page

numbers, is optionally saved as JSON files (e.g., in "data/documents/<issuer.id>/process/") for inspection and to avoid redundant re-parsing.

Text Chunking Strategy

As detailed in the Methodology (Section 3.3.2), the implemented text chunking strategy was based on page boundaries. Each page extracted by the PDF parser (regardless of its content length) was treated as a single distinct chunk. This was implemented directly during the parsing step within "preprocessing.py", ensuring that each resulting Document object produced by the parser corresponded to a single page and accurately retained its page number metadata. This simple page-based approach was chosen for several practical and domain-specific reasons, including ease of implementation across diverse document formats, reliable mapping for citation accuracy, and the common structuring of corporate reports with page-level content coherence.

Vectorization

Following the page-based chunking, the text content of each page was transformed into a dense numerical vector, or embedding. This process was performed using a pre-trained sentence transformer model loaded via LangChain's HuggingFaceEmbeddings. Models like BAAI/bge-large-en-v1.5 were frequently selected for their strong performance on semantic similarity tasks relevant to retrieval. The implementation handled model loading, specifying the execution device (cpu, cuda or mps), and applying specific encoding arguments (such as normalization, which was beneficial for models like BGE).

Vector Store Management

The implementation supported several vector database backends (FAISS, Chroma, LanceDB), allowing flexibility in storage choices. The process involved two main stages orchestrated within "preprocessing.py":

1. Building individual vector stores for the chunks derived from each source PDF file for a given issuer. This modular approach simplified the debugging and allowed processing of files independently. This step leveraged Python's ThreadPoolExecutor for parallel processing across multiple PDF files when using faster parsers, although computationally intensive or LLM-dependent parsers like Marker were processed sequentially for stability.
2. Creating or loading a single combined vector store for the issuer. This combined store aggregated all document chunks from the individual stores associated with the currently selected source PDF files. A manifest file, stored alongside the combined vector store,

tracked which original PDF files were included in its creation. This enabled the system to detect if the set of source files has changed since the last run and automatically triggered a rebuild of the combined store if necessary or if explicitly requested via configuration flags.

The "process_issuer_pdfs_individual_then_combined" function orchestrated this aggregation process, ensuring that the RAG pipeline consistently interacted with a unified, up-to-date, and efficiently searchable knowledge base derived from all relevant source documents for the selected company.

4.3.3 RAG Pipeline Implementation

The core RAG logic resides in the "rag_pipeline.py" module, responsible for fetching relevant context and generating profile sections.

LLM Initialization

The "initialize_llm" function handled the connection to the chosen Large Language Model provider, supporting both local inference via Ollama (OllamaLLM) and cloud-based access via the Azure OpenAI Service API (AzureChatOpenAI) using LangChain wrappers. It included logic to load Azure credentials securely from the "credentials.env" file and performed basic checks for Ollama server availability to provide user feedback. The configuration parameters were specified in "config.py", such as the specific model name or deployment, temperature, and request timeout, were passed dynamically during initialization. Other Ollama-specific parameters, such as "num_ctx" (context window size) and "num_gpu" were also adjustable.

Prompt Design and Management

Prompt templates were defined in the "prompts.py" module using LangChain's PromptTemplate. The core generation prompt was implemented as a complex, multi-part template. It was designed to reliably guide the LLM by incorporating detailed system instructions, defining the LLM's role as an ESG analyst, outlining a structured task decomposition (inspired by Chain-of-Thought and Least-to-Most), specifying the desired output format (SWOT structure with strict citation rules), including few-shot examples loaded from "K1EXAMPLES.json" (distinct by Key Issue), and dynamically inserting the formatted internal ESG data and the retrieved document content. The retrieval prompts, used internally to formulate queries for the vector store, and verification prompts, used to instruct the verification LLM, were also defined as specific templates.

The "rag_pipeline.py" module handled the dynamic formatting of these prompts before sending them to the respective LLMs. For debugging and traceability, the final prompts sent to the generation and verification LLMs were optionally saved to disk (in "data/documents/<issuer_id>/prompts/" and "data/documents/<issuer_id>/prompt_verification/").

Information Retrieval Execution

Retrieval was triggered for each Key Issue being analyzed. The corresponding retrieval prompt was embedded using the initialized embedding model and the resulting query embedding was then used to perform a similarity search against the combined vector store for the selected issuer, using the appropriate method for the chosen vector store type (e.g., LanceDB table search or FAISS/Chroma retriever invoke). The implementation retrieved the top k most semantically similar document chunks (pages), ensuring that their original text content and associated metadata (particularly the source document name and page number) were returned. The retrieved content was then formatted into a structured string for inclusion in the generation prompt.

Generation Execution

The "generate_profile_section" function orchestrated the text generation for a single ESG Key Issue for a specific attempt. It first performed document retrieval based on the Key Issue, formatted the complex generation prompt using the retrieved text, internal data, and loaded examples, and then sent this assembled prompt to the configured generation LLM using the "llm.invoke()" method, and the raw text output received from the LLM was captured. The implementation included a simple retry mechanism for the LLM invocation call in case of transient network issues or API errors, attempting the call multiple times before failing. The generated text, along with the specific retrieved documents and prompts used for this attempt, was returned for subsequent verification or storage.

4.3.4 Post-Generation Verification Layer Implementation

The verification layer, implemented within "rag_pipeline.py", aimed to automatically assess the factual accuracy and citation correctness of the generated text output by the Generation LLM.

If verification was enabled in the configuration, a separate verification LLM was initialized and the text generated for a specific Key Issue section by the Generation LLM was then passed to the "verify_section" function. This function first attempted to extract distinct statements or bullet points from the generated text, and then constructed a specific verification prompt that provided the verification LLM with the list of extracted bullets, the exact same internal data,

and the exact same retrieved external context that the generation LLM received. The prompt instructed the verification LLM to act as a meticulous fact-checker and analyze each individual bullet point against the provided context. It mandated that the verification LLM output its assessment in a structured JSON format, including a faithfulness score (rating how well the statement is supported by the provided context on a 1-5 scale) and a citation correctness flag (True/False indicating if the cited source actually contains the supporting information) for every input bullet.

This score, along with the detailed per-bullet evaluations, was returned and the main "generate_full_profile" function orchestrated a re-generation loop for the Key Issue, which was used if the overall confidence score from the verification step fell below a predefined verification threshold. This loop ran for a configurable number of attempts per key issue. The results (generated text, detailed verification data, retrieved sources, prompts) of each generation and verification attempt for a Key Issue were meticulously stored in the final output structure.

4.3.5 Structured Output and Saving

The results for each processed Key Issue, including the generated text from each generation attempt, the detailed assessment from the verification step (if enabled), the list of retrieved documents used as context, and the full prompts sent to the LLMs, were collected into a structured Python dictionary. This dictionary, along with essential metadata about the pipeline run (issuer ID, generation timestamp, complete configuration parameters used, list of source files processed), was formatted and saved as a JSON file in the "output/" directory. This standardized, comprehensive output format was designed to be easily parsed and displayed by the Streamlit UI for analyst review, providing complete traceability back to the configuration, inputs, intermediate steps, and verification outcomes for each generated section.

4.4 User Interface Implementation

The user interface (UI) was implemented using the Streamlit framework ("streamlit_app.py") and served as the primary interaction point for the sustainability analyst. It abstracted away the underlying pipeline execution and file management complexities, offering an intuitive workflow.

The main page of the application (Figure 4.2) allows the analyst to select the target company from a drop-down menu. Displays the current available source documents for processing for the selected company and provides an interface for uploading new PDF documents. Key pipeline configurations, such as the choice of PDF parser (e.g., Precise (Marker), Medium (PyMuPDF4LLM), Fast (PyPDFium2)), and LLM models for generation and verification, can be adjusted directly on this page. In addition, advanced settings are also available for the

retrieval depth and verification parameters. A "Run Pipeline" button triggers the entire pre-processing and RAG analysis, with progress updates displayed in a process log area.

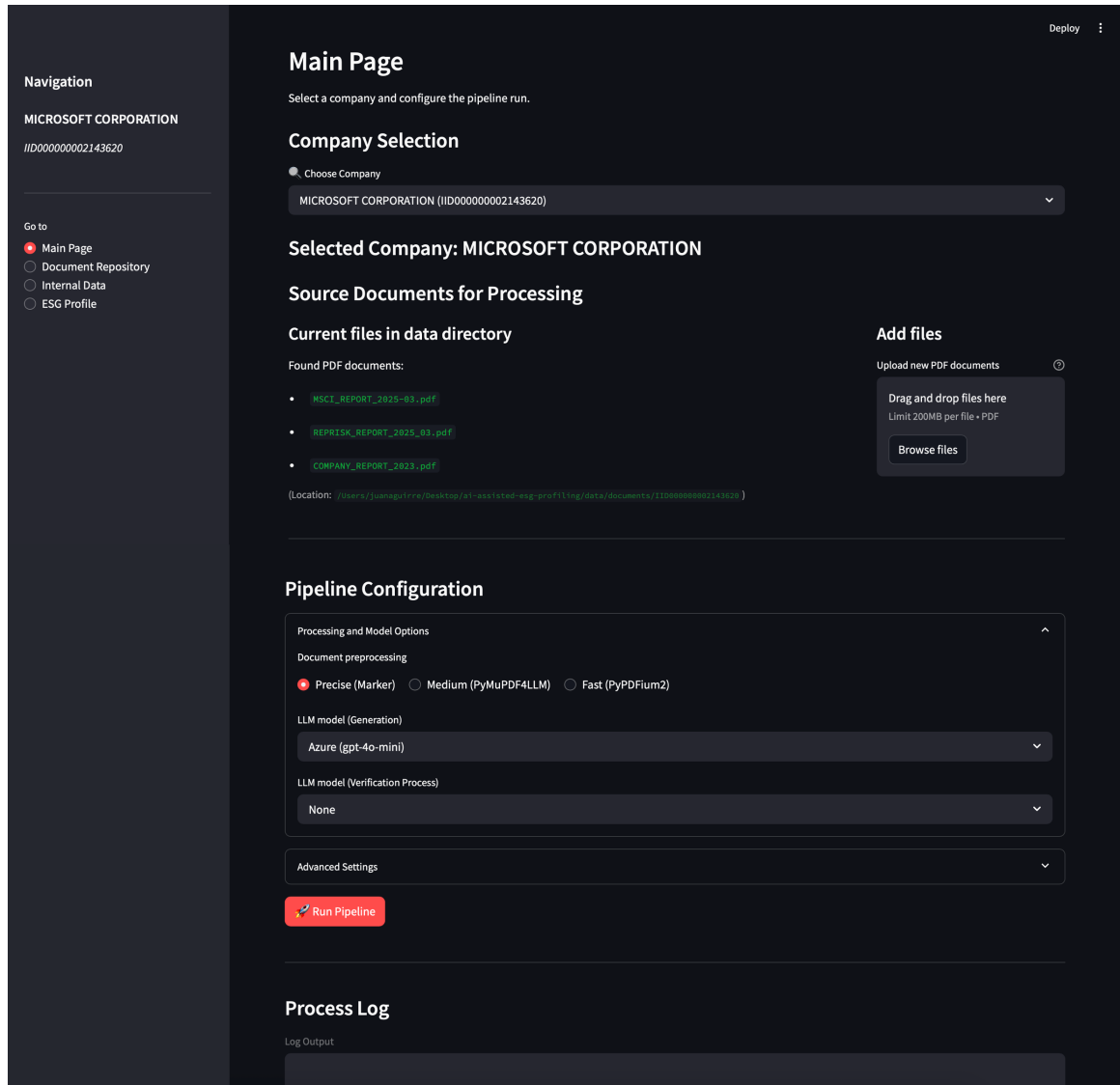


Figure 4.2: Main page of the Streamlit UI showing company selection, source document management, and pipeline configuration options.

The UI incorporates dedicated navigation pages, accessible via a sidebar, allowing analysts to browse and inspect various data and outputs.

The document repository view page (Figure 4.3) displays the processed content of the source PDF documents. Analysts can select a specific processed file and view its content page by page, with an option to filter by the original page number. This is crucial for understanding how the source material was parsed and chunked.

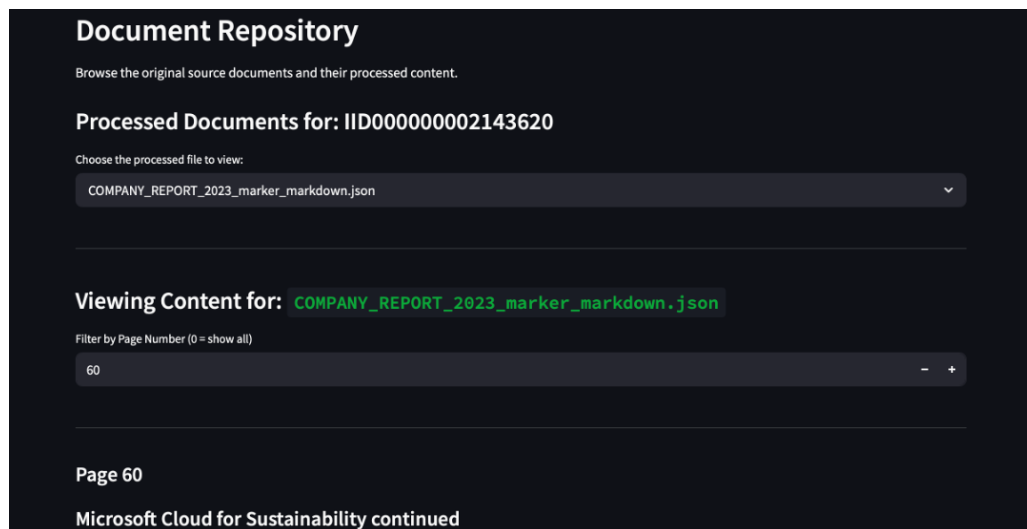


Figure 4.3: Document Repository view in the Streamlit UI.

The internal data page (Figure 4.4) presents the formatted internal ESG metrics and company information that are loaded for the selected company and subsequently used as context in the generation prompts.

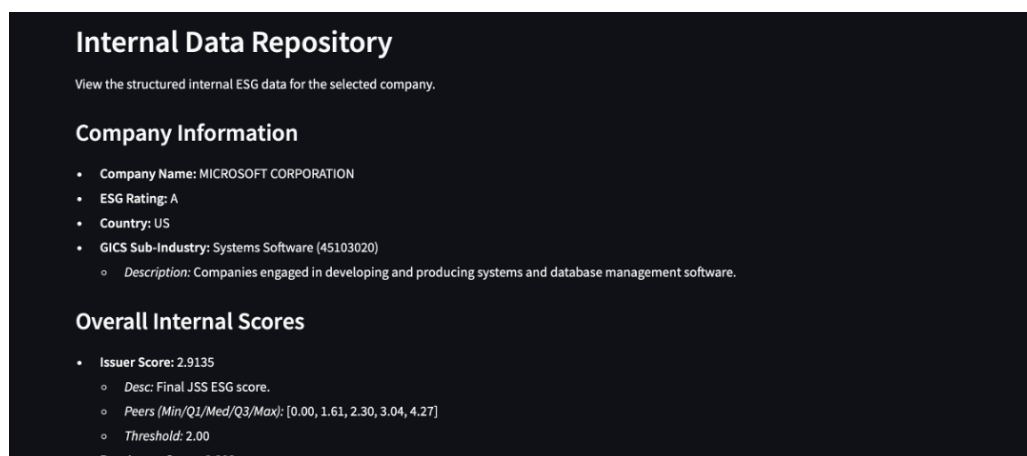


Figure 4.4: Internal Data page.

The ESG profile page is where the final AI-generated ESG profile summaries are viewed (Figure 4.5). Analysts can select from previously generated profiles (which are named based on generation time, LLM used, parser, etc.). For a chosen profile, the UI displays the generated SWOT analysis for each Key Issue. Critically, it also shows the detailed verification results from the automated check, including the Overall Confidence score for the section and the faithfulness, correctness of the citation, and relevance flags per bullet. Options to view the exact source snippets retrieved and used as context, and the full generation/verification prompts sent to the

LLMs, are provided for complete transparency and debugging.

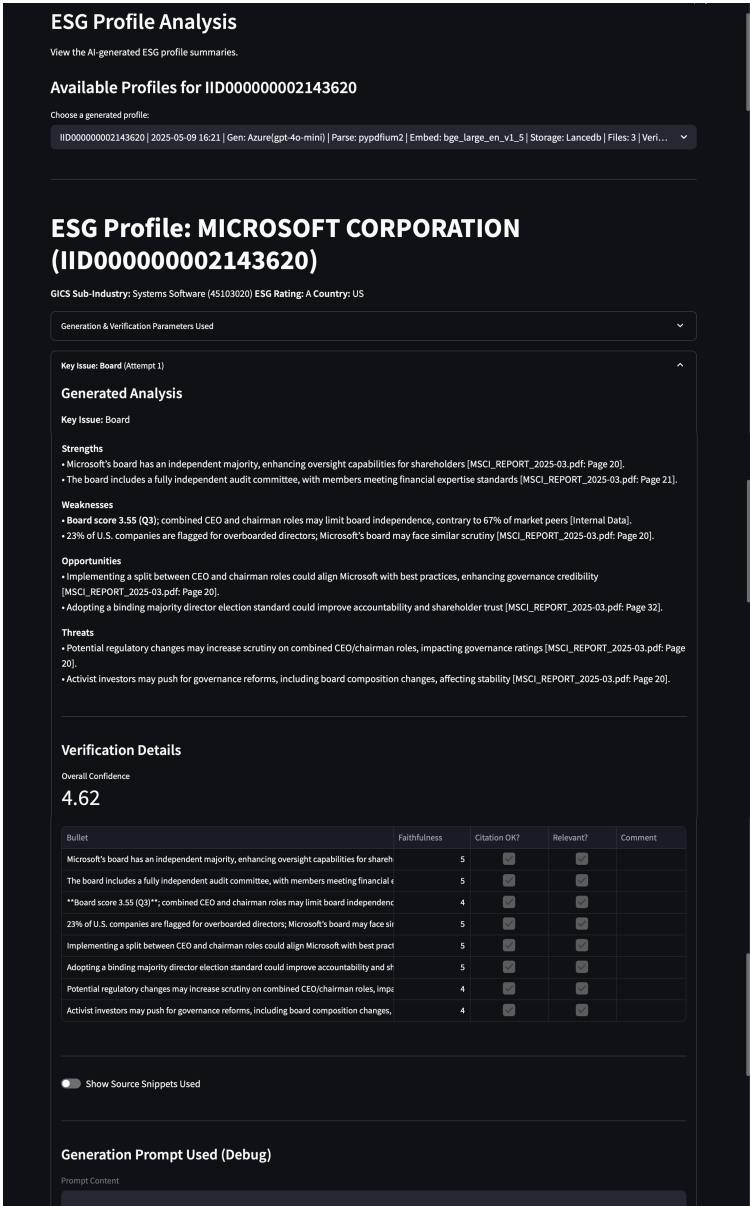


Figure 4.5: ESG Profile Analysis page.

The Streamlit UI was designed to enhance the analyst’s workflow by providing a user-friendly environment to control the RAG pipeline and to quickly access and validate the AI-generated outputs alongside their original sources, verification details, and the logic steps taken by the system. This transparency was considered key to fostering analyst trust and facilitating efficient review.

4.5 Infrastructure and Deployment Considerations

The system was developed and tested primarily on a standard laptop computer. This development environment, while convenient for initial coding, posed significant practical limitations, particularly regarding the computational resources required for running larger, more capable Large Language Models locally. Although the implemented architecture supported running models via Ollama for local inference, the GPU memory (VRAM) and processing power of the development machine were often insufficient to host state-of-the-art open-source models (like larger Gemma, Llama, or DeepSeek variants) or handle the large context windows required for complex, long-document analysis within reasonable timeframes. Furthermore, performance variability and configuration complexities were notable challenges encountered in the local environment.

To mitigate these limitations during the development and testing phase and ensure a stable platform for building and validating the complex prompt logic and verification mechanisms, the decision was made to leverage the Azure OpenAI Service API as the primary LLM provider. Accessing models like gpt-4o-mini via a cloud API provided a more consistent, performant, and scalable environment for development, even if gpt-4o-mini is not necessarily the largest or most performant model currently available compared to some state-of-the-art open-source alternatives discussed in Chapter 2. This pragmatic choice enabled rapid iteration and reliable testing of the pipeline's core functionality.

Critically, the underlying architecture and code implementation were designed with the target production environment in mind, which is expected to be an internal corporate infrastructure equipped with sufficient computational resources. The project's modularity, demonstrated support for different LLM providers (including local/internal options like Ollama or direct integration libraries), and the focus on managing data and vector stores locally or within a controlled internal network means that the system is readily adaptable. The intention is for future deployment on corporate servers with adequate GPUs, which would enable the use of more powerful open-source models (e.g., larger Gemma, Llama, DeepSeek variants, or other models identified in the State of the Art that meet the specific requirements of the firm regarding performance, cost, and licensing) running within the private network. This aligns with the project's fundamental goals of ensuring data privacy, maintaining control over the process, and potentially achieving more predictable and lower long-term operational costs compared to extensive public cloud API usage. The current implementation structure, with its abstracted LLM interaction through a provider gateway and localized data processing, facilitates this planned transition.

4.6 Key Implementation Challenges and Solutions

Implementing the RAG pipeline presented several key technical and practical challenges:

- **Package Compatibility and Setup:** Integrating numerous Python libraries with complex dependencies for PDF parsing, embeddings, vector stores, and LLM interactions (especially when using a framework like LangChain that orchestrates many components) led to frequent version conflicts and installation complexities. Managing dependencies rigorously with Conda environments and a detailed "requirements.txt" was essential, though resolving specific library conflicts and ensuring compatibility between different components required considerable trial and error.
- **PDF Parsing Evaluation and Selection:** Identifying and implementing the optimal PDF parser for diverse corporate reports was challenging. Different libraries varied significantly in their speed, accuracy, and ability to handle complex layouts, tables, and images while preserving text fidelity and metadata (like page numbers). Extensive testing with sample documents representing the expected variety of input sources was necessary to empirically determine the most reliable and performant parsers (such as Marker for structured and robust output or, PyMuPDF4LLM or PyPDFium2 for faster results) suitable for the project's requirements.
- **LLM Performance and Context Management on Local Hardware:** Running sophisticated LLMs locally via Ollama on a standard laptop proved difficult due to hardware constraints, as already described in the previous section. Larger models capable of delivering high-quality results (e.g., gemma3:27b or llama3.3) often exceeded the available GPU memory (VRAM), while smaller models (e.g., gemma3:1b) frequently struggled with the complexity of the task and the large context window size required to accommodate the extensive prompts (instructions, internal data, retrieved context). Issues with configuring Ollama's xontext window (num_ctx) setting to match model and prompt requirements, and ensuring the entire prompt content fit reliably, were encountered. This practical limitation directly necessitated the decision to use the Azure OpenAI API during the development phase to ensure a stable and performant environment for building and validating the core pipeline logic, as discussed in [Section 4.5](#).
- **Prompt Engineering Refinement:** Designing the complex, multi-part prompts to reliably guide the LLM to produce accurate, structured, and cited output required significant iterative testing and refinement. Ensuring the model correctly interpreted and followed detailed instructions (including citing rules, formatting constraints, using specific data

sources), effectively utilized all provided context (both internal data and retrieved text), and avoided common LLM pitfalls (hallucinations, ignoring negative constraints, deviating from the required format) was challenging. Incorporating detailed task breakdown instructions, explicit constraints, and providing effective few-shot examples proved critical to improving output quality and consistency.

- **Verification Layer Robustness:** Implementing the automated verification step using a second LLM call presented challenges in designing a prompt that reliably instructed the verification LLM to act as a strict fact-checker. Ensuring it consistently outputted valid and accurately structured JSON assessments, correctly evaluated the faithfulness and citation correctness of each generated bullet based only on the provided context, and not rely on its internal knowledge was key. Parsing the JSON output from the verification LLM and correctly interpreting the scores and flags for use in the re-generation loop added another layer of implementation complexity.
- **Streamlit Development and UI Logic:** Building the interactive user interface with Streamlit involved standard web application development challenges, such as managing application state across user interactions, handling file uploads securely to the local data directory, and structuring the back-end pipeline logic ("rag_pipeline.py") to be called effectively from the UI. Displaying verbose process logs, presenting complex structured JSON output from the pipeline runs, and creating intuitive navigation between different views (Document Repository, Internal Data, ESG Profile) required careful use and styling of Streamlit components.

These challenges, covering software architecture, data processing, LLM interaction, and user interface design, highlighted the complexities inherent in building robust AI pipelines that integrate multiple external tools and services in a real-world application context, requiring flexibility, systematic testing, and iterative refinement throughout the development process.

5 Results Analysis

5.1 Introduction

This chapter presents the results derived from the implementation and evaluation of the AI-assisted ESG profiling system. Building upon the methodology detailed in Chapter 3 and the implementation specifics outlined in Chapter 4, this analysis assesses the system performance across key dimensions: efficiency, factual accuracy, citation correctness, output quality, and overall utility from an ESG analyst’s perspective. As was discussed in the methodology, given the domain-specific nature of the task, a tailored evaluation approach was used, incorporating automated metrics derived from the verification layer and qualitative human expert assessments.

5.2 Evaluation Methodology

The evaluation of the implemented RAG system was designed to measure its effectiveness in generating accurate and verifiable ESG profiles and its efficiency in accelerating the manual process. The methodology involved both quantitative and qualitative components.

The quantitative metrics were mainly derived from the automated post-generation verification layer for each generated Key Issue section:

- **Overall Confidence Score:** An aggregate measure (average faithfulness score) indicating the verification LLM’s confidence in the accuracy and contextual grounding of the generated text for a Key Issue.
- **Faithfulness Score:** Per-bullet score (1-5) assessing if the generated statement is directly supported by the provided internal data or retrieved external context.
- **Citation Correctness:** Per-bullet flag (True/False) verifying if the statement’s citation accurately points to a supporting source within the provided context.
- **Relevance:** Per-bullet flag (True/False) indicating if the statement is relevant to the specific Key Issue being analyzed.

In addition, efficiency was assessed by quantifying the time savings achieved when analysts used the automated pipeline compared to performing the full profiling process manually.

The qualitative assessment involved expert review by sustainability analysts using the Streamlit

interface. Analysts evaluated:

- The general quality, coherence, and relevance of the text generated as a draft.
- The usefulness and usability of the structured output format and integrated traceability features (citations, viewing sources).
- The perceived reduction in the manual effort for data gathering and initial drafting.
- Identification of instances where the AI output was particularly insightful or where it required significant correction or interpretation.

This combined approach provided a holistic view of the system’s performance, capturing both its technical accuracy and its practical value in a real-world ESG analysis workflow.

5.3 Quantitative Results: Efficiency and Accuracy

5.3.1 Efficiency Gains

A primary objective of this project was to reduce the manual effort and time required to create an ESG company profile. The traditional manual process involved searching, aggregating, and synthesizing information from disparate sources for a single company. This process could often take an analyst a full day or more, depending on the complexity of the company and the availability of the data. However, the implemented RAG pipeline automated the most time-consuming steps of data ingestion, pre-processing, and initial synthesis, leading to substantial improvements in efficiency, as was confirmed by preliminary time tracking during pilot testing and feedback from sustainability analysts.

The exact processing times for the automated pipeline depended on several factors, including the number and complexity of the input PDF documents, the selected PDF parser, the size of the vector store, the specific LLM, and the hardware used to run it. All tests reported here were conducted on a MacBook Pro (M3 Pro), with generation and verification steps utilizing the Azure GPT-4o mini model. Table 5.1 presents the pre-processing and RAG pipeline execution times for three profiled companies using different PDF parsers. The RAG pipeline time was reported as an average per Key Issue, as the total time for this stage scaled with the number of Key Issues analyzed for a given company (11 for Apple Inc., 10 for Wells Fargo & Co., and 9 for Microsoft Corp. in these tests). The execution times of these three companies are shown in the notebook “3_example_results.ipynb” which can be found in the repository associated with this thesis [123].

Table 5.1: Pipeline Execution Times by Company and PDF Parser

Company	PDF Parser	Preprocessing Time (PDFs → Text → VectorDB)	RAG Pipeline Time (Avg. per KI)	Total Automated Time (Approximate for all KIs)
Apple Inc.	Marker	5 min 48 sec	~20.3 sec/KI	~9 min 31 sec
Apple Inc.	PyMuPDF4LLM	4 min 52 sec	~20.7 sec/KI	~8 min 40 sec
Apple Inc.	PyPDFium2	1 min 7 sec	~15.5 sec/KI	~3 min 57 sec
Wells Fargo & Co.	Marker	4 min 10 sec	~16.7 sec/KI	~6 min 57 sec
Wells Fargo & Co.	PyMuPDF4LLM	2 min 6 sec	~17.2 sec/KI	~4 min 57 sec
Wells Fargo & Co.	PyPDFium2	0 min 59 sec	~13.6 sec/KI	~3 min 15 sec
Microsoft Corp.	Marker	4 min 24 sec	~17.5 sec/KI	~7 min 0 sec
Microsoft Corp.	PyMuPDF4LLM	2 min 37 sec	~19.5 sec/KI	~5 min 33 sec
Microsoft Corp.	PyPDFium2	0 min 56 sec	~12.8 sec/KI	~2 min 51 sec

As can be seen in Table 5.1, pre-processing times varied significantly based on the parser. The Marker parser, which performs more complex structural analysis, was generally the most time-intensive. In contrast, PyPDFium2, focused on faster plain text extraction, showed considerably reduced pre-processing times. PyMuPDF4LLM offered a balance between these two. The RAG pipeline time (generation and verification per Key Issue) was more consistent across parsers for a given company, due to the fact that it is run in Azure hardware and not locally, showing some variation. This variation may have been due to slight differences in the extracted context quality or length influencing LLM processing. For example, the more structured output from Marker might occasionally have led to slightly longer LLM processing if the context provided is richer or more verbose, whereas the faster, potentially less structured output from PyPDFium2 might have led to quicker LLM responses.

It should be emphasized that these timings, particularly for PDF parsing, were strongly influenced by the underlying hardware. The planned deployment on corporate infrastructure with GPU resources is expected to reduce processing times substantially, especially for computationally demanding parsers like Marker. Even so, with the current setup, using Marker parsing, a full company profile took approximately 6 to 10 minutes for the three examples evaluated.

This automation of initial data handling and draft generation translated into a significant reduction in the time that an analyst needs to produce a final ESG company profile. Analysts reported that the end-to-end profiling time, from process initiation to final version, dropped from approximately one day (roughly 8 hours) to about one hour and a half (90 minutes). This represented an efficiency gain of roughly **80%**. The system thus shifted the analyst's focus from tedious data collection to activities of higher value such as interpretation, strategic analysis, and final refinement of the AI-generated draft.

5.3.2 Accuracy and Verification Metrics

The automated post-generation verification layer provided quantitative insight into the factual accuracy and citation correctness of the generated text. The generation and verification steps for the results presented here were performed using Azure GPT-4o mini. Table 5.2 summarizes these metrics, averaged across all Key Issues for each profiled company and PDF parser combination. The metrics included the average Overall Confidence Score (an aggregate measure from the verification LLM, per Key Issue), the percentage of citations considered correct, and the percentage of bullet points determined relevant to the Key Issue by the verification LLM.

Table 5.2: Automated Verification Metrics Summary by Company and PDF Parser

Company	PDF Parser	Avg. Overall Confidence Score	Citation Correctness (%)	Relevance (%)
Apple Inc.	Marker	4.76	98.86%	100%
Apple Inc.	PyMuPDF4LLM	4.76	100%	100%
Apple Inc.	PyPDFium2	4.68	98.86%	100%
Wells Fargo & Co.	Marker	4.69	100%	100%
Wells Fargo & Co.	PyMuPDF4LLM	4.63	100%	100%
Wells Fargo & Co.	PyPDFium2	4.71	98.75%	100%
Microsoft Corp.	Marker	4.71	98.61%	100%
Microsoft Corp.	PyMuPDF4LLM	4.70	100%	100%
Microsoft Corp.	PyPDFium2	4.79	100%	100%

Across the profiled companies and parser configurations (Table 5.2), the average Overall Confidence Scores from the verification LLM were consistently high, generally ranging from approximately 4.6 to 4.8 out of 5. This indicated that on average, the generated text was considered highly faithful to the source material provided by the verification LLM, and consistently surpassed the predefined operational threshold of 3.5.

A deeper look at the per-bullet evaluations revealed the system’s strengths and areas for potential improvement:

- High Relevance and Citation Correctness:** As shown in Table 5.2, Relevance scores were consistently 100%, indicating that generated bullet points were correctly identified by the verification LLM as pertinent to the specific Key Issue. Citation Correctness was also very high, typically above 98.5%, with many configurations achieving 100%. The few instances of citation errors (e.g., Apple Inc. with PyPDFium2 for Human Capital Development, or Microsoft Corp. with Marker for Tax Transparency) were isolated and often related to minor misattributions rather than complete fabrications. This confirms

the general effectiveness of the RAG process in retrieving relevant context and the LLM's ability to follow explicit citation instructions.

- **Strong Faithfulness, with Occasional Complexity or Factual Errors:** Average Faithfulness Scores per bullet were also strong, typically ranging between 4.60 and 4.81. While many bullet points received a perfect Faithfulness Score of 5 (indicating direct and accurate support from sources), instances of scores like 4 were observed. Checker comments for these (visible in the full JSON outputs) often indicated that the statement was a plausible inference or a slight paraphrase of the source, rather than a direct quote or strictly verifiable fact, which the verification LLM might penalize slightly. More critically, the verification process successfully identified instances of factual errors or misinterpretations in the generated text, assigning lower Faithfulness Scores (e.g., a score of 2 for a statement misrepresenting a median value for Microsoft Corp.'s Human Capital Development with both PyMuPDF4LLM and PyPDFium2 parsers, or another score of 2 for Wells Fargo's Access to Finance with PyMuPDF4LLM).
- **Verification Effectiveness:** The instances in which the verification LLM assigned low faithfulness scores (2 or 3) demonstrated the critical value of this layer. It successfully caught specific errors, such as misreporting numerical values, making incorrect comparisons based on internal data, or making claims contradicted by the provided text. This validated the methodology's premise that an automated check is necessary to mitigate residual LLM inaccuracies even within a RAG setup. The detailed checker comments accompanying these lower scores provided valuable feedback for analysts and potential retraining of the system.

The examples studied did not utilize the re-generation loop because their average Overall Confidence Scores were always above the operational threshold. However, a lower verification score for a specific Key Issue could have automatically triggered another generation attempt with the same context, potentially leading to an improved output version that would pass the verification threshold, reducing the need for manual correction by the analyst. This capability, built into the pipeline design, offered a path to further enhance the reliability of the output.

5.3.3 Comparative Analysis and Discussion of Results

The quantitative results presented in Sections 5.3.1 and 5.3.2 offered an opportunity for a more detailed discussion. Due to the highly specific objectives of this thesis, specifically, the generation of narrative SWOT analyses for predefined ESG Key Issues by synthesizing proprietary internal metrics with multiple external documents through a custom RAG and verification

pipeline, direct quantitative comparisons with existing published research were challenging. Most of the existing literature focuses on either broader ESG data extraction tasks without this specific synthesis goal or different architectural approaches, such as [49]. Therefore, this subsection discussion focused on the internal comparative analysis of the system’s performance across the profiled companies and different PDF parsers.

Comparison Across Companies and Parsers

Observing the pipeline execution times (Table 5.1), the choice of PDF parser significantly influenced the duration of the pre-processing, in the hardware that was utilized. The PyPDFium2 parser consistently offered the fastest pre-processing, making it suitable for scenarios where rapid ingestion of numerous documents is essential. In contrast, Marker, which performs more complex structural analysis to generate Markdown output, was the most time-intensive for pre-processing.

Regarding accuracy and verification metrics (Table 5.2), most of the combination of companies’ parsers produced high average overall confidence and citation scores. For example, Microsoft Corp. with the PyPDFium2 parser achieved an average Overall Confidence Score of 4.79. Although this high-level average suggested strong performance across parsers, a closer examination of individual Key Issue scores revealed variations. For instance, in the analysis of Apple Inc.’s ”Human Capital Development” Key Issue, both Marker and PyMuPDF4LLM resulted in an Overall Confidence Score of 5.0 from the verification LLM, while PyPDFium2 yielded a score of 4.375, with specific bullets flagged for lower faithfulness or citation issues. This suggested that for certain types of content or Key Issues, the superior structural preservation of parsers like Marker might have led to an initial generation by the LLM that is more directly verifiable.

It was crucial to understand the capabilities and limitations of the automated verification layer. While the verification LLM effectively checked if cited textual content was present on the indicated page and assessed direct factual support, it had inherent difficulties in judging the *correct interpretation of complex document structures*, such as intricate tables, nested hierarchies or the complex meaning conveyed by page layouts. Parsers like Marker, by converting PDFs into a more semantically structured format (Markdown) before chunking, aimed to preserve this contextual information better than parsers that perform simpler text extraction.

This preservation of structure by Marker was highly valued by analysts. Although it incurs a higher pre-processing time cost, analysts indicated a preference for Marker because the resulting generated text often better reflected the intricate information and complex relationships present in the original documents. This, in turn, made the human analyst’s final review more efficient and reliable, as the AI’s initial interpretation was more likely to be aligned with a sophisticated

understanding of the source. Given that a primary goal of the project was to provide precise and contextually accurate ESG profiles, the quality of information interpretation, facilitated by better structural parsing, became a critical factor complementing the automated verification scores. Thus, while faster parsers like PyPDFium2 were viable and performed well on average, the choice of Marker could be justified when the fidelity of complex information is paramount, aligning with the analysts' need for a trustworthy and deeply understood first draft. The RAG pipeline and detailed prompting demonstrated robustness in handling varied inputs, but the initial quality of context, shaped by the parser, evidently influenced the subtleties of the output and the subsequent analyst validation effort.

5.4 Qualitative Analysis: Output Quality and Usability

Beyond quantitative metrics, the qualitative assessment by sustainability analysts provided crucial validation of the system's value proposition. Analysts who reviewed the generated profiles offered positive feedback on several aspects.

- **Quality of the Draft Output:** Analysts found the generated SWOT analyses to be a strong starting point for their work. The text successfully synthesized information from both the structured internal data (scores, peer comparisons) and the unstructured external documents, presenting a coherent overview for each Key Issue. Although the tone was generally appropriate, minor editing was sometimes needed for flow or to add specific interpretative refinements.
- **Value of Integrated Data:** The explicit inclusion of internal scores and peer data within the generated text was highly appreciated. Analysts noted that this immediately contextualized the qualitative findings from the documents, allowing for a rapid assessment of a company's performance relative to its industry peers on each specific Key Issue.
- **Traceability and Trust:** The automatic citation of sources for every statement was highlighted as a critical feature. Analysts could easily trace information back to the original document page or confirm it came from internal data. This transparency built trust in the AI's output and significantly accelerated the validation process, as analysts did not have to manually search source documents to verify claims.
- **Streamlit UI Usability and Transparency:** The Streamlit web interface was perceived as user-friendly and intuitive. Features allowing easy selection of companies, viewing available documents, reviewing parsed content page-by-page in the Document Repository, inspecting the formatted Internal Data, and browsing generated profiles were

considered highly valuable. Crucially, the UI provided direct access to the outputs of the verification layer; analysts could view the generated text for each Key Issue alongside the detailed verification scores (Overall Confidence, Faithfulness, Citation Correctness, Relevance per bullet point) and the specific source snippets retrieved and utilized by the LLM for each generation attempt. This comprehensive visibility into both the AI's work and its automated assessment offers unparalleled insight and control compared to a black-box generative system, supporting the analyst's workflow for reviewing and validating the AI's output.

An example of this successful integration was the internal pilot test conducted using Microsoft's profile. A sustainability analyst used the system to generate the profile and reported that the output significantly reduced the time required for their analysis. The ability to quickly access the AI-generated SWOT, verify the cited points against the source documents via the UI, and see the supporting internal metrics and verification scores allowed the analyst to focus on formulating strategic insights rather than laboring over data extraction and initial drafting. Although minor corrections to the generated text were still necessary, the system transformed the process from primarily data-gathering to primarily data-interpreting.

The identification of errors by the verification layer, although present, was not seen as a failure of the system but rather a validation of the methodology's design. The AI provided a powerful first draft and identified potential issues through its automated verification, but the human expert remained essential for the final quality assurance.

5.5 Summary of Results

The evaluation demonstrated that the implemented AI-assisted ESG profiling system successfully achieved its primary objectives. Quantitatively, it delivered significant efficiency gains, reducing the time required for the initial drafting phase of an ESG profile by an estimated 80%. The automated verification layer provided valuable metrics on factual accuracy and citation correctness, successfully identifying specific errors in the generated output. Qualitatively, the system produced high-quality draft analyzes that effectively synthesized complex information, were well-structured, and crucially provided transparent and easily verifiable citations. The user-friendly Streamlit interface improved overall usability, enabling sustainability analysts to seamlessly integrate the tool into their workflow and focus on expert interpretation and strategy rather than manual data processing. Although some refinement and human supervision remained necessary, the system proved to be a valuable assistant, significantly accelerating the ESG profiling process while maintaining essential standards of factual integrity and traceability.

6 Conclusion and Future Work

6.1 Conclusion

The landscape of ESG analysis is rapidly evolving, driven by increasing regulatory demands and investor expectations for comprehensive, data-driven corporate sustainability profiles. As highlighted in this thesis, the manual process of compiling such profiles from diverse, unstructured sources is inherently time-consuming, resource-intensive, and susceptible to inconsistencies. Although LLMs offer significant potential for automating text synthesis, their fundamental limitations regarding factual accuracy and source traceability render standard implementations unsuitable for risk-sensitive financial domains.

This Master’s thesis successfully addressed these challenges by designing and implementing an AI-assisted ESG company profiling tool based on a RAG methodology. The core contribution lay in the integration of external document analysis (utilizing pre-processing, page-based chunking, and vectorization) with proprietary internal ESG metrics, which was orchestrated through a sophisticated RAG pipeline featuring detailed prompt engineering and an automated post-generation verification layer.

The evaluation demonstrated that the implemented system achieved its main objectives:

- **Significant Efficiency Gains:** By automating the laborious data collection, processing, and initial drafting steps, the system dramatically reduced the time required to produce a comprehensive ESG profile draft ready for expert review, translating to an estimated 80% reduction in the initial profiling effort.
- **Enhanced Factual Accuracy and Traceability:** The RAG approach successfully grounded the LLM’s output in verifiable sources, and the implemented verification layer provided crucial automated checks for factual correctness and citation accuracy. The structured output format, with explicit source citations for every statement, ensured transparency and facilitated efficient validation by human analysts.
- **Improved Output Quality and Usability:** The generated drafts were evaluated by domain experts as coherent, relevant, and valuable starting points for their analysis. The intuitive Streamlit interface enabled analysts to easily interact with the system, control the process, and inspect the AI’s outputs and underlying sources, fostering trust and optimizing the review workflow.

Through this work, the project successfully transformed the process from a manual, labor-intensive task into an AI-assisted workflow where human expertise is leveraged for high-level

interpretation and validation rather than rote data compilation. The implemented system served as a robust prototype that demonstrated the power of RAG for domain-specific knowledge synthesis and laid a solid foundation for future enhancements and deployment in a production environment.

6.2 Future Work

Based on the successful implementation and evaluation of the RAG-based ESG profiling system, there are several avenues for future work to further enhance its capabilities, robustness, and scalability.

6.2.1 Infrastructure and Model Enhancements

The most critical next step towards production readiness is the full deployment of the system on internal corporate infrastructure. This will enable the utilization of more powerful local LLMs that were constrained by the development environment's hardware limitations. Leveraging corporate GPU resources will allow for faster PDF parsing, experimentation with more complex parsing options, and the deployment of larger, state-of-the-art open-source LLM models. Access to more powerful models is expected to improve the qualitative aspects of the generated output, including coherence, fluency, and potentially the ability to synthesize more complex relationships between data points, while maintaining data privacy and potentially reducing long-term API costs associated with cloud models.

6.2.2 Preprocessing Pipeline Refinements

The document pre-processing pipeline can be further refined. Although page-based fragmentation offers simplicity and supports direct citation, exploring hybrid chunking strategies that combine page boundaries with semantic segmentation might be beneficial for certain document types. Additionally, leveraging advanced features available in parsing libraries like Marker, such as OCR for scanned documents and the integration of LLM-powered parsing, could improve the processing of complex or image-heavy reports, ensuring that all relevant information is captured and structured appropriately for retrieval.

6.2.3 RAG Pipeline Advancements

Several enhancements can be made to the core RAG pipeline:

- **Sophisticated Retrieval Strategies:** The current retrieval is primarily based on the vector similarity of the query to the pages. Future work could explore re-ranking techniques to ensure the most relevant information is prioritized and implement a more dynamic retrieval strategy that adjusts the number of retrieved chunks (k) based on the LLM's context window size and/or the retrieval confidence scores to optimize performance. Additionally, exploring methods to ensure representation from multiple distinct source documents in the retrieved context, even if some are slightly less similar overall, could provide a broader perspective to the LLM.
- **Prompt Optimization:** Continuously refining the prompt templates based on analyst feedback and observed LLM behaviors is essential.

6.2.4 Verification and Quality Improvements

The automated verification layer is a critical component. Future work could focus on improving its efficiency and capabilities. Evaluating the feasibility of using smaller LLMs for the verification task could reduce computational overhead compared to using the same large model for both generation and verification.

6.2.5 Output Enhancements and Application Development

While the current Streamlit UI provides essential functionality, developing a final overall output that, based on all key issue outputs, creates a SWOT analysis with a more narrative or "story-telling" component would directly align with the analysts' need to articulate a cohesive narrative for the final ESG company profile.

Additionally, transitioning the Streamlit prototype to a more robust, multi-user architecture, using a Flask backend with a dedicated frontend framework, would be necessary for production deployment, offering improved scalability, security, and integration capabilities within the corporate environment.

6.2.6 Continuous Learning and Feedback Loop

Formalizing the human-in-the-loop feedback mechanism is crucial to long-term improvement. Building tools within the application to easily capture analyst edits and corrections and storing this data would provide a valuable dataset for iteratively refining the RAG pipeline, improving prompt strategies, and potentially fine-tuning models on domain-specific corrections. This continuous learning cycle, powered by expert human validation, will be key to the system's ongoing accuracy and relevance.

The thesis represented a significant step towards leveraging AI for improved ESG company analysis. The implemented RAG system demonstrated that combining LLMs with domain-specific data and a robust verification process can produce efficient and verifiable outputs, positioning it as a valuable tool for sustainability analysts and highlighting promising directions for future development in this evolving field.

Bibliography

- [1] Wikipedia. Transformer (deep learning architecture), 2023.
- [2] Société Générale Private Banking. Esg analysis: Assessing company competitiveness and risk-adjusted returns. *Société Générale Private Banking Insights*, 2025.
- [3] J. Safra Sarasin. Esg integration is no longer optional. *J. Safra Sarasin Perspectives*, 2025.
- [4] Principles for Responsible Investment. Part iii: Esg factors and returns – a review of recent research. *PRI Blog*, 2024.
- [5] NYU Stern Center for Sustainable Business and Rockefeller Asset Management. Esg and corporate financial performance: A research paper. Technical report, NYU Stern Center for Sustainable Business, 2021.
- [6] Adrian Bancu. A meta-analysis of esg disclosure and company’s economic performance. *Proceedings of the International Conference on Business Excellence*, 18(1):2042–2056, 2024.
- [7] Swiss Sustainable Finance and CFA Society Switzerland. *Handbook on Sustainable Investments: Background Information and Practical Examples for Institutional Asset Owners*. CFA Institute Research Foundation, 2017.
- [8] Global Sustainable Investment Alliance. Global sustainable investment review 2020, 2020.
- [9] International Platform on Sustainable Finance. Stewardship and engagement in transition finance, nov 2024.
- [10] Rickard Nilsson. Increasing accountability in investment stewardship: A spotlight on engagement types and time horizons. *Illuminem Voices*, jan 2025.
- [11] Bloomberg Intelligence. Global esg assets predicted to hit \$40 trillion by 2030 despite challenging environment. *Bloomberg Press Releases*, jan 2024.
- [12] Professional Wealth Management. Greenwashing backlash hinders esg investment in us. *Professional Wealth Management*, 2024.
- [13] Boston University Impact Measurement and Allocation Program. Financial materiality of esg risks. Technical report, Boston University, 2025.

- [14] Sustainalytics. Esg materiality: What investors need to know. *Sustainalytics ESG Research*, 2024.
- [15] MIT Sloan Sustainability Initiative. Aggregate confusion project.
- [16] The Economist Intelligence Unit. Esg reporting: Challenges and opportunities for financial services firms. *Economist Impact*, 2025.
- [17] Sustainability Directory. What are the challenges of standardizing esg data? *Sustainability Directory*, jan 2025.
- [18] Deepki. Greenwashing in esg reporting: Risks, consequences, and strategies for transparency, 2025.
- [19] PricewaterhouseCoopers. Pwc’s global investor survey 2024, 2024.
- [20] Sustainability Directory. Is there a difference between ai-driven and human-led esg analysis? *Sustainability Directory*, 2024.
- [21] SESAMm. Esg risk assessment: How ai can solve traditional challenges. *SESAMm Blog*, nov 2024.
- [22] Wipro. The ai imperative in institutional esg investing. *Wipro Capital Markets Insights*, 2024.
- [23] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. Attention is all you need. *Advances in Neural Information Processing Systems (NeurIPS)*, 30:5998–6008, 2017.
- [24] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. BERT: Pre-training of deep bidirectional transformers for language understanding. In *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics (NAACL)*, pages 4171–4186, 2019.
- [25] Alec Radford, Jeffrey Wu, Rewon Child, David Luan, Dario Amodei, and Ilya Sutskever. Language models are unsupervised multitask learners. Technical report, OpenAI, 2019.
- [26] Harsha Nori, Yi Tay, Mostafa Dehghani, Jai Gupta, and Donald Metzler. Scaling transformers to longer contexts. *arXiv preprint arXiv:2402.06196*, 2025.
- [27] Tom B. Brown et al. Language models are few-shot learners. *arXiv preprint arXiv:2005.14165*, 2020.

-
- [28] Colin Raffel et al. Exploring the limits of transfer learning with a unified text-to-text transformer. *Journal of Machine Learning Research*, 2019.
- [29] Google Research. Palm: Scaling language modeling with pathways. *arXiv preprint arXiv:2204.02311*, 2022.
- [30] Hugo Touvron, Thibaut Lavril, Gautier Izacard, Xavier Martinet, Marie-Anne Lachaux, et al. Llama: Open and efficient foundation language models, 2023.
- [31] Anthem Creation. Openai gpt-4 and 4.5: Improved conversational abilities, 2025.
- [32] Google. New features for the gemini api and google ai studio. *Google Developers Blog*, 2024.
- [33] BBC News. Global esg assets predicted to hit \$40 trillion by 2030 despite challenging environment. *BBC News*, feb 2025.
- [34] ArXiv. Retrieval-augmented generation for large language models: A survey. *arXiv preprint arXiv:2312.10997*, dec 2023.
- [35] ArXiv. Training language models to follow instructions with human feedback. *arXiv preprint arXiv:2203.02155*, 2022.
- [36] PMC. Evaluation of large language model performance and reliability for citations and references in scholarly writing: Cross-disciplinary study. *Journal of Medical Internet Research*, 2025.
- [37] MSCI. Sustainable finance: Esg investing solutions, 2025.
- [38] Environmental Finance. Esg guide entry: Sustainalytics, 2024.
- [39] Margaret Capetz, Christina Chance, Rebecca Pattichis, Avalon Vinella, Reshmi Ghosh, and Kai-Wei Chang. Leveraging language models to detect greenwashing. *arXiv preprint arXiv:2311.01469*, 2024.
- [40] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. Retrieval-augmented generation for knowledge-intensive nlp tasks. In *Advances in Neural Information Processing Systems 33 (NeurIPS 2020)*, 2020.
- [41] Jie Huang and Kevin Chen-Chuan Chang. Citation: A key to building responsible and accountable large language models. *Findings of the North American Chapter of the Association for Computational Linguistics (NAACL)*, 2024.

-
- [42] Anthropic. Introducing the next generation of claude. *Anthropic News*, 2024.
- [43] ISS Governance. The intersection of ai and esg: Discrimination. *ISS Governance Insights*, 2023.
- [44] Holistic AI. Ai and esg: Understanding the environmental impact of ai and llms. *Holistic AI Blog*, 2024.
- [45] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. Retrieval-augmented generation for knowledge-intensive nlp tasks. *arXiv preprint arXiv:2005.11401*, 2020.
- [46] Ayman Asad Khan, Md Toufique Hasan, Kai Kristian Kemell, Jussi Rasku, and Pekka Abrahamsson. Developing retrieval augmented generation (rag) based llm systems from pdfs: An experience report. *arXiv preprint arXiv:2410.15944*, 2024.
- [47] Zhuowan Li, Cheng Li, Mingyang Zhang, Qiaozhu Mei, and Michael Bendersky. Retrieval augmented generation or long-context llms? a comprehensive study and hybrid approach. *arXiv preprint arXiv:2407.16833*, 2024.
- [48] Siyun Zhao, Yuqing Yang, Zilong Wang, Zhiyuan He, Luna K. Qiu, and Lili Qiu. Retrieval augmented generation (rag) and beyond: A comprehensive survey on how to make your llms use external data more wisely. *arXiv preprint arXiv:2409.14924*, 2024.
- [49] Yi Zou, Mengying Shi, Zhongjie Chen, Zhu Deng, ZongXiong Lei, Zihan Zeng, Shiming Yang, HongXiang Tong, Lei Xiao, and Wenwen Zhou. Esgreveal: An llm-based approach for extracting structured data from esg reports. *arXiv preprint arXiv:2312.17264*, 2023.
- [50] Edward Beeching, Clémentine Fourrier, Nathan Habib, Sheon Han, Nathan Lambert, Nazneen Rajani, Omar Sanseviero, Lewis Tunstall, and Thomas Wolf. Open llm leaderboard, 2023.
- [51] Hugging Face. Hugging face models, 2023.
- [52] Author(s) Unknown. Comparative analysis of deepseek r1, chatgpt, gemini, alibaba, and llama: Performance, reasoning capabilities, and political bias. *Authorea*, 2023.
- [53] LLM Stats. Llm leaderboard 2025: Verified ai rankings, 2025.
- [54] Jared Kaplan, Sam McCandlish, Tom Henighan, Tom B. Brown, Benjamin Chess, Rewon Child, Scott Gray, Alec Radford, Jeffrey Wu, and Dario Amodei. Scaling laws for neural language models. *arXiv preprint arXiv:2001.08361*, 2020.

- [55] Jordan Hoffmann, Sebastian Borgeaud, Arthur Mensch, Elena Buchatskaya, Trevor Cai, Eliza Rutherford, Diego de Las Casas, Lisa Anne Hendricks, Johannes Welbl, Aidan Clark, et al. Training compute-optimal large language models. *arXiv preprint arXiv:2203.15556*, 2022.
- [56] Houyi Li, Wenzheng Zheng, Jingcheng Hu, Qiufeng Wang, Hanshan Zhang, Zili Wang, Shijie Xuyang, Yuantao Fan, Shuigeng Zhou, Xiangyu Zhang, and Daxin Jiang. Predictable scale: Part i — optimal hyperparameter scaling law in large language model pretraining. *arXiv preprint arXiv:2503.04715*, 2025.
- [57] Chien-Ping Lu. The race to efficiency: A new perspective on ai scaling laws. *arXiv preprint arXiv:2501.02156*, 2025.
- [58] Jonas Vetterle. A brief history of llm scaling laws and what to expect in 2025, 2024.
- [59] Sebastian Raschka. Noteworthy llm research papers of 2024, 2025.
- [60] Weilin Cai, Juyong Jiang, Fan Wang, Jing Tang, Sunghun Kim, and Jiayi Huang. A survey on mixture of experts. *arXiv preprint arXiv:2407.06204*, 2024.
- [61] DeepSeek-AI et al. Deepseek-r1: Incentivizing reasoning capability in llms via reinforcement learning. *arXiv preprint arXiv:2501.12948*, 2025.
- [62] Dan Busbridge, Amitis Shidani, Floris Weers, Jason Ramapuram, Etai Littwin, and Russ Webb. Distillation scaling laws. *arXiv preprint arXiv:2502.08606*, 2025.
- [63] Google DeepMind Gemma Team. Gemma 3 technical report, 2025.
- [64] Tanishq Kumar, Zachary Ankner, Benjamin F. Spector, Blake Bordelon, Niklas Muenighoff, Mansheej Paul, Cengiz Pehlevan, Christopher Ré, and Aditi Raghunathan. Scaling laws for precision. *arXiv preprint arXiv:2411.04330*, 2024.
- [65] Pranav Nair, Puranjay Datta, Jeff Dean, Prateek Jain, and Aditya Kusupati. Matryoshka quantization. *arXiv preprint arXiv:2502.06786*, 2025.
- [66] Charlie Snell, Jaehoon Lee, Kelvin Xu, and Aviral Kumar. Scaling llm test-time compute optimally can be more effective than scaling model parameters. *arXiv preprint arXiv:2408.03314*, 2024.
- [67] Ruslan O. Popov, Nadiia V. Karpenko, and Volodymyr V. Gerasimov. Overview of small language models in practice. In *Proceedings of the 7th Workshop for Young Scientists in Computer Science & Software Engineering (CS&SE@SW 2024)*, 2025.

-
- [68] Sarah Oberbichler and Cindarella Petz. Working paper: Implementing generative ai in the historical studies, 2025.
 - [69] OpenAI. Prompt engineering guide.
 - [70] Google AI. Introduction to prompt design.
 - [71] Anthropic. Prompt engineering overview.
 - [72] Llama. Prompting how-to guide.
 - [73] Mistral AI. Prompting capabilities guide.
 - [74] Sander Schulhoff, Michael Ilie, Nishant Balepur, Konstantine Kahadze, Amanda Liu, Chenglei Si, Yinheng Li, Aayush Gupta, HyoJung Han, Sevien Schulhoff, et al. The prompt report: A systematic survey of prompting techniques. *arXiv preprint arXiv:2406.06608*, 2024.
 - [75] Shubham Vatsal and Harsh Dubey. A survey of prompt engineering methods in large language models for different nlp tasks. *arXiv preprint arXiv:2407.12994*, 2024.
 - [76] Meta AI. Llama 3.3 70b instruct model, 2024.
 - [77] Sundar Pichai and Demis Hassabis. Introducing gemini 2.0: Our new ai model for the agentic era, 2024.
 - [78] Anthropic. Claude models overview, 2025.
 - [79] Hao Ge, Junda Feng, Qi Huang, Fangcheng Fu, Xiaonan Nie, Lei Zuo, Haibin Lin, Bin Cui, and Xin Liu. Bytescale: Efficient scaling of llm training with a 2048k context length on more than 12,000 gpus. *arXiv preprint arXiv:2502.21231*, 2025.
 - [80] Anonymous. Omnikv: Dynamic context selection for efficient long-context llms. *International Conference on Learning Representations (ICLR)*, 2025. Under review.
 - [81] Daniil Suhoi. Efficient deep learning: A comprehensive overview of optimization techniques, 2024. Community article on Hugging Face.
 - [82] Tri Dao, Daniel Y. Fu, Stefano Ermon, Atri Rudra, and Christopher Ré. Flashat-tention: Fast and memory-efficient exact attention with io-awareness. *arXiv preprint arXiv:2205.14135*, 2022.

- [83] Haoran You, Yichao Fu, Zheng Wang, Amir Yazdanbakhsh, and Yingyan Celine Lin. When linear attention meets autoregressive decoding: Towards more effective and efficient linearized large language models. *arXiv preprint arXiv:2406.07368*, 2024.
- [84] Jingyang Yuan, Huazuo Gao, Damai Dai, Junyu Luo, Liang Zhao, Zhengyan Zhang, Zhenda Xie, Y. X. Wei, Lean Wang, Zhiping Xiao, Yuqing Wang, Chong Ruan, Ming Zhang, Wenfeng Liang, and Wangding Zeng. Native sparse attention: Hardware-aligned and natively trainable sparse attention. *arXiv preprint arXiv:2502.11089*, 2025.
- [85] Weilin Cai, Juyong Jiang, Fan Wang, Jing Tang, Sunghun Kim, and Jiayi Huang. A survey on mixture of experts. *arXiv preprint arXiv:2407.06204*, 2024.
- [86] Zichao Yang, Diyi Yang, Chris Dyer, Xiaodong He, Alex Smola, and Eduard Hovy. Hierarchical attention networks for document classification. In *Proceedings of the 2016 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*, pages 1480–1489, 2016.
- [87] Xupeng Miao, Shenhan Zhu, Fangcheng Fu, Ziyu Guo, Zhi Yang, Yaofeng Tu, Zhihao Jia, and Bin Cui. X-former elucidator: Reviving efficient attention for long context language modeling. In *Proceedings of the 33rd International Joint Conference on Artificial Intelligence (IJCAI)*, page 904, 2024.
- [88] Renzo Arturo Alva Principe. *The Problem of Long Document Representation and Processing in the Era of Transformer-Based Large Language Models*. PhD thesis, Università degli Studi di Milano-Bicocca, 2024.
- [89] LangChain. How to load pdfs, 2025. Guide on loading PDFs using LangChain.
- [90] LlamaIndex. Llaparse update: new and upcoming features, 2025. Blog post about recent updates to LlamaParse.
- [91] Nikolaos Livathinos, Christoph Auer, Maksym Lysak, Ahmed Nassar, Michele Dolfi, Panagiotis Vagenas, Cesar Berrospi, Matteo Omenetti, Kasper Dinkla, Yusik Kim, Shubham Gupta, Rafael Teixeira de Lima, Valery Weber, Lucas Morin, Ingmar Meijer, Viktor Kuropiatnyk, and Peter W. J. Staar. Docling: An efficient open-source toolkit for ai-driven document conversion. *arXiv preprint arXiv:2501.17887*, 2025.
- [92] Unstructured. How to process pdfs in python: A step-by-step guide, 2023. Blog post on processing PDFs using Python.

-
- [93] Qihui Zhou, Peiqi Yin, Pengfei Zuo, and James Cheng. Progressive sparse attention: Algorithm and system co-design for efficient attention in llm serving. *arXiv preprint arXiv:2503.00392*, 2025.
 - [94] Zhanke Zhou, Rong Tao, Jianing Zhu, Yiwen Luo, Zengmao Wang, and Bo Han. Can language models perform robust reasoning in chain-of-thought prompting with noisy rationales? *arXiv preprint arXiv:2410.23856*, 2024.
 - [95] Ziwei Ji, Nayeon Lee, Jason Fries, Tianyi Yu, Faisal Ahmed, Hady Elsahar Awadalla, Richard Sutcliffe, Kate Saenko, Russ Altman, and Farah E. Ahmad. A survey on hallucination in large language models. *ACM Computing Surveys*, 56(2):1–39, 2023.
 - [96] Hyo Jin Do, Rachel Ostrand, Justin D. Weisz, Casey Dugan, Prasanna Sattigeri, Dennis Wei, Keerthiram Murugesan, and Werner Geyer. Facilitating human-llm collaboration through factuality scores and source attributions. *arXiv preprint arXiv:2405.20434*, 2024.
 - [97] Hanxing Ding, Liang Pang, Zihao Wei, Huawei Shen, and Xueqi Cheng. Retrieve only when it needs: Adaptive retrieval augmentation for hallucination mitigation in large language models. *arXiv preprint arXiv:2402.10612*, 2024.
 - [98] S.M Towhidul Islam Tonmoy, S M Mehedi Zaman, Vinija Jain, Anku Rani, Vipula Rawte, Aman Chadha, and Amitava Das. A comprehensive survey of hallucination mitigation techniques in large language models. *arXiv preprint arXiv:2401.01313*, 2024.
 - [99] Mikhail Rumiantsev, Aliaksei Vertsel, Ilya Hrytsuk, and Isaiah Ballah. Beyond fine-tuning: Effective strategies for mitigating hallucinations in large language models for data analytics. *arXiv preprint arXiv:2410.20024*, 2024.
 - [100] Guangsheng Liu, Xinbo Ai, Wenbin Luo, and Ange Li. Mitigating hallucination in large language models by leveraging decoder layer contrasting. In *Pattern Recognition. ICPR International Workshops and Challenges*, pages 45–54. Springer, 2024.
 - [101] Huandong Wang, Wenjie Fu, Yingzhou Tang, Zhilong Chen, Yuxi Huang, Jinghua Piao, Chen Gao, Fengli Xu, Tao Jiang, and Yong Li. A survey on responsible llms: Inherent risk, malicious use, and mitigation strategy. *arXiv preprint arXiv:2501.09431*, 2025.
 - [102] Imama Shehzad. Beyond traditional fine-tuning: Exploring advanced techniques to mitigate llm hallucinations, 2024. Blog post on Hugging Face.
 - [103] Tom B. Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, Sandhini

- Agarwal, Ariel Herbert-Voss, Gretchen Krueger, Tom Henighan, Rewon Child, Aditya Ramesh, Daniel M. Ziegler, Jeffrey Wu, Clemens Winter, Christopher Hesse, Mark Chen, Eric Sigler, Mateusz Litwin, Scott Gray, Benjamin Chess, Jack Clark, Christopher Berner, Sam McCandlish, Alec Radford, Ilya Sutskever, and Dario Amodei. Language models are few-shot learners. *arXiv preprint arXiv:2005.11401*, 2020.
- [104] Siran Li, Linus Stenzel, Carsten Eickhoff, and Seyed Ali Bahrainian. Enhancing retrieval-augmented generation: A study of best practices. *arXiv preprint arXiv:2501.07391*, 2025.
- [105] Mingyue Cheng, Yucong Luo, Jie Ouyang, Qi Liu, Huijie Liu, Li Li, Shuo Yu, Bohou Zhang, Jiawei Cao, Jie Ma, Daoyu Wang, and Enhong Chen. A survey on knowledge-oriented retrieval-augmented generation. *arXiv preprint arXiv:2503.10677*, 2025.
- [106] Hugging Face. Text embedding models, 2025. Documentation on configuring text embedding models in Chat UI.
- [107] Jeff Johnson, Matthijs Douze, and Hervé Jégou. Billion-scale similarity search with GPUs, 2017. FAISS: A library for efficient similarity search and clustering of dense vectors.
- [108] Chroma. Chroma: The open-source AI application database, 2025. Official website of Chroma, an open-source AI application database.
- [109] Pinecone. The vector database to build knowledgeable AI, 2025. Official website of Pinecone, a vector database service.
- [110] Weaviate. The AI-native database developers love, 2025. Official website of Weaviate, an open-source vector database.
- [111] Qdrant. Qdrant: High-performance vector search at scale, 2025. Official website of Qdrant, a vector database and similarity search engine.
- [112] Satyadhar Joshi. Introduction to vector databases for generative AI applications: Performance, future projections, and cost considerations. *Advanced Research Journal of Science, Engineering and Technology*, 12(2):10–20, 2025.
- [113] Demiao Lin. Revolutionizing retrieval-augmented generation with enhanced pdf structure recognition. *arXiv preprint arXiv:2401.12599*, 2024.
- [114] LangChain. Langchain documentation: Introduction, 2025. Introduction to LangChain, a framework for developing applications powered by language models.

-
- [115] LlamaIndex. Llamaindex: Build knowledge assistants over your enterprise data, 2025. Official website of LlamaIndex, a platform for building AI knowledge assistants over enterprise data.
 - [116] Haystack. Haystack: The production-ready open source ai framework, 2025. Official website of Haystack, an open-source AI framework for building production-ready applications.
 - [117] Demiao Li, Haotian Zhang, Weizhi Chen, Jing Liu, Yuxuan Wang, and Lei Li. Chatdoc: A large language model based interactive system for efficient document retrieval and comprehension. In *Proceedings of the 2024 IEEE Conference on Artificial Intelligence (IEEE AI 2024)*, 2024.
 - [118] Qinggang Zhang, Shengyuan Chen, Yuanchen Bei, Zheng Yuan, Huachi Zhou, Zijin Hong, Junnan Dong, Hao Chen, Yi Chang, and Xiao Huang. A survey of graph retrieval-augmented generation for customized large language models. *arXiv preprint arXiv:2501.13958*, 2025.
 - [119] Jiawei Chen, Hongyu Lin, Xianpei Han, and Le Sun. Benchmarking large language models in retrieval-augmented generation. *arXiv preprint arXiv:2309.01431*, 2023.
 - [120] Yuanjie Lyu, Zhiyu Li, Simin Niu, Feiyu Xiong, Bo Tang, Wenjin Wang, Hao Wu, Huanyong Liu, Tong Xu, and Enhong Chen. CRUD-RAG: A comprehensive chinese benchmark for retrieval-augmented generation of large language models. *ACM Transactions on Information Systems*, 2025.
 - [121] Yug V. Patel, Vijaykumar Salvia, and Indrajeet Kumar. A survey on retrieval-augmented generation: From naive to adaptive approaches with financial insights. *Kronika Journal*, 2025.
 - [122] Aditi Singh, Abul Ehtesham, Saket Kumar, and Tala Talaei Khoei. Agentic retrieval-augmented generation: A survey on agentic rag. *arXiv preprint arXiv:2501.09136*, 2025.
 - [123] wanaguirre. AI-Assisted ESG Profiling, 2025.
 - [124] OpenAI. Gpt-4.5 preview, 2025.
 - [125] OpenAI. Gpt-4o, 2024.
 - [126] OpenAI. Gpt-4o mini, 2024.
 - [127] OpenAI. o1, 2024.

- [128] OpenAI. o3 mini, 2024.
- [129] Anthropic. Claude 3.7 sonnet and claude code, 2025.
- [130] LLM Stats. Llm model comparison, 2025.
- [131] Google. Gemma 3: Google’s new open model based on gemini 2.0, 2025.
- [132] Omar Sanseviero and Philipp Schmid. Introducing gemma 3: The developer guide, 2025.
- [133] Aritra Roy Gosthipaty, Merve Noyan, Pedro Cuenca, and Vaibhav Srivastav. Welcome gemma 3: Google’s all-new multimodal, multilingual, long context open llm, 2025.
- [134] Meta AI. Llama 3.3 model card, 2024.
- [135] DeepSeek AI. Deepseek official website, 2025.
- [136] DeepSeek AI. Deepseek-r1 model, 2025.
- [137] DeepSeek AI. Deepseek-v3 model, 2025.
- [138] LLM Stats. Mistral small 3: Pricing, context window, benchmarks, and more, 2025.
- [139] Mistral AI. Mistral-small-24b-instruct-2501 model, 2025.
- [140] Qwen Team. Qwen2.5-max: Exploring the intelligence of large-scale moe model, 2025.
- [141] Qwen Team. Qwen2.5-14b-instruct-1m model, 2025.
- [142] Qwen Team. Qwen2.5-72b-instruct model, 2025.
- [143] Microsoft Research. Phi-4 model, 2025.
- [144] Microsoft. Introducing phi-4: Microsoft’s newest small language model specializing in complex reasoning, 2025.
- [145] Marah Abdin, Jyoti Aneja, Harkirat Behl, Sébastien Bubeck, Ronen Eldan, Suriya Gunasekar, Michael Harrison, Russell J. Hewett, Mojan Javaheripi, Piero Kauffmann, James R. Lee, Yin Tat Lee, Yuanzhi Li, Weishung Liu, Caio C. T. Mendes, Anh Nguyen, Eric Price, Gustavo de Rosa, Olli Saarikivi, Adil Salim, Shital Shah, Xin Wang, Rachel Ward, Yue Wu, Dingli Yu, Cyril Zhang, and Yi Zhang. Phi-4 technical report, 2024.
- [146] xAI. Model constraints, 2025.

-
- [147] DataCamp. Llama.cpp tutorial: A complete guide to efficient llm inference and implementation.
 - [148] Georgi Gerganov. llama.cpp: Llm inference in c/c++.
 - [149] Ollama Inc. Ollama: Run large language models locally.
 - [150] Ollama Inc. Ollama library.
 - [151] CyberArk Engineering. How to run llms locally with ollama.
 - [152] Nomic AI. Gpt4all: Run local llms on any device.
 - [153] Ashley Innocent. The 5 best llm tools to run models locally.
 - [154] Nomic AI. Gpt4all github repository.
 - [155] LM Studio Team. Lm studio: Discover, download, and run local llms.
 - [156] LM Studio Team. Lm studio documentation.
 - [157] Chroma Team. Chroma: The open-source ai application database.
 - [158] Facebook AI Research. Faiss: A library for efficient similarity search and clustering of dense vectors.
 - [159] LangChain Team. Langchain: A framework for developing applications powered by large language models.
 - [160] Arthur Vitui and Tse-Hsun (Peter) Chen. Empowering aiops: Leveraging large language models for it operations management. *arXiv preprint arXiv:2501.12461*, 2025.
 - [161] LlamaIndex Team. Llamaindex: An interface between llms and your data.
 - [162] Microsoft. Azure openai service.
 - [163] Microsoft. Azure ai foundry.
 - [164] Google Cloud. Vertex ai.
 - [165] Google Cloud. Model garden.
 - [166] Amazon Web Services. Amazon sagemaker.
 - [167] Amazon Web Services. Amazon bedrock.

[168] Cohere Inc. The world's leading ai platform for enterprise.

[169] Cursor. Cursor: The ai code editor.

[170] Codeium. Windsurf editor by codeium.

Appendix A: Supplementary Details for State of the Art

A.1 Detailed Overview of Core LLM Developments (as of early 2025)

The following provides a more detailed summary of notable LLM releases and updates from key industry players in early 2025, expanding on the summary presented in Section [2.1.1](#).

OpenAI

OpenAI’s GPT models continue to set industry standards. The latest iteration, GPT-4.5 (released in February 2025), further improves natural language understanding and generation, showing reduced hallucinations and more fluid interactions [\[124\]](#). GPT-4.5 also integrates web browsing, allowing it to fetch up-to-date information during conversations. OpenAI’s model lineup has diversified into specialized variants: for example, GPT-4o (“Omni”) is a multimodal model that handles text, voice, and vision, and GPT-4o mini provides similar multimodal capabilities in a lighter version, at least through its platform, because when using these models through API, audio is not a possible input [\[125, 126\]](#). Meanwhile, the o1 and o3 series are reasoning, focused models, o1 emphasizes reflective step-by-step reasoning, and o3-mini and o3-mini-high are smaller models that surprisingly excel in tasks such as math, science, and logical reasoning, and the o3 version is not yet publicly available [\[127, 128\]](#). In terms of context window, OpenAI offers variants up to 128k tokens for GPT-4.5, 4o or 4mini and 200k for o1 or o3mini models, enabling very long documents or conversations as input.

OpenAI’s models are proprietary (exact architecture and size are not public), offered via API or ChatGPT with a usage-based cost and requiring data sharing with the service. They are generally the most powerful option for broad tasks (they consistently achieve top-tier performance across benchmarks), but come with considerations about cost (OpenAI’s top models are expensive at higher volumes), data privacy (sending sensitive data to an external API), and access.

Anthropic

Anthropic’s Claude models have also firmly established themselves among industry leaders. The latest model, Claude 3.7 ”Sonnet” (released in February 2025), offers significant advancements in natural language processing, emphasizing enhanced capabilities in deliberate reasoning and self-reflection [129]. Claude 3.7 ”Sonnet” and 3.5 ”Haiku” support an exceptionally large context window of 200k tokens, matching OpenAI’s highest offerings [78]. In practical terms, it can ingest between 300 and 600 pages, depending on the complexity of the text, and then answer questions or produce a summary, perform due diligence on companies, or summarize regulatory documents.

Anthropic models, such as OpenAI, are proprietary, with details about their architecture and parameter counts remaining confidential. These models are accessible through their API or via the Anthropic chat platform. Although Anthropic’s overall performance closely rivals OpenAI’s, its pricing structure varies: it offers competitive, lower prices for high-performance models but relatively higher prices for smaller-scale models (e.g., Haiku compared to GPT-4o mini). This makes Anthropic an attractive alternative for performance, but, as with all proprietary models, users should also carefully consider costs, data privacy implications, and access limitations. [130]

Google

Google has established a strong presence in the LLM space, offering models both for internal applications and as cloud-based solutions via Vertex AI. In its newest series, Google prominently showcases Gemini 2 and Gemma 3, each addressing different use cases and computational requirements.

Gemini 2, developed by DeepMind and released in December 2024, remains Google’s flagship proprietary AI model. It is a multimodal system optimized for advanced reasoning, coding, and visual interpretation. Gemini 2 Flash, a lighter variant, offers increased efficiency while maintaining strong language performance. Both versions support context windows of up to 1 million tokens and are exclusively available through Google AI Studio and Vertex AI. [77]

Gemma 3, released in March 2025, introduced a significant advancement in its open-model offerings. Gemma 3 is available in four sizes: 1B, 4B, 12B, and 27B parameters, with the larger variants (4B, 12B, 27B) offering multimodal capabilities for processing text and images. Supports context windows of up to 128k tokens (32k tokens for the 1B model), facilitating the handling of extensive documents and dialogues. The benchmarks position Gemma 3 as one of the most efficient single-accelerator models, capable of outperforming several competitors, including DeepSeek-V3, Llama 3.3 (72B), GPT-4o mini, or Llama 3.1 (405B), which are

described later [131, 132, 133].

Google’s Gemini 2 model, much like those from OpenAI and Anthropic, is among the leading LLM models and requires consideration of its cost, data privacy, and access restrictions. Gemma 3, on the other hand, offers greater deployment flexibility, robust performance, and its open accessibility through platforms such as Hugging Face makes it a particularly compelling choice for organizations prioritizing customization and data privacy.

Meta

Meta’s LLaMA series continues to push the boundaries of open-source LLMs, providing powerful alternatives to proprietary models. The latest iteration, LLaMA 3.3 (released December 2024), is a 70B parameter model. It also features an extended context window of 128k tokens, making it suitable for handling long-form documents, complex reasoning, and advanced multilingual dialogue.

A key advantage of LLaMA models, apart from having slightly better performance than GPT-4o mini and Claude 3.5 Haiku, is their open-source availability, which, as we mentioned with Gemma 3, allows full customization and on-premise deployment, with a strong performance. [134, 76, 130]

DeepSeek

DeepSeek has rapidly gained recognition in the AI landscape, particularly for its advancements in Mixture-of-Experts (MoE) architecture. The latest iteration, DeepSeek-R1 (released January 2025), is a 671B-parameter MoE model, though only 37B parameters are active per token generation, and it allows 131k input tokens. This architecture enables DeepSeek to deliver state-of-the-art reasoning capabilities while maintaining efficiency. The benchmark tests establish DeepSeek-R1 as the leading open-source LLM, often placing it among the top-ranking models overall, across both open-source and proprietary categories, in some benchmarks.

The cost-efficiency rate of DeepSeek-R1 is another standout feature. Compared to OpenAI’s o1 model, DeepSeek-R1 is reportedly 30 times more cost-effective and 5 times faster, providing high-level performance at a fraction of the computational cost. Its open-source availability allows businesses to fine-tune and deploy it on private infrastructure, ensuring full control over data privacy and governance.

DeepSeek has also released DeepSeek-V3, an optimized version that enhances scalability and inference speed, removing the more complex reasoning feature of R1. Therefore, both models offer cutting-edge reasoning capabilities with unparalleled cost-efficiency. Their scalability, open accessibility, and MoE-driven performance make them a compelling choice for organizations seeking advanced AI solutions without relying on proprietary models. However, deploying these

models requires robust infrastructure, as MoE frameworks involve efficiently managing multiple experts, which can introduce complexity compared to monolithic models such as LLaMA 3.3 or Gemma 3.[135, 136, 137, 130]

Mistral AI

Mistral AI has established itself as a leader in efficient and open-source AI models, offering competitive alternatives to larger proprietary systems. The company's latest model, Mistral Small 3 (released in January 2025), is a 24B-parameter model optimized for low latency and high-speed inference, and it allows 32k input tokens.

A major advantage of Mistral Small 3 is its Apache 2.0 open-source license, which provides users with complete freedom to use, modify, and commercialize the model. Although it may not match models like Gemma 3, it scores better than GTP-4o mini in some benchmarks. Businesses looking for a balance between performance, speed, and cost-effectiveness can deploy Mistral Small 3 in scenarios requiring rapid and localized processing without reliance on external APIs.

Mistral AI continues to push the trend for efficient and high-performance LLMs, demonstrating that intelligent model design can rival larger models without excessive compute requirements. For organizations seeking private and scalable AI solutions, Mistral's models offer a strong alternative to closed models from OpenAI, Anthropic, and Google while maintaining full control over data security and customization.[138, 139, 130]

Alibaba

Alibaba has been actively developing its Qwen series of LLMs, designed to handle multilingual and general-purpose AI tasks. The latest iteration, Qwen 2.5-Max (prereleased in January 2025), is optimized for high-performance text generation.

Qwen 2.5-Max is part of a broader Qwen model family that spans from 500M to 72B parameters. It promises to outperform DeepSeek-V3 and rivals top proprietary models in many benchmarks, but this remains an unanswered question.

A key feature of the Qwen series is its scalability and efficiency. Alibaba has optimized the model for low-latency inference, making it suitable for both cloud-based applications and on-device deployment. They support context windows as large as 131k tokens (Qwen 2.5 72B) or 1 million tokens (Qwen 2.5-14B-Instruct-1M). For example, the benchmark results of Qwen 2.5 72B are comparable to those of LLaMA 3.3, and slightly better than Mistral Small 3.

Alibaba has made several Qwen models publicly available. This accessibility, as previously observed, makes Qwen an attractive option for organizations requiring fine-tuned, domain-specific AI deployments. [140, 141, 142, 130]

Microsoft

Microsoft has been advancing its language models with the Phi series, focusing on efficiency and specialized reasoning capabilities. The latest model, Phi-4 (introduced in December 2024), is a 14.7B parameter language model designed to excel in complex reasoning tasks, particularly in mathematics. It permits the input of 16k tokens.

Phi-4 demonstrates notable performance on various benchmarks; however, it generally performs slightly below LLaMA 3.3 and Qwen 2.5 72B, its results are impressive given its smaller parameter size.

A key aspect of the Phi series is its emphasis on data quality over sheer size. Phi-4 incorporates high-quality synthetic datasets and refined training techniques, enabling it to handle complex reasoning tasks effectively despite its relatively smaller scale.

Additionally, Phi-4, as some of the previous models, is an open source option available through Azure AI Foundry and platforms like Hugging Face. It provides a cost-effective and efficient alternative to larger language models, making it a practical choice for organizations looking for high reasoning performance within a smaller computational footprint.[\[143, 144, 145, 130\]](#)

xAI

xAI has introduced its latest language models, Grok-3 and Grok-3 Mini, in order to compete with the leading AI systems in the market.

Grok-3, released in February 2025, features advanced reasoning capabilities, including modes like "Think" and "Big Brain," designed to tackle complex problems by breaking them down and self-verifying results. Grok-3 outperforms GPT-4o, Gemini 2, DeepSeek's R1, and Claude 3.7 in some benchmarks, positioning it among the top-tier models in the industry.

Grok-3 Mini offers a lighter and faster alternative for users who prioritize speed over some accuracy, with both models supporting an input context of 128k tokens. They are available through xAI's chat platform and via the API. As with all proprietary models, users should carefully consider costs, data privacy implications, and access limitations. [\[146, 130\]](#)

A.2 Detailed Overview of LLM Scaling Laws and Efficiency Dimensions

This section provides a more detailed discussion of the various dimensions contributing to the modern understanding of LLM scaling and efficiency, as summarized in [Section 2.1.2](#).

Hyperparameter Scaling Laws: The Emerging Frontier

A significant advancement in 2025's understanding of scaling comes from research into hyperparameter optimization across model scales. The "Step Law" introduced by extensive empirical research establishes mathematical relationships between optimal learning rates, batch sizes, model parameters, and dataset sizes. It reveals a previously underexplored dimension of scaling efficiency.

Through exhaustive experimentation involving 3,700 different model configurations and nearly one million GPU hours of training, researchers discovered universal scaling laws governing these hyperparameters: the optimal learning rate follows a power-law relationship with both model parameters and data sizes, while the optimal batch size scales primarily with data sizes. This finding is particularly valuable, as it provides a plug-and-play formula for determining optimal hyperparameters without requiring expensive grid searches at each new scale.

The Step Law demonstrates remarkable robustness across variations in model sparsity, training data distribution, and model architectures, unifying optimization approaches for both dense transformers and Mixture-of-Experts models. The practical implication is a significant cost reduction in model development by eliminating much of the trial-and-error typically required for hyperparameter tuning when scaling to larger models. [56]

Better Training Methods and Optimization Strategies

The emphasis on training methodology rather than sheer scale represents another dimension of efficiency gains. Advanced techniques, including human feedback reinforcement learning (RLHF), curriculum learning, and sophisticated optimization strategies, have allowed smaller models to achieve capabilities previously associated with much larger architectures. OpenAI's "reasoning model" o1 and Anthropic's Claude exemplify how emergent reasoning abilities can arise from training strategy rather than size alone.

The focus on post-training techniques, including fine-tuning, instruction tuning, and RLHF, has become particularly pronounced. These methods have enabled substantial capability improvements without proportional increases in model size. For instance, a 70B parameter model in 2025 demonstrates capabilities far exceeding a 70B model from 2022, despite identical parameter counts, primarily due to refinements in these post-training approaches. [58]

Data Quality and Quantity: Refining the Scaling Equation

Recent research studies have highlighted the critical importance of data quality as a separate dimension that affects model performance. The success of models like LLaMA 2 and 3, trained on carefully curated corpora of up to 2 and 15 trillion tokens, demonstrated that superior data

selection can allow smaller models to outperform larger ones trained on inferior datasets.

Furthermore, other empirical studies have revealed that the saturation point for high-quality training data may be substantially higher than previously thought. Meta’s research on LLaMA 3 suggests that continued performance improvements were observed even after 15 trillion tokens of training, contradicting earlier assumptions about diminishing returns. This finding suggests that with sufficient computational resources, the ceiling for scaling benefits may extend further than anticipated, provided the training data maintain sufficient quality and diversity. [59, 56]

Architecturally Efficient Models

Beyond data and training methodologies, architectural innovations are key to achieving efficient scaling. Mixture-of-Experts (MoE) architectures, exemplified by models like DeepSeek-R1, represent a significant departure from monolithic model designs. By employing an ensemble of specialized subnetworks and selectively activating only relevant components for each input, MoE models achieve superior performance-to-cost ratios. This method enables the development of models with significantly large overall parameter sizes, while maintaining computational efficiency during inference because only a portion of the parameters is utilized for each specific query. [56, 60, 61]

Another key architectural strategy is model distillation. Research on distillation scaling laws demonstrates the effectiveness of training smaller ”student” models to emulate larger, more complex ”teacher” models. This knowledge transfer process enables the creation of compact models that can achieve a substantial proportion of the performance of their larger counterparts, offering a pathway to enhanced ”capability per watt” and efficient deployment in resource-constrained environments. [62]

Projects like Google DeepMind with Gemma 3 demonstrate the practical impact of these principles, showing that properly distilled 27B parameter models can achieve most of the capabilities of much bigger teachers when trained on carefully designed synthetic data incorporating explanations and reasoning chains. This approach effectively democratizes access to advanced capabilities by making them available on more modest hardware. [63]

Precision Reduction and Hardware Optimization

Advances in quantization techniques have established another dimension of scaling efficiency. Research into Scaling Laws for Precision (Wei et al., 2024) has formalized the relationship between numeric precision and model performance, demonstrating that carefully optimized low-precision representations can maintain nearly equivalent capabilities while dramatically reducing memory requirements and computational demands. [64]

Techniques such as DeepMind’s Matryoshka quantization enable models to be trained in ways that make them inherently robust to precision reduction, minimizing accuracy losses when deployed at lower bit widths. The practical result has been a proliferation of 4-bit and 8-bit model variants that can run on consumer hardware that would be incapable of supporting their full-precision counterparts. [65]

The efficiency gains from precision optimization interact synergistically with other scaling dimensions. For example, a 70B parameter model quantized to 4-bit precision might occupy about the same memory footprint as a 16-bit 13B model, effectively allowing users to “scale up” their accessible model size through precision reduction rather than hardware upgrades.

Inference-Time Compute Scaling and Recurrent Models

The newest frontier in scaling research focuses on the dynamic allocation of computation at inference time rather than fixing it during training. Studies published in 2024 and 2025 demonstrate that allowing models to perform additional computation steps during inference, such as through recurrent thought processes or search algorithms, can yield performance improvements comparable to training substantially larger models. [66, 59]

This approach parallels human cognition, where additional thinking time often produces better solutions. Techniques including self-consistency decoding (generating multiple solutions and selecting the most consistent) and recurrent processing modules enable models to effectively “think harder” about challenging problems. Commercial implementations like OpenAI o1 or xAI’s Grok 3 with its “Think” and “Big Brain” modes exemplify this approach, demonstrating meaningful improvements in reasoning accuracy through additional compute allocation.

The practical implication is that model selection now requires consideration of inference-time scaling strategies alongside traditional metrics like parameter count. For applications with variable complexity requirements, dynamic compute allocation can provide capabilities otherwise associated with much larger models, particularly for reasoning-intensive tasks.

A.3 Detailed Overview of Accessibility and Cost Considerations

This section provides a more detailed exploration of the considerations surrounding LLM accessibility and cost, including specific tools and platforms that facilitate local and remote execution, expanding upon the summary provided in Section 2.1.3.

Local Execution of LLMs: Control and Privacy

Local execution entails running LLMs directly on your own hardware infrastructure, whether it is personal computers, on-premises servers, or edge devices. This approach is characterized by loading model weights onto local machines and performing all inference computations locally. Local execution offers several compelling benefits [67, 68]:

- **Enhanced Data Privacy and Security:** A primary advantage is the increased control over sensitive data. By processing data within your own environment, organizations can minimize the risks associated with transmitting confidential information to external third-party services, aligning with stringent data governance and compliance requirements.
- **Potentially Lower Long-Term Costs:** Although initial hardware investments might be substantial, local execution can lead to lower variable costs in the long run, particularly for high-volume usage. Eliminating per-query API fees can be economically advantageous, especially if hardware infrastructure is already in place or can be amortized over time.
- **Full Customization and Control:** Local deployment grants users complete control over the LLM environment. This includes the ability to fine-tune models on, for example, proprietary ESG datasets, customize model parameters, and optimize performance according to specific analytical needs.
- **Offline Functionality:** Local models can operate independently of internet connectivity, ensuring continuous operation in environments with limited or unreliable network access.

However, local execution also presents certain challenges.

- **Significant Hardware Requirements:** Running state-of-the-art LLMs, especially larger models, demands substantial computational resources, particularly powerful GPUs with ample VRAM and sufficient RAM. This can lead to a significant upfront investment in specialized hardware.
- **Increased Maintenance and Management Overhead:** Organizations assume responsibility for all aspects of model deployment, including installation, updates, optimization, security, and ongoing maintenance. This requires in-house technical expertise and resources for effective management.
- **Scalability Limitations:** Scaling local deployments to handle fluctuating or increasing workloads can be complex and resource intensive, often requiring significant infrastructure upgrades and careful capacity planning.

Despite these challenges, a rich ecosystem of tools and frameworks has emerged to simplify local LLM deployment, enhancing accessibility, and mitigating some of the technical complexities.

- **Hugging Face Transformers:** The Hugging Face transformers library is a cornerstone for local LLM interaction. It provides a user-friendly interface for downloading pretrained models from the Hugging Face Hub and running them with popular frameworks like LangChain or LlamaIndex [51].
- **Llama.cpp:** This highly optimized C++ library is specifically designed for efficient inference of Llama-family models and similar architectures, primarily on CPUs, with GPU offloading capabilities. Its memory efficiency and minimal dependencies have made it instrumental in enabling LLM execution on resource-constrained devices like laptops and mobile phones, particularly through aggressive quantization techniques like 4-bit quantization. [147, 148]
- **Ollama:** Ollama is an open-source tool designed to simplify running LLMs locally, especially on macOS (Apple Silicon) and Linux systems. It offers a straightforward command-line interface and gRPC API for downloading and serving pre-quantized models from its model library. By abstracting away backend complexities (often leveraging llama.cpp under the hood), Ollama provides a seamless "one-stop" solution for local LLM deployment and interaction. [149, 150, 151]
- **GPT4All:** Developed by Nomic AI, GPT4All focuses on accessibility for running smaller, instruction-tuned LLMs locally, primarily on CPUs. It provides a desktop application with a model catalog for easy selection and offline chat capabilities, along with Python bindings for programmatic model access. GPT4All prioritizes ease of use for personal and small-scale applications, offering pre-packaged models and optimizations for efficient CPU inference. [152, 153, 154]
- **LM Studio:** LM Studio is a cross-platform GUI application and SDK that simplifies the discovery, download, and execution of local LLMs. It provides a user-friendly interface and facilitates connecting local data sources to LLMs, making it particularly suitable for building local question-answering systems and RAG applications over custom document sets. LM Studio abstracts away technical complexities, making local LLM technology accessible to a wider range of users, including non-developers. [155, 156]
- **LangChain:** LangChain is a comprehensive open-source framework designed for building a wide range of LLM-powered applications, from simple chatbots to complex agents. It excels at orchestrating LLM interactions, supporting both local and remote models and

offering a rich ecosystem of components. For local LLM execution, LangChain integrates with models from Hugging Face Transformers and other local pipelines, providing tools for prompt management, chain-of-thought reasoning, and connecting to external data and tools. Crucially, LangChain is a powerful enabler for RAG applications, offering integrations with various embedding libraries, for creating document embeddings, and connecting to local vector databases (like Chroma [157] or FAISS [158]) for efficient retrieval of relevant context. Beyond core functionalities, LangChain's ecosystem includes LangGraph, a library for building robust and stateful conversational agents. LangChain's abstraction layer simplifies switching between different LLM backends (local or remote) and managing complex workflows. [159, 160]

- **LlamaIndex (GPT Index):** LlamaIndex is specialized in connecting LLMs to external data sources, particularly excelling in enabling RAG over local files and knowledge bases. It allows indexing of diverse data formats (documents, databases, APIs) and facilitates querying these indexed sources using LLMs, effectively addressing long-context challenges in local document analysis, such as ESG report processing. LlamaIndex supports integration with local models (through Hugging Face and llama.cpp connectors) and remote APIs, offering a comprehensive solution for building data-aware LLM applications. [161]

In summary, local execution offers enhanced privacy, potential long-term cost savings, and customization, but requires upfront hardware investment and technical expertise. Tools such as those mentioned above significantly improve the accessibility and usability of local LLMs.

Remote Execution of LLMs: Scalability and Convenience

Remote execution refers to using models hosted on cloud platforms or provided "as a service" through APIs. This approach prioritizes ease of use and scalability, outsourcing infrastructure management to third-party providers. The key advantages of remote execution are:

- **Simplified Infrastructure Management:** Cloud providers handle all aspects of model hosting, scaling, updates, and maintenance, eliminating the need for in-house infrastructure management and reducing operational overhead.
- **Access to State-of-the-Art Models:** Remote APIs provide access to the most advanced and computationally intensive LLMs, including models with billions or trillions of parameters that would be impractical to run locally.
- **Scalability and Reliability:** Cloud platforms offer robust infrastructure with automatic scaling and high availability, ensuring reliable performance even in high demand scenarios and fluctuating workloads.

- **Lower Upfront Costs:** Remote execution typically involves pay-as-you-go pricing models, eliminating the need for substantial upfront hardware investments and shifting costs to operational expenses.

However, remote execution also has drawbacks:

- **Variable Usage-Based Costs:** API usage is typically priced per token or compute time, which can become expensive for high-volume applications. Cost predictability can be challenging, especially with fluctuating usage patterns.
- **Data Privacy and Security Concerns:** Sending sensitive data to external API services raises data privacy and security concerns, particularly for organizations operating under strict regulatory frameworks. Although providers offer security measures and data processing agreements, data governance remains a critical consideration.
- **Dependency on Network Connectivity:** Remote execution requires reliable internet connectivity. Performance can be affected by network latency and outages, potentially disrupting operations in areas with poor or unstable internet access.
- **Limited Customization and Control:** Users have less direct control over model configurations, fine-tuning, and underlying infrastructure when relying on remote APIs, potentially limiting customization options for specialized tasks.

Leading cloud providers offer comprehensive platforms for accessing and deploying LLMs remotely:

- **Azure AI Studio (Azure OpenAI Service & AI Foundry):** Microsoft Azure provides the Azure OpenAI Service, offering API access to OpenAI models like o1, GPT-4.5, and embedding models within a secure Azure environment. Azure AI Studio (Foundry) is a unified platform for building, managing, and deploying AI applications. They emphasize the enterprise-grade security, data privacy, and integration with other Azure services. [162, 163]
- **Google Vertex AI:** Google Cloud's Vertex AI platform offers access to Google's PaLM 2 models through the Vertex AI PaLM API (including text generation and embedding models) and supports third-party and open models via its Model Garden. It provides tools for model deployment, prompt design, and integration with other Google Cloud services like BigQuery and Cloud Storage. Vertex AI also offers features for RAG implementation, including embedding support and a matching engine, and conversational AI tuning tools. Google emphasizes data privacy and offers configurations to prevent prompt data from being used for model training beyond request fulfillment. [164, 165]

- **AWS SageMaker and Bedrock:** Amazon Web Services (AWS) offers a dual approach: SageMaker is a comprehensive machine learning service allowing users to deploy custom models or pre-built models from SageMaker JumpStart, including some LLMs, on managed infrastructure. SageMaker provides flexibility for fine-tuning models and deploying them with auto-scaling. Amazon Bedrock is a newer service offering LLMs-as-a-service from various providers, including AI21, Anthropic, Stability AI, and Amazon’s Titan models, through a unified API. [166, 167]

Beyond the comprehensive cloud platforms like Azure AI, Google Vertex AI, and AWS, a diverse ecosystem of API providers offers direct access to cutting-edge Large Language Models. This includes originators like OpenAI, xAI or Anthropic (Claude), offering unique model architectures and capabilities accessible through their respective APIs. To facilitate integration with these remote LLM APIs, various frameworks and tools are available. Provider-specific Software Development Kits (SDKs), like those offered by OpenAI and Cohere (a company known for its enterprise-focused Command models optimized for Retrieval-Augmented Generation [168]), streamline API interaction by handling tasks such as authentication, request formatting, and response processing. Furthermore, orchestration frameworks like LangChain offer a higher level of abstraction, enabling developers to seamlessly interact with multiple LLM providers through a unified interface, manage API keys, and implement complex workflows involving chained prompts and data integration.

Remote execution, facilitated by these diverse API providers and integration tools, presents a compelling approach characterized by scalability, convenience, and access to state-of-the-art models without significant upfront infrastructure investment. However, it is crucial to acknowledge the inherent trade-offs, including variable costs based on usage, potential data privacy implications due to reliance on external services, and dependence on network connectivity. Choosing between different remote execution options and providers requires careful consideration of specific application needs, balancing model capabilities, cost efficiency, data security requirements, and the level of integration flexibility offered by various platforms and frameworks.

Furthermore, access to LLM power is also expanding through AI-enhanced development platforms such as Cursor and Windsurf. These platforms integrate LLM capabilities directly into coding environments, offering features such as code completion and generation powered by AI. Their accessibility lies in their ease of adoption for developers within their existing workflows, often utilizing subscription-based models that indirectly cover the cost of the underlying LLM API services, thus further lowering the barrier to entry to use AI in software development. [169, 170]

A.4 Detailed Overview of Key Prompt Engineering Techniques

This section provides a more detailed exploration of key prompt engineering techniques, expanding on the categories summarized in Section 2.2.1. The following classifications and descriptions are based on recent comprehensive surveys and research studies [74, 75].

Zero-Shot Prompting

Zero-shot prompting involves instructing an LLM to perform a task without providing examples. This technique has evolved from simple instructions to more sophisticated approaches such as:

- **Role Prompting:** Assigns a role (e.g., "You are a sustainability analyst.") to influence behavior, commonly used for task-specific guidance.
- **Style Prompting:** Instructs the model to generate in a specific style (e.g., formal, technical), useful for tailoring outputs.
- **Emotion Prompting:** Incorporating emotional states into prompts (e.g., "You are excited to analyze this sustainability report") has been shown to improve model reasoning on certain tasks.
- **Self-Ask:** Encouraging the model to ask and answer its own clarifying questions, which is particularly effective for complex research queries that require multiple information-seeking steps.
- Other subtechniques are System 2 Attention (S2A), SimToM, Rephrase and Respond (RaR) or Re-reading (RE2).

Few-Shot Techniques

Few-shot prompting requires expressing the desires of what is aimed for by examples before presenting the actual task.

- **Example Generation:** Creating informative examples, crucial for task understanding (Self-Generated In-Context Learning). For example, providing examples of well-structured reports.
- **Example Ordering:** The sequence of examples affects performance; research suggests that ordering from simple to complex can help.

- **Exemplar Selection:** Choosing relevant examples, with techniques like K-Nearest Neighbors (KNN) selecting similar instances, and Vote-K possibly involving voting mechanisms for selection.

Thought Generation

Chain-of-Thought (CoT) prompting encourages LLMs to generate intermediate reasoning steps before providing final answers, as it is already implemented in the most recent models.

- **Zero-Shot CoT:** Simply adding "Let us think step by step" before tackling complex problems has demonstrated significant performance improvements across reasoning tasks.
 - Sub-techniques include Analogical Prompting (using analogies), Step-Back Prompting (taking a broader view), Thread-of-Thought (maintaining a thread of reasoning), or Tab-CoT (Tabular Chain-of-Thought (for table-based inputs)).
- **Few-Shot CoT:** Uses examples demonstrating reasoning, e.g. showing how to analyze governance step by step.
 - Sub-techniques include Active-Prompt, Auto-CoT, Complexity-Based, Contrastive, Memory-of-Thought, Uncertainty-Routed CoT, or Prompt Mining enhance CoT by tailoring prompts based on task complexity or model uncertainty.

Ensembling

Ensembling combines multiple prompts or outputs for better results, reducing variability.

- **Self-Consistency:** Generating multiple reasoning paths for the same problem and taking the most consistent answer, e.g. "Generate three ESG profiles for Company X and select the most consistent one based on key metrics."
- Sub-techniques include COSP, DENSE, DiVeRSe, Max Mutual Information, Meta-CoT, MoRE, Prompt Paraphrasing, or USP.

Self-Criticism

Self-criticism methods prompt the model to evaluate and refine its own responses, enhancing reliability.

- **Chain-of-Verification (CoVe):** The model generates a response and then independently verifies each claim or step. This would be crucial for ESG reporting, where factual accuracy is paramount.

- **Self-Refine:** A multi-turn approach where the model iteratively improves its own outputs based on self-critique.
- Sub-techniques include Self-Calibration that adjusts confidence, Self-Verification checks correctness, ReverseCoT reverses reasoning for verification, or Cumulative Reasoning builds up evidence.

Decomposition

Decomposition breaks complex problems into manageable subtasks, handling complexity effectively.

- **Tree-of-Thought (ToT):** Extends CoT by exploring multiple reasoning branches concurrently. The model considers different approaches to a problem, evaluates them, and pursues the most promising paths. For example, "First, assess Company X's environmental policies, then its social initiatives, and finally governance, combining into a profile".
- **Least-to-Most Prompting:** Solves problems incrementally by tackling simple components before addressing more complex aspects.
- **Plan-and-Solve:** Explicitly prompts the model to create a plan before execution. For example, "First plan how to assess social impact, then solve".
- **Implicit RAG:** Asks the model to retrieve the important sections from the given context to afterwards answer the queries based on these chunks.
- Sub-techniques include Faithful CoT ensures reasoning aligns with tasks, Program-of-Thought uses programming-like steps, Recursive-of-Thought applies recursion or Skeleton-of-Thought.

A.5 Detailed Techniques for Managing Long Contexts in LLMs

This section provides a more detailed exploration of the innovative approaches developed by researchers to address the computational challenges of long context windows in LLMs, expanding on the categories summarized in Section [2.3.1](#).

Optimized Memory Techniques

Modern LLMs employ sophisticated memory management strategies to handle long contexts more efficiently. These include, among others: [81]

- **FlashAttention:** Reduces GPU memory usage by recomputing attention scores on-the-fly rather than storing them, enabling efficient processing of long sequences. [93, 82]
- **Activation checkpointing:** Trades computation for memory by selectively recomputing activations during backpropagation.
- **Temporary buffers:** Optimizes memory allocation for intermediate computations.
- **Memory Fragmentation:** Implements strategies to efficiently allocate and reuse GPU memory, reducing fragmentation by minimizing gaps and enhancing contiguous memory utilization. This optimization ensures better memory occupancy and improved computational throughput for large-scale models.
- **Quantization:** Employs numerical precision reduction techniques, including symmetric and asymmetric linear quantization (e.g., `LLM.int8()`), to significantly reduce model size and memory consumption without substantial loss in model accuracy, facilitating efficient deployment and inference on resource-constrained hardware.
- **Gradient Accumulation:** Allows training on larger batch sizes by accumulating gradients over multiple forward passes.
- **Parallelization:** Distributes attention computations across multiple GPUs. [79]

Linear and Sparse Attention Mechanisms

Several research efforts have focused on developing attention variants that scale linearly rather than quadratically with sequence length. Models like Transformer-XL or Longformer implement efficient attention patterns that approximate full attention while substantially reducing computational requirements. [83]

To further enhance efficiency, sparse-attention mechanisms selectively focus on the most relevant token relationships rather than computing all possible interactions. Techniques such as block-sparse attention, sliding-window attention, and dilated attention have proven effective in managing long documents while preserving performance. [84]

Architectural Innovations

As discussed in Section 2.1.2 (*Scaling Laws and Efficiency*), Mixture-of-Experts (MoE) architectures represent one of the most significant recent advancements. These architectures dynamically distribute computations across specialized sub-networks, selectively activating them based on input-specific characteristics. This design enables efficient management of extended contexts by routing different segments of a document to the most suitable expert networks. Models like DeepSeek-R1 exemplify this approach, activating only a subset of parameters per token, thus substantially enhancing computational efficiency for long-context tasks. [85]

Another significant advancement is hierarchical attention architectures, which organize tokens into multi-level structures, enabling models to efficiently capture both local and global relationships. This approach breaks documents into chunks, applying attention locally within segments before aggregating global context. [86]

To handle extremely long contexts, models have implemented enhanced positional encoding schemes such as rotary position embeddings (RoPE) and relative positional encoding that maintain position awareness across extended sequences. [87]

A.6 Detailed Strategies for Mitigating LLM Hallucinations

This section provides a more detailed exploration of the various strategies developed to mitigate hallucinations in LLMs, expanding on the categories summarized in Section 2.4.2. These approaches are drawn from recent research [98, 99, 100, 101, 97].

Prompting and Instructions

As discussed in Section 2.2 (*Prompt Engineering*), carefully crafted prompts and instructions can significantly influence LLM output and reduce the likelihood of fabrications [102]. Tactics such as explicitly instructing the model to avoid guessing and to strictly utilize the provided context can be effective.

Furthermore, employing verification prompts, where the model is asked to review and check its own answer as a follow-up step, can also help in detecting and correcting falsehoods. This iterative approach encourages the model to critically evaluate its initial response, potentially identifying inconsistencies or invented details in a second pass.

Retrieval-Augmented Generation

RAG stands out as a highly effective technique for minimizing hallucinations in many applications. RAG methodologies ground the LLM generation process in real verifiable data by retrieving relevant documents or factual snippets from an external knowledge source and incorporating them into the prompt. This approach significantly reduces the model's reliance on its parametric knowledge, which is often the source of hallucinations. For example, instead of directly asking an LLM "What are the ESG goals of Company Z?", an RAG approach would first perform a search within curated data of Company Z's official statements, sustainability reports and related documents. This method compels the model to base its response on the provided contextually relevant text snippets, enabling it to quote directly from the sources or synthesize information grounded in known facts rather than inventing details.

Although RAG significantly reduces hallucinations, it is not always foolproof, and a secondary review of the AI-generated answer remains a prudent practice to ensure accuracy and completeness. For these reasons, RAG is widely adopted in enterprise LLM deployments, particularly in applications that require factual precision and traceability, transforming the LLM from a purely generative model into a more reliable system for intelligent information retrieval and explanation.

Fine-Tuning and Domain Adaptation

Fine-tuning the LLM on domain-specific knowledge or with examples of accurate, source-backed outputs represents another strategy for hallucination mitigation [99]. If a substantial dataset of a specific field or question-answer pairs were available, an LLM could be fine-tuned to prioritize reliance on specific types of information and to generate responses aligned with domain-specific conventions. Fine-tuning can also be designed to train the model to explicitly indicate uncertainty when information is lacking or to consistently verify its responses against a trusted knowledge base. Research studies also explore fine-tuning models to internalize a factual corpus, differentiating between "open-book" question answering, where models can access external knowledge, and "closed-book" question answering, where they must rely solely on their internal knowledge. However, fine-tuning large LLMs is a resource-intensive process, both computationally and in terms of data preparation. Furthermore, if the domain knowledge is dynamic frequent retraining would be necessary to maintain accuracy, which can be impractical.

There is also a risk that aggressive fine-tuning, if not carefully managed, could potentially degrade the model's general capabilities or even introduce new types of hallucinations by overfitting to specific patterns in the training data, making the model less versatile than its pre-tuned state. A more lightweight approach is in-context learning, employing few-shot prompting with

examples that consistently reference sources and avoid speculative answers. By demonstrating desired behavior through carefully selected examples, in-context learning can subtly bias the model towards more reliable and factually grounded outputs without the need for extensive fine-tuning. Optimal methods for effectively leveraging fine-tuning and in-context learning to minimize hallucinations are still areas of active research.

Tools and External Checks

Integrating external tools for automated verification offers a promising approach to detect and mitigate hallucinations. In this setup, after an LLM generates a response, the system can automatically cross-reference statements from the answer against trusted sources to validate their accuracy. If discrepancies are found, it could indicate a hallucination and the answer could be flagged as potentially unreliable, or the system could prompt the model to revise its response. This two-stage process, generating and then verifying, mirrors the diligence of a human analyst who would naturally cross-check AI-generated claims. One could even program a separate LLM specifically for this verification task, where it assesses the primary LLM's answers and source citations. Such techniques, sometimes referred to as self-verification or knowledge retrieval and verification, are under active development to enhance the reliability of LLM output. Automating this cross-checking process is crucial for scaling up the use of LLMs in applications requiring high factual integrity.

Controlled Generation and Calibration

Experimental approaches focused on modifying the LLM generation process itself aim to intrinsically reduce hallucinations. Calibration, in this context, refers to improving a model's ability to accurately express its confidence level or uncertainty. Ideally, an LLM model should only confidently answer when it possesses a high degree of certainty and express uncertainty or abstain from answering when unsure. Techniques such as Stochastic Beam Search and variance reduction methods are explored to refine the generation process, with the aim of preventing scenarios where random token selections lead the model down a path of hallucination [96]. In addition, constrained decoding techniques can be implemented to impose restrictions on the model's output, preventing specific types of hallucinations. For instance, the model could be limited to generating numerical values that are not explicitly present in the input context, thereby preventing the fabrication of statistics or metrics. Implementing constrained decoding often requires custom logic within the decoding process and is not typically a standard feature in commercial APIs, but it remains an active area of research for enhancing factual control over LLM output.

A.7 Detailed Overview of RAG Archetypes

This section provides a more detailed exploration of the various specialized Retrieval-Augmented Generation (RAG) approaches that have emerged, expanding upon the categories summarized in Section 2.5.3. These descriptions are based on recent surveys and research in the field [121, 105].

Naïve Retrieval Augmented Generation

Naïve RAG represents the foundational and most straightforward implementation of retrieval augmentation. It follows the basic pipeline described earlier: a query is embedded, relevant documents are retrieved from a vector database, and these documents are concatenated with the original query to form a prompt for the LLM. The LLM then generates an answer based on this augmented prompt.

Although effective as a baseline, Naïve RAG is characterized by its simplicity and ease of implementation, making it a common starting point for many RAG applications. However, it often lacks advanced techniques to optimize retrieval relevance, prompt construction, or answer refinement, potentially limiting its performance in complex scenarios.

Advanced RAG

Advanced RAG techniques are built on the basic framework of Naïve RAG by incorporating more sophisticated methods to improve retrieval quality, context utilization, and generation effectiveness. These advancements might include:

- **Query Optimization:** Techniques for reformulating or expanding the initial query to better match relevant documents in the knowledge repository. This might involve query expansion using synonyms, query rewriting based on historical search data, or query decomposition for complex multi-intent queries.
- **Chunking and Indexing Strategies:** More intelligent methods for segmenting and indexing documents beyond simple fixed-size chunking. Semantic chunking, for example, aims to create chunks that are semantically self-contained, improving context coherence and retrieval relevance. Metadata indexing and hierarchical indexing can also be used to enhance retrieval precision and efficiency.
- **Re-ranking and Filtering:** Steps to re-rank the initially retrieved documents based on more sophisticated relevance criteria, often using smaller models or even the LLM itself

to assess the actual relevance of snippets to the query. Filtering techniques can also be applied to remove irrelevant or noisy retrieved documents prior to prompt assembly.

- **Prompt Engineering Enhancements:** More elaborate prompt templates and strategies are needed to better instruct the LLM on how to use the retrieved context effectively. This can include techniques like chain-of-thought prompting within the RAG framework or context compression to fit more relevant information within the context window.

Modular RAG

Modular RAG architectures emphasize flexibility and dynamic configuration by breaking the RAG pipeline into distinct, interchangeable modules. This modularity allows developers to tailor each component of the RAG system to specific needs and to easily swap out or upgrade individual modules without overhauling the entire system. Typical modules in a Modular RAG system might include:

- **Multiple Retrievers:** Employ different retrieval mechanisms (e.g., vector search, keyword search, graph traversal) in parallel and combine their results to improve recall and robustness.
- **Context Enrichment Modules:** Components that further process retrieved contexts to extract key information, summarize, or reformat them before feeding them to the LLM, optimizing context utilization, and reducing noise.
- **Generation Strategy Modules:** Different generation approaches that can be dynamically selected based on the query or the retrieved context. This might include different prompting techniques, decoding strategies, or even the use of multiple LLMs for different generation tasks.
- **Output Processing Modules:** Modules for post-processing the LLM's output, such as citation augmentation, answer verification, or response formatting for specific applications.

Adaptive RAG

Adaptive RAG systems are designed to dynamically adjust their retrieval and generation strategies based on the characteristics of the input query and the retrieved information. These systems incorporate mechanisms to analyze the query complexity, the quality and relevance of retrieved documents, and the LLM's confidence in its generated answer.

- **Adjust Retrieval Depth:** Retrieve more or fewer documents based on the query's ambiguity or the initial retrieval results' quality. For simple queries with high-confidence retrievals, fewer documents might be sufficient, while complex or ambiguous queries might necessitate retrieving a larger context.
- **Dynamically Modify Prompts:** Alter the prompt structure or instructions based on the retrieved context. If the retrieved documents are highly relevant and directly answer the query, the prompt might be simplified to focus on the extraction of direct answers. If the retrieved context is less direct, the prompt might be adjusted to guide more inferential reasoning or synthesis.
- **Engage in Iterative Retrieval:** If the initial retrieval results are unsatisfactory, the system can iteratively refine the query or adjust the retrieval parameters to seek more relevant information.

Corrective RAG

Corrective RAG focuses specifically on enhancing the factual accuracy and reliability of RAG systems by incorporating mechanisms for error detection and correction. These systems aim to actively identify and rectify potential inaccuracies or hallucinations in the generated answers. Corrective RAG strategies include the following.

- **Fact Verification Modules:** Integrating external fact-checking tools or models to verify the factual claims made in the LLM's generated response against trusted knowledge sources.
- **Self-Correction Loops:** Implement feedback loops in which the RAG system evaluates its own generated responses for potential errors and iteratively refines the response to improve accuracy. This can involve prompting the LLM to self-critique its answer or using a separate verification model to assess the output.
- **Uncertainty Estimation:** Developing methods to estimate the LLM's confidence or uncertainty in its answer and flagging responses with low confidence for human review or further verification.

Self-RAG

Self-RAG innovates by training the generative model itself to decide when and what to retrieve, and how to integrate retrieved information into its generation process. In Self-RAG, the LLM

is not just a generator but also learns to act as its own retriever and evaluator. Key aspects of Self-RAG include:

- **Retrieval Tokens:** The model is trained to generate special "retrieval tokens" during the generation process, signaling when it needs to retrieve external information.
- **Learnable Retrieval Mechanism:** The retrieval process is integrated into the model's training, allowing it to learn optimal retrieval strategies and to adapt its retrieval behavior based on the input and the task.
- **Reflection Tokens:** Self-RAG models also generate "reflection tokens" that allow them to assess the quality and relevance of the retrieved documents and to control how this information is incorporated into the final output. This enables the model to be more discerning about the information it uses and to potentially filter out less reliable or relevant content.

Graph RAG

Graph RAG, as briefly mentioned earlier, utilizes knowledge graphs as the primary knowledge source for retrieval. Instead of indexing and retrieving text chunks, Graph RAG operates on structured knowledge represented as entities and relationships within a graph. Key features of Graph RAG include:

- **Graph Traversal for Retrieval:** Retrieval is performed by traversing paths in the knowledge graph, starting from entities related to the user query, and exploring connected entities and relationships to find relevant information.
- **Structured Context Representation:** The retrieved context is represented as sub-graphs or paths from the knowledge graph, providing structured and semantically rich information to the LLM.
- **Reasoning over Relationships:** Graph RAG excels at answering complex queries that require reasoning over relationships between entities, leveraging the structured nature of the knowledge graph to infer and synthesize information.

Hybrid RAG

Hybrid RAG architectures combine different retrieval methods and knowledge sources to leverage their complementary strengths and improve overall retrieval effectiveness. Common hybrid approaches include:

- **Vector and Keyword Search Fusion:** Combine vector-based semantic search with traditional keyword-based search (e.g., BM25) to improve recall and precision. Vector search captures semantic similarity, while keyword search ensures that relevant terms are matched, leading to a more comprehensive retrieval.
- **Knowledge Graph and Text Retrieval Integration:** Combining retrieval from unstructured text with structured knowledge from knowledge graphs. This allows the RAG system to leverage both factual knowledge and relational information for more contextually rich retrieval.
- **Multiple Vector Database Integration:** Query multiple vector databases, each potentially indexing different types of document or using different embedding models, and aggregating the results to broaden the scope of retrieval and improve coverage.

Agentic RAG

Agentic RAG extends its capabilities by integrating LLMs with agent-like functionalities, enabling more complex and interactive information seeking and processing [122]. Agentic RAG systems can:

- **Plan and Execute Retrieval Strategies:** Develop complex retrieval plans, breaking down complex queries into sub-queries, and iteratively refining search strategies to gather comprehensive information.
- **Interact with External Tools and APIs:** Integrate with external tools and APIs to access and process information beyond the pre-indexed knowledge repository. This might include a web search, database queries, or API calls to external data sources.
- **Engage in Dialogue and Clarification:** Engage in interactive dialogues with users to clarify ambiguous queries or to gather additional information needed for effective retrieval and answer generation.
- **Reason over Retrieved Information:** Employ more sophisticated reasoning processes over the retrieved information, going beyond simple concatenation to perform synthesis, inference, and multi-hop reasoning.

A.8 Detailed Metrics and Frameworks for RAG System Evaluation

This section provides a more detailed exploration of key evaluation metrics and practical approaches to measuring the performance of RAG systems, expanding on the summary presented in Section 2.5.4.

Assessing the Effectiveness of RAG

The effectiveness evaluation for RAG systems is focused on how well the system leverages external knowledge to generate reliable responses. This involves evaluating three core aspects: the relevance of the retrieved context to the query, the coherence of the generated answer with the retrieved context, and the overall accuracy of the answer in relation to the query.

Assessing Query-Context Relevance

The cornerstone of an effective RAG system is its ability to retrieve context that is truly relevant to the user's query. The relevant context provides the necessary foundation for LLM to reason and generate accurate and informed responses. Frameworks like ARES, RAGAS, and TruLens utilize Context Relevance metrics to evaluate the alignment between the retrieved context and the user's query. The KILT benchmark, designed for knowledge-intensive tasks, offers metrics such as R precision and Recall@k to evaluate the quality of retrieved context, focusing on its relevance and comprehensiveness in relation to the original query. The precision R measures the precision at rank R, where R is the number of relevant documents for a given query, while the recall@k measures the proportion of relevant documents retrieved within the top k results. [105]

Evaluating Context-Answer Coherence

Beyond retrieving the relevant context, an RAG system must ensure that the generated answer is coherent with and faithful to the retrieved information. This evaluation aspect focuses on minimizing hallucinations and ensuring that the LLM's output is grounded in the provided context [105]. Benchmarks and metrics designed to assess context-answer coherence include:

- **Answer Faithfulness:** Frameworks like ARES and RAGAS introduce answer-faithfulness metrics, which quantify the degree to which the generated answer is supported by and consistent with the retrieved context passages. High faithfulness scores indicate that the LLM effectively uses the retrieved context as its primary source of information and minimizes fabrication.

- **Recall and Precision (CRUD):** The CRUD benchmark employs traditional recall and precision metrics adapted to assess the accuracy and completeness of the answers generated in relation to the retrieved context. High recall indicates that the answer covers most of the relevant information in the context, while high precision suggests that the information in the answer is accurate and present in the context.
- **Accuracy and Misleading Rate (RECALL):** RECALL proposes accuracy, measuring the factual correctness of the generated answer, and misleading rate (M-Rate), evaluating the frequency of misleading or unsupported information in the output. Minimizing the misleading rate is crucial to building reliable RAG systems.
- **Groundedness (TruLens):** TruLens emphasizes groundedness, a metric that involves decomposing the LLM's answer into individual claims and verifying each claim against the retrieved context. This granular approach ensures that every component of the answer is directly supported by evidence from the retrieved documents.
- **Citation-Focused Metrics:** For RAG systems designed to provide citations, metrics such as the Correct Attribution Score (CAS) and Citation Redundancy Score (CRS) from the RECLAIM framework are essential. CAS assesses whether the responses are fully supported by provided citations, while CRS minimizes unnecessary or redundant citations, improving the clarity of response. LongBench-Cite further evaluates the accuracy of the citation in answering long-context questions. ALCE uses citation recall and precision to assess citation relevance and conciseness.

Measuring Query-Answer Accuracy

The accuracy of the generated answer in addressing the user's query is a primary concern for the evaluation of the RAG system. Metrics for query-answer accuracy in RAG include: [105]

- **Hallucination Rate and Missing Rate (CRAG):** The CRAG benchmark incorporates the hallucination rate, measuring the frequency of fabricated content, and missing rate, assessing the extent to which relevant information from the context is omitted in the answer. CRAG penalizes hallucinations by calculating the difference between accuracy and hallucination rate, emphasizing factual correctness.
- **Answer Relevance (RAGAS, ARES, TruLens):** Frameworks like RAGAS, ARES, and TruLens include Answer Relevance metrics, which evaluate how well the generated response directly and completely addresses the original user query. High answer relevance scores indicate that the RAG system is effectively answering the user's question without straying into tangential or irrelevant topics.

- **Answer Relevance and Correctness (CRUD/RAGQuestEval):** CRUD introduces RAGQuestEval to jointly assess answer relevance and correctness, providing a comprehensive measure of answer quality in the context of the query.
- **Task-Oriented Evaluation (KILT):** The KILT benchmark evaluates RAG systems based on performance in specific downstream tasks, offering a task-centric perspective on answer accuracy. This approach assesses how well the RAG system performs in real-world applications.
- **Error Handling Metrics (RGB):** The RGB framework extends the evaluation to error handling capabilities, introducing the rejection rate, error detection rate, and error correction rate. These metrics test the system's ability to identify and reject or correct incorrect or misleading information, reflecting robustness in handling uncertainty and potential errors.

Evaluating the Efficiency of RAG

Efficiency evaluation is as critical as effectiveness for deploying RAG systems in real-world applications. Key efficiency metrics include: [\[105\]](#)

- **Latency:** Measures the end-to-end response time, from query input to answer output, encompassing both retrieval and generation phases. Lower latency is crucial for interactive applications.
- **Throughput:** Indicates the number of queries the system can process per unit of time. Higher throughput is essential for handling high-volume workloads.
- **Resource Utilization:** Reflects the computational resources consumed by the system, including CPU/GPU usage and memory consumption. Efficient utilization of resources minimizes operational costs and maximizes scalability. In addition, profiling tools like cProfile or memory_profiler are able to identify performance bottlenecks in the RAG pipeline and measure resource consumption at each stage (retrieval, generation).